RTL Design Sherpa

RTL AMBA AXI4 Module Reference — Rev 0.90

July 2026

Table of Contents

1 AXI4 Clock-Gated Variants Guide

Location: `rtl/amba/axi4/*_cg.sv` **Status:** √ Production Ready

1.1 Overview

All AXI4 modules in this subsystem have clock-gated (_cg) variants that add power management capabilities through dynamic clock gating. These modules automatically gate the clock when interfaces are idle for a configurable period, reducing dynamic power consumption in low-activity scenarios.

1.1.1 Available Clock-Gated Modules

Module	Base Module	Description
axi4_master_rd_cg	axi4_master_rd	Clock-gated read master
axi4_master_wr_cg	axi4_master_wr	Clock-gated write master
axi4_slave_rd_cg	axi4_slave_rd	Clock-gated read slave
axi4_slave_wr_cg	axi4_slave_wr	Clock-gated write slave
axi4_master_rd_mon_cg	axi4_master_rd_mon	Clock-gated read master with monitoring
axi4_master_wr_mon_cg	axi4_master_wr_mon	Clock-gated write master with monitoring
axi4_slave_rd_mon_cg	axi4_slave_rd_mon	Clock-gated read slave with monitoring
axi4_slave_wr_mon_cg	axi4_slave_wr_mon	Clock-gated write slave with monitoring

1.1.2 Key Features

- √ **Dynamic Clock Gating:** Automatic clock disable during idle periods
- √ **Configurable Idle Threshold:** Programmable idle count before gating
- √ **Full Functional Equivalence:** Identical functionality to base modules
- √ **Status Monitoring:** Real-time gating status and cumulative clock count
- √ **Test Mode Support:** Bypass capability for scan testing
- √ **Zero Performance Impact:** Immediate ungating on activity

1.2 Additional Parameters (All _cg Modules)

Parameter	Type	Default	Description
`CG_IDLE_COUNT_WIDTH`	int	4	Width of idle counter (max idle = 2^N-1 cycles)

All other parameters are identical to the base module.

1.3 Additional Ports (All _cg Modules)

1.3.1 Clock Gating Configuration Inputs

Port	Direction	Width	Description
`cfg_cg_enable`	Input	1	Enable clock gating (0=disabled, 1=enabled)
`cfg_cg_idle_count`	Input	CG_IDLE_COUNT_WIDTH	Idle cycles before gating (0=immediate gating)

1.3.2 Clock Gating Status Outputs

Port	Direction	Width	Description
`cg_gating`	Output	1	Clock currently gated (1=gated, 0=running)
`cg_clk_count`	Output	32	Cumulative gated clock cycles (power metric)

All other ports are identical to the base module.

1.4 Clock Gating Behavior

1.4.1 Gating Conditions (All Must Be True)

1. **Interface Idle:** No valid/ready handshakes on any AXI channel
2. **Idle Duration:** Idle for ≥ `cfg_cg_idle_count` consecutive cycles
3. **Gating Enabled:** `cfg_cg_enable = 1`

1.4.2 Ungating Conditions (Any Triggers Immediate Ungating)

1. Any `*valid` signal asserted on any channel

2. Configuration change (`cfg_cg_enable` or `cfg_cg_idle_count`)

Ungating Latency: 0 cycles (immediate)

```
stateDiagram-v2
    [*] --> RUNNING

    RUNNING --> IDLE : activity = 0<br/>(no valid signals)
    IDLE --> RUNNING : idle_count < threshold
    IDLE --> GATED : count >= threshold<br/>&& cg_enable
    GATED --> RUNNING : activity detected<br/>(any valid signal)

    state RUNNING {
        note right of RUNNING : Clock running normally
    }

    state IDLE {
        note right of IDLE : Counting idle cycles
    }

    state GATED {
        note right of GATED : Clock stopped
    }
```

1.5 Usage Examples

1.5.1 Basic Clock Gating (Master Read)

```
axi4_master_rd_cg #(
    // Base parameters
    .SKID_DEPTH_AR      (2),
    .SKID_DEPTH_R       (4),
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_DATA_WIDTH     (64),

    // Clock gating parameters
    .CG_IDLE_COUNT_WIDTH(4)   // Up to 15 idle cycles
) u_rd_master_cg (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Clock gating configuration
    .cfg_cg_enable      (1'b1),      // Enable gating
    .cfg_cg_idle_count  (4'd5),      // Gate after 5 idle cycles
```

```
    // AXI signals (identical to base module)
    .fub_axi_arid       (cpu_arid),
    .fub_axi_araddr     (cpu_araddr),
    // ... rest of AR/R signals ...

    .m_axi_arid         (mem_arid),
    .m_axi_araddr       (mem_araddr),
    // ... rest of AR/R signals ...

    // Clock gating status
    .cg_gating          (rd_clk_gated),
    .cg_clk_count       (rd_gated_cycles)
);

// Monitor power savings
always_ff @(posedge axi_clk) begin
    if (rd_clk_gated) begin
        $display("Read master clock gated - saving power");
    end
end
```

1.5.2 Dynamic Configuration (Runtime Adjustment)

```
// Power management controller
logic [3:0] idle_threshold;

always_comb begin
    case (power_mode)
        POWER_HIGH_PERF:  idle_threshold = 4'd15;  // Conservative
gating
        POWER_BALANCED:   idle_threshold = 4'd5;   // Moderate gating
        POWER_LOW:        idle_threshold = 4'd1;   // Aggressive
gating
        default:          idle_threshold = 4'd5;
    endcase
end

axi4_master_wr_cg #(
    // ... parameters ...
) u_wr_master_cg (
    // ... ports ...

    // Dynamic clock gating control
    .cfg_cg_enable      (power_mgmt_enable),
    .cfg_cg_idle_count  (idle_threshold),
```

```
    .cg_gating          (wr_clk_gated),
    .cg_clk_count       (wr_gated_cycles)
);
```

1.5.3 System-Level Power Monitoring

```
// Aggregate power metrics from all interfaces
wire master_rd_gated, master_wr_gated;
wire slave_rd_gated, slave_wr_gated;

wire [31:0] master_rd_gated_cycles;
wire [31:0] master_wr_gated_cycles;
wire [31:0] slave_rd_gated_cycles;
wire [31:0] slave_wr_gated_cycles;

// Total gated cycles (power savings metric)
wire [31:0] total_gated_cycles = master_rd_gated_cycles +
                                 master_wr_gated_cycles +
                                 slave_rd_gated_cycles +
                                 slave_wr_gated_cycles;

// Instantaneous gating status
wire any_gating = master_rd_gated | master_wr_gated |
                  slave_rd_gated | slave_wr_gated;

// Power savings estimate (assuming 50% power reduction when gated)
wire [31:0] power_savings_percent = (total_gated_cycles * 50) /
total_cycles;
```

1.6 Configuration Guidelines

1.6.1 Idle Count Selection

Aggressive Gating (High Idle Time):

```
.cfg_cg_idle_count(4'd1)   // Gate after 1 idle cycle
// Use when: Burst traffic, low duty cycle, max power savings
```

Moderate Gating (Balanced):

```
.cfg_cg_idle_count(4'd5)   // Gate after 5 idle cycles
// Use when: Mixed traffic, balanced power/performance
```

Conservative Gating (Low Latency):

```
.cfg_cg_idle_count(4'd10)  // Gate after 10 idle cycles
// Use when: High throughput, latency-sensitive, minimal power
overhead
```

Disable Gating:

```
.cfg_cg_enable(1'b0)       // Clock gating disabled
// Use when: 100% utilization, ultra-low latency, debug/test
```

1.6.2 When to Use Clock Gating

√ **Recommended For:** - Burst traffic patterns (idle periods between bursts) - Low-duty-cycle interfaces (< 50% utilization) - Power-constrained systems (battery, thermal limits) - Variable workloads (idle states common)

✗ **Not Recommended For:** - 100% utilization scenarios (no idle time = no power benefit) - Ultra-low-latency requirements (gating overhead unacceptable) - Continuous streaming (no natural idle points)

1.6.3 Traffic Pattern Analysis

Burst Traffic (Good for Clock Gating):

```
Time:    |----BURST----|idle|----BURST----|idle|----BURST----|
Gating:  |             |GG|               |GG|               |
Savings: ~30-40% depending on burst duty cycle
```

Continuous Traffic (Poor for Clock Gating):

```
Time:    |----DATA----DATA----DATA----DATA----DATA----|
Gating:  |                                            |
Savings: ~0% (never idle long enough to gate)
```

1.7 Power Savings Estimates

1.7.1 Typical Power Savings

Traffic Pattern	Duty Cycle	Idle Count	Power Savings
Burst (aggressive)	30%	1	35-40%
Burst (moderate)	30%	5	30-35%
Burst (conservative)	30%	10	25-30%

Traffic Pattern	Duty Cycle	Idle Count	Power Savings
Mixed workload	50%	5	20-25%
Low activity	10%	1	40-45%
High activity	80%	5	5-10%
Continuous	100%	any	0%

Note: Actual savings depend on: - Process technology (clock tree power) - Traffic patterns (idle duration distribution) - Configuration (idle count threshold) - Logic behind clock gate (amount of logic gated)

1.7.2 Measuring Power Savings

```
// Calculate power savings percentage
logic [31:0] total_cycles;
logic [31:0] gated_cycles;

always_ff @(posedge axi_clk or negedge aresetn) begin
    if (!aresetn) begin
        total_cycles <= '0;
        gated_cycles <= '0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (cg_gating) begin
            gated_cycles <= gated_cycles + 1;
        end
    end
end

// Power savings = (gated_cycles / total_cycles) × 100%
assign power_savings_pct = (gated_cycles * 100) / total_cycles;
```

1.8 Design Notes

1.8.1 Test Mode Support

For scan testing, disable clock gating:

```
.cfg_cg_enable(~scan_mode)  // Disable during DFT scan
```

1.8.2 Simulation Considerations

Waveform Analysis: - `cg_gating=1`: Clock gated (idle) - `cg_gating=0`: Clock running (active)

Power Estimation:

```
initial begin
    $monitor("Time %0t: Gating=%b, GatedCycles=%0d",
             $time, cg_gating, cg_clk_count);
end
```

1.8.3 Synthesis Considerations

Clock Gating Cell: - Uses standard cell library clock gate (GTECH_AND, ICG, etc.) - Synthesis tool automatically inserts appropriate cell - No vendor-specific primitives required

Timing: - Clock gating logic adds minimal delay - Ungating path is critical (0-cycle requirement) - Gating path is non-critical

---

1.9 Performance Impact

1.9.1 Latency Analysis

Ungating Latency: 0 cycles (immediate)

```
Cycle N:   Interface idle, clock gated
Cycle N+1: ARVALID asserted → clock immediately ungated
Cycle N+1: Transaction proceeds normally
```

No Performance Penalty: - Clock ungates same cycle as activity - AXI handshake proceeds without delay - Functionally equivalent to non-gated module

1.9.2 Throughput

Sustained Throughput: Identical to base module - Gating only occurs during idle - Active transactions unaffected

---

1.10 Related Documentation

1.10.1 Base Module Documentation

- **axi4_master_rd** - Base read master
- **axi4_master_wr** - Base write master
- **axi4_slave_rd** - Base read slave
- **axi4_slave_wr** - Base write slave
- **axi4_master_rd_mon** - Base master read monitor

- **axi4_master_wr_mon** - Base master write monitor
- **axi4_slave_rd_mon** - Base slave read monitor
- **axi4_slave_wr_mon** - Base slave write monitor

1.10.2 Architecture

- **RTLAmba Overview** - Complete AMBA subsystem
- **AXI4 Index** - AXI4 module index

Last Updated: 2025-10-20

1.11 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

2 AXI4 Data Width Converter (Bidirectional)

Module: `axi4_dwidth_converter.sv` **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

2.1 Overview

The AXI4 Data Width Converter provides bidirectional data width conversion between AXI4 interfaces of different data widths while maintaining full protocol compliance. A single parameterized module handles both upsizing (narrow→wide) and downsizing (wide→narrow) operations, automatically recalculating burst parameters and managing data packing/unpacking.

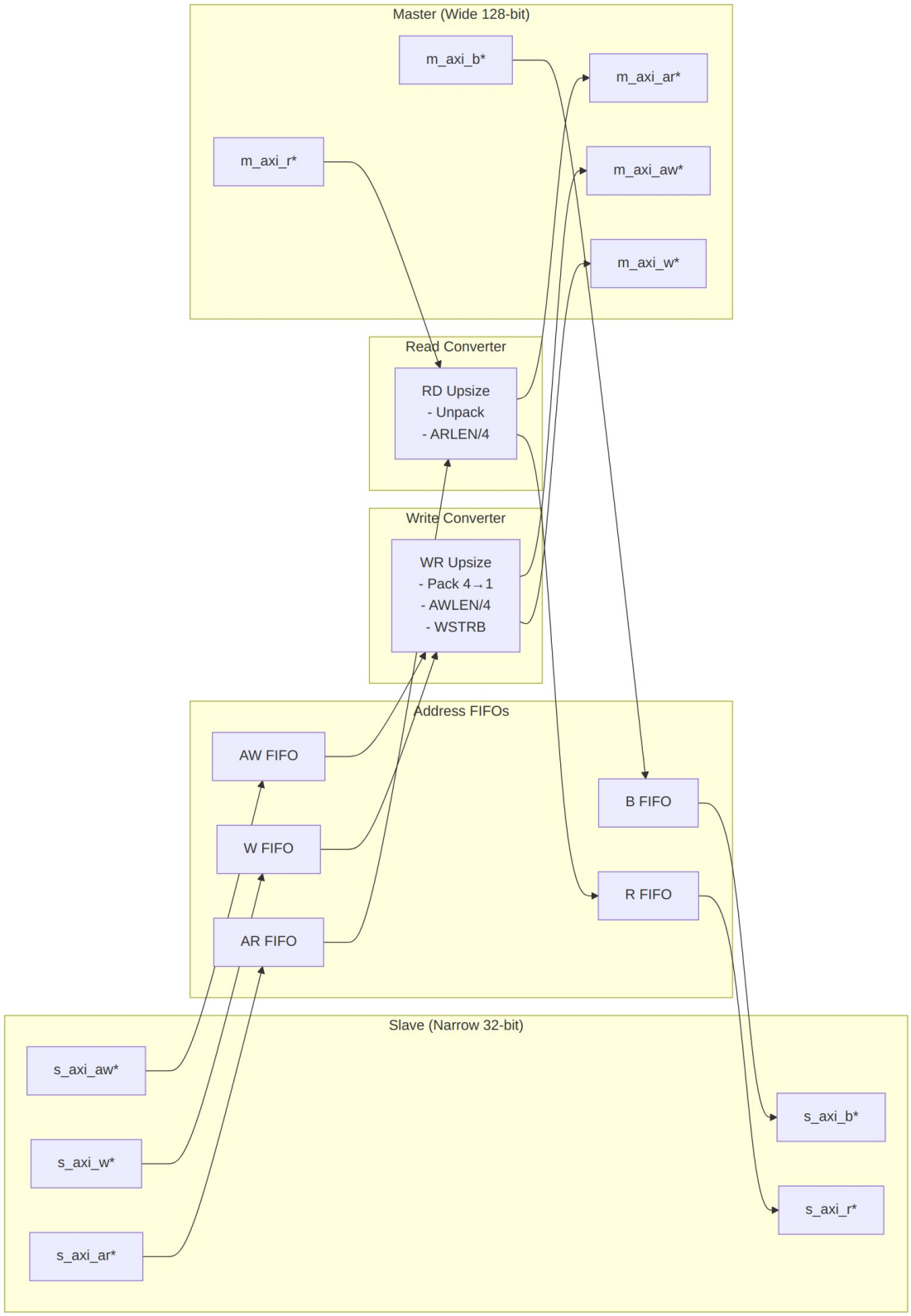
2.1.1 Key Features

- √ **Bidirectional Conversion:** Single module supports both upsize and downsize
- √ **Full AXI4 Protocol:** All 5 channels (AW, W, B, AR, R) with complete signal support
- √ **Burst Preservation:** Maintains burst semantics across width conversion
- √ **Auto Burst Recalculation:** Adjusts AWLEN/ARLEN and AWSIZE/ARSIZE automatically

- ✓ **Error Propagation:** Correctly forwards SLVERR/DECERR responses
- ✓ **Elastic Buffering:** Configurable FIFO depths for each channel
- ✓ **Status Outputs:** Busy signal and pending transaction counters
- ✓ **Parameter Validation:** Compile-time checks for valid configurations

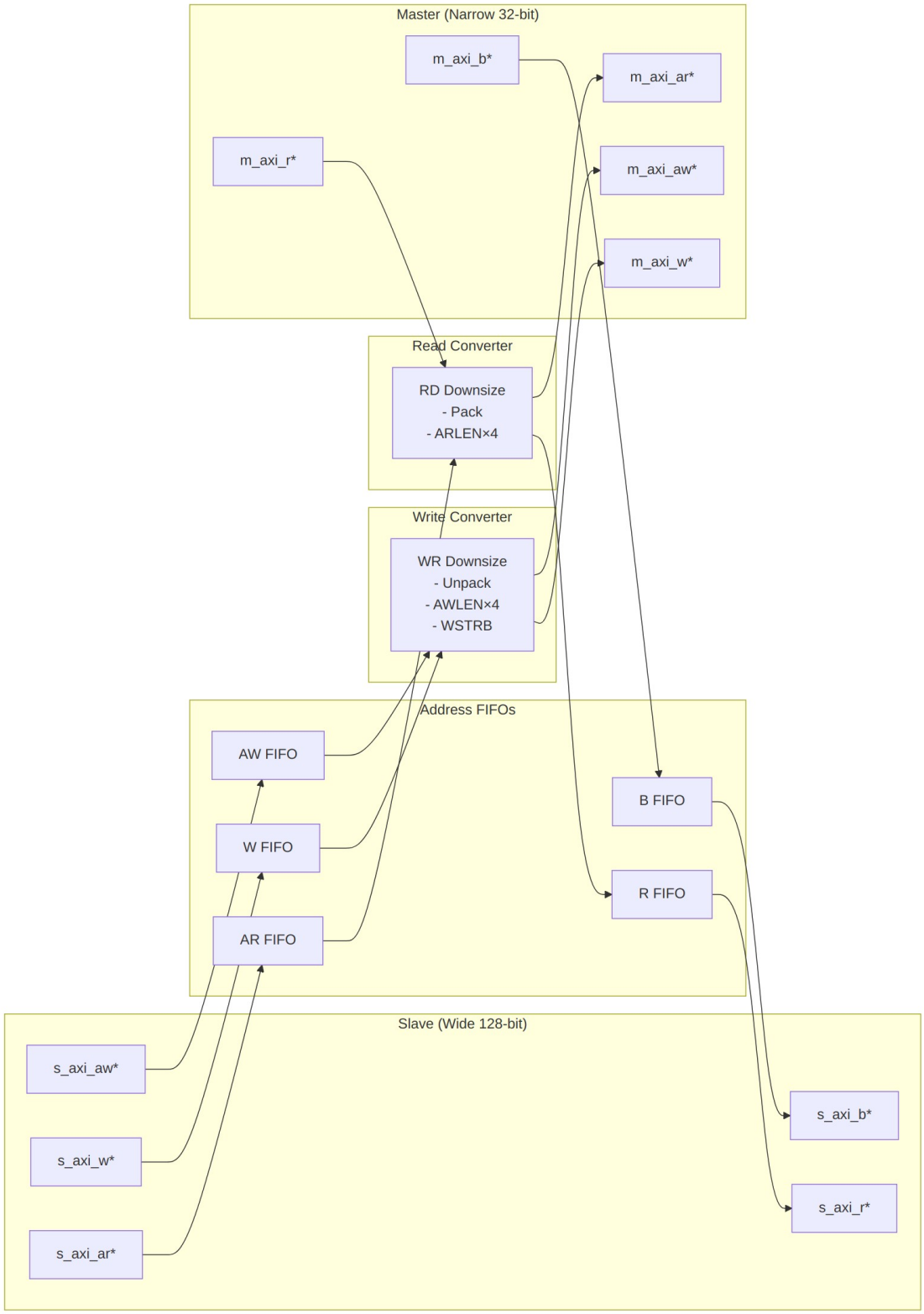
---

2.2 Module Architecture


Master (Wide 128-bit)
m_axi_b*
m_axi_ar*
m_axi_r*
m_axi_aw*
m_axi_w*
Read Converter
RD Upsize
- Unpack
- ARLEN/4
Write Converter
WR Upsize
- Pack 4→1
- AWLEN/4
- WSTRB
Address FIFOs
AW FIFO
B FIFO
W FIFO
R FIFO
AR FIFO
Slave (Narrow 32-bit)
s_axi_aw*
s_axi_b*
s_axi_w*
s_axi_r*
s_axi_ar*

Upsize Mode (Narrow → Wide)

WIDTH_RATIO = **4** (128/32): Burst 16 beats @ 32-bit → 4 beats @ 128-bit



Downsize Mode (Wide → Narrow)

WIDTH_RATIO = **4** (128/32): Burst 4 beats @ 128-bit → 16 beats @ 32-bit

2.3 Parameters

2.3.1 Width Configuration

Parameter	Type	Default	Range	Description
`S_AXI_DATA_WIDTH`	int	32	8-1024	Slave interface data width (power of 2)
`M_AXI_DATA_WIDTH`	int	128	8-1024	Master interface data width (power of 2)
`AXI_ID_WIDTH`	int	8	1-16	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	12-64	Address bus width
`AXI_USER_WIDTH`	int	1	0-1024	User signal width

2.3.2 Buffer Depths

Parameter	Type	Default	Description
`AW_FIFO_DEPTH`	int	4	Write address FIFO depth (power of 2)
`W_FIFO_DEPTH`	int	8	Write data FIFO depth (power of 2)
`B_FIFO_DEPTH`	int	4	Write response FIFO depth (power of 2)
`AR_FIFO_DEPTH`	int	4	Read address FIFO depth (power of 2)
`R_FIFO_DEPTH`	int	8	Read data FIFO depth (power of 2)

2.3.3 Calculated Parameters (Auto)

Parameter	Calculation	Description
S_STRB_WIDTH	S_AXI_DATA_WIDTH / 8	Slave write strobe width
M_STRB_WIDTH	M_AXI_DATA_WIDTH / 8	Master write strobe width
WIDTH_RATIO	MAX / MIN	Data width ratio (2-16)
UPSIZE	S < M	1 if upsizing, 0 if downsizing
DOWNSIZE	S > M	1 if downsizing, 0 if upsizing
PTR_WIDTH	$clog2(WIDTH_RATIO)	Beat pointer width

2.4 Port Groups

2.4.1 AXI4 Slave Interface

Write Channels: - AW channel: s_axi_awid, s_axi_awaddr, s_axi_awlen, s_axi_awsize, s_axi_awburst, s_axi_awlock, s_axi_awcache, s_axi_awprot, s_axi_awqos, s_axi_awregion, s_axi_awuser, s_axi_awvalid, s_axi_awready - W channel: s_axi_wdata[S_AXI_DATA_WIDTH], s_axi_wstrb[S_STRB_WIDTH], s_axi_wlast, s_axi_wuser, s_axi_wvalid, s_axi_wready - B channel: s_axi_bid, s_axi_bresp, s_axi_buser, s_axi_bvalid, s_axi_bready

Read Channels: - AR channel: s_axi_arid, s_axi_araddr, s_axi_arlen, s_axi_arsize, s_axi_arburst, s_axi_arlock, s_axi_arcache, s_axi_arprot, s_axi_arqos, s_axi_arregion, s_axi_aruser, s_axi_arvalid, s_axi_arready - R channel: s_axi_rid, s_axi_rdata[S_AXI_DATA_WIDTH], s_axi_rresp, s_axi_rlast, s_axi_ruser, s_axi_rvalid, s_axi_rready

2.4.2 AXI4 Master Interface

Write Channels: - AW channel: m_axi_awid, m_axi_awaddr, m_axi_awlen, m_axi_awsize, m_axi_awburst, m_axi_awlock, m_axi_awcache, m_axi_awprot, m_axi_awqos, m_axi_awregion, m_axi_awuser, m_axi_awvalid, m_axi_awready - W channel: m_axi_wdata[M_AXI_DATA_WIDTH], m_axi_wstrb[M_STRB_WIDTH], m_axi_wlast, m_axi_wuser, m_axi_wvalid, m_axi_wready - B channel: m_axi_bid, m_axi_bresp, m_axi_buser, m_axi_bvalid, m_axi_bready

Read Channels: - AR channel: `m_axi_arid, m_axi_araddr, m_axi_arlen, m_axi_arsize, m_axi_arburst, m_axi_arlock, m_axi_arcache, m_axi_arprot, m_axi_arqos, m_axi_arregion, m_axi_aruser, m_axi_arvalid, m_axi_arready` - R channel: `m_axi_rid, m_axi_rdata[M_AXI_DATA_WIDTH], m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid, m_axi_rready`

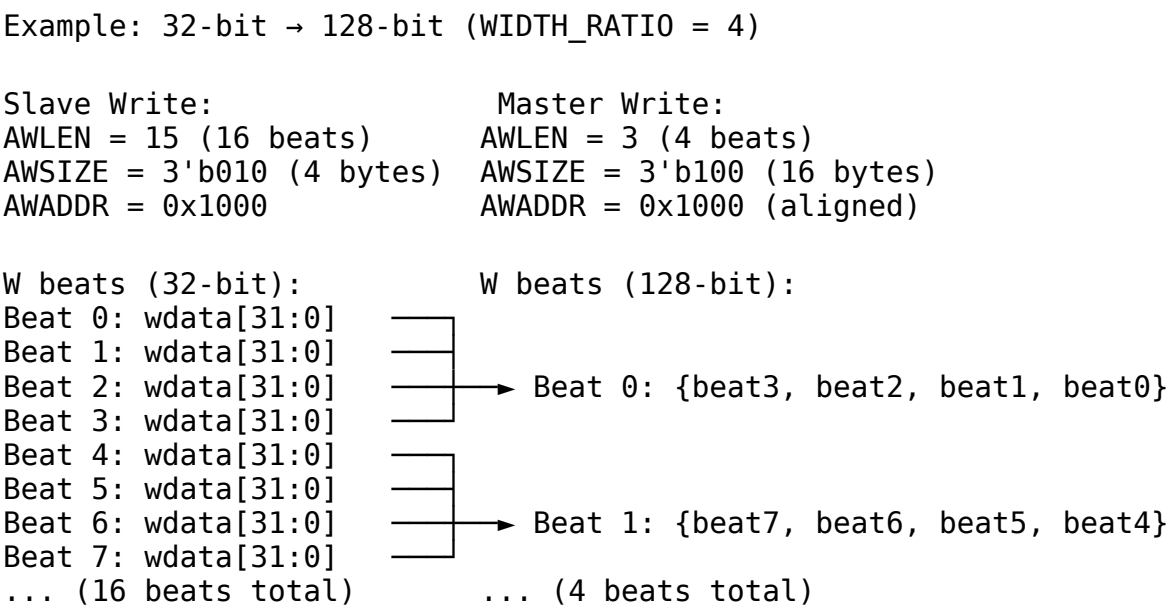
2.4.3 Status/Debug Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active conversions in progress
`wr_transactions_pending`	Output	16	Number of pending write transactions
`rd_transactions_pending`	Output	16	Number of pending read transactions

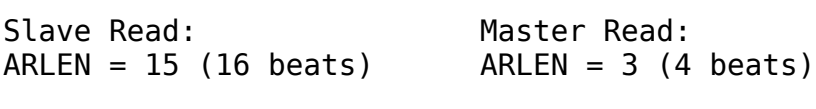
2.5 Conversion Mechanics

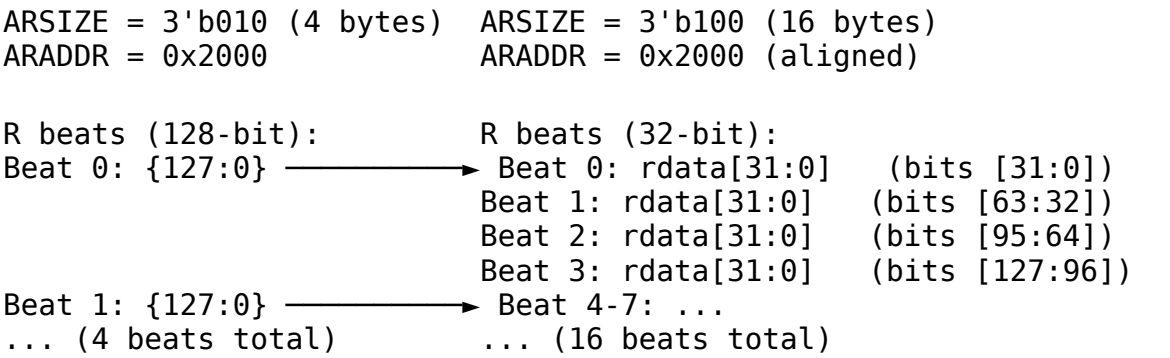
2.5.1 Upsize Conversion (Narrow → Wide)

Write Path:



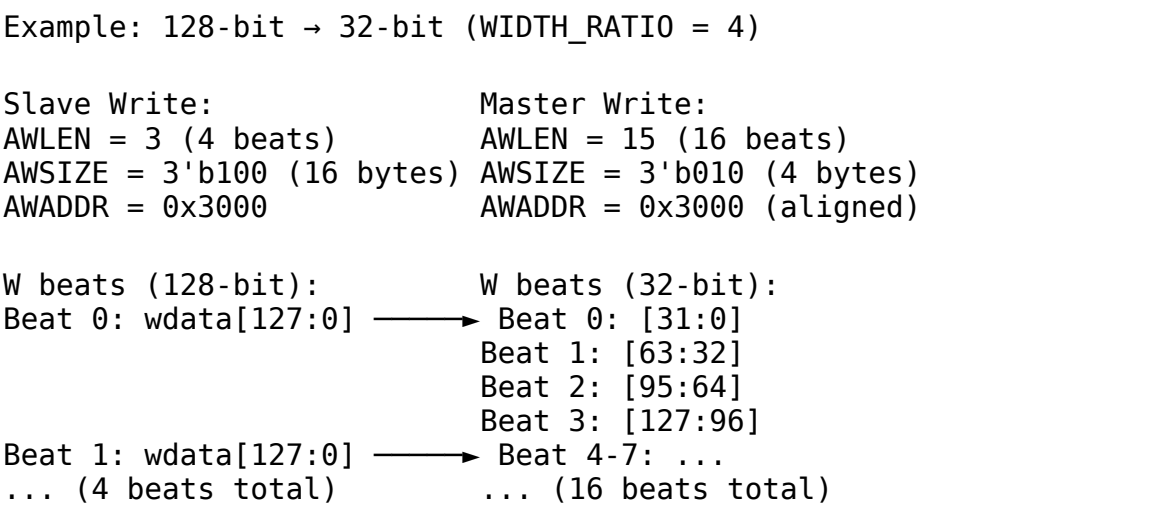
Read Path:



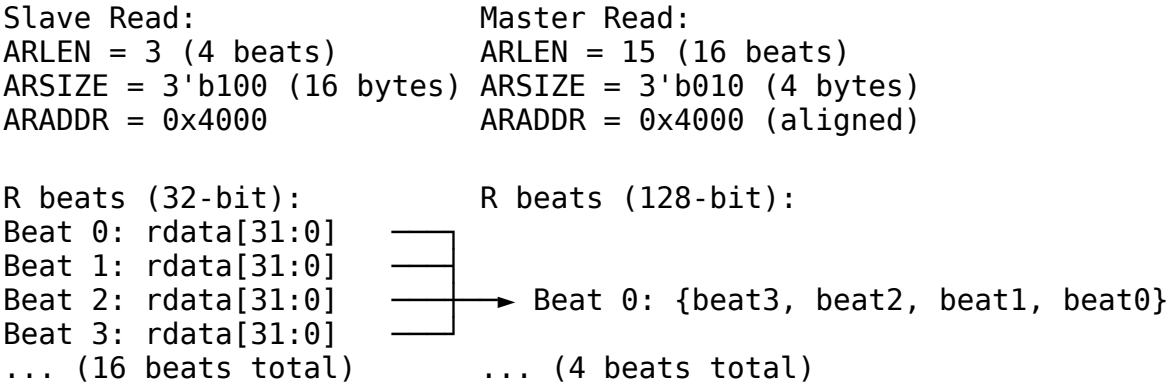


2.5.2 Downsize Conversion (Wide → Narrow)

Write Path:



Read Path:



2.5.3 WSTRB Handling

Upsize: Packs narrow WSTRB beats into wide WSTRB

```
32-bit WSTRB:    4'b1111, 4'b1111, 4'b1111, 4'b1111
                    ↓        ↓        ↓        ↓
128-bit WSTRB:   16'b1111_1111_1111_1111
```

Downsize: Unpacks wide WSTRB into narrow WSTRB beats

```
128-bit WSTRB:   16'b1111_0000_1111_0000
                    ↓
32-bit WSTRB:    4'b0000, 4'b1111, 4'b0000, 4'b1111
```

2.6 Usage Example

2.6.1 Basic Upsize Converter (32-bit → 128-bit)

```
axi4_dwidth_converter #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (32),     // Narrow slave
    .M_AXI_DATA_WIDTH   (128),    // Wide master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_USER_WIDTH     (1),

    // Buffer depths
    .AW_FIFO_DEPTH      (4),
    .W_FIFO_DEPTH       (16),     // Deeper for burst accumulation
    .B_FIFO_DEPTH       (4),
    .AR_FIFO_DEPTH      (4),
    .R_FIFO_DEPTH       (16)      // Deeper for burst unpacking
) u_upsize_conv (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (narrow) interface
    .s_axi_awid         (narrow_awid),
    .s_axi_awaddr       (narrow_awaddr),
    .s_axi_awlen        (narrow_awlen),     // e.g., 15 (16 beats)
    .s_axi_awsize       (narrow_awsize),    // e.g., 3'b010 (4 bytes)
    .s_axi_awvalid      (narrow_awvalid),
    .s_axi_awready      (narrow_awready),
    // ... rest of slave AW/W/B/AR/R signals

    // Master (wide) interface
    .m_axi_awid         (wide_awid),
    .m_axi_awaddr       (wide_awaddr),
    .m_axi_awlen        (wide_awlen),       // e.g., 3 (4 beats)
    .m_axi_awsize       (wide_awsize),      // e.g., 3'b100 (16 bytes)
```

```
    .m_axi_awvalid      (wide_awvalid),
    .m_axi_awready      (wide_awready),
    // ... rest of master AW/W/B/AR/R signals

    // Status
    .busy                   (conv_busy),
    .wr_transactions_pending(wr_pend),
    .rd_transactions_pending(rd_pend)
);

// Typical upsize scenario: CPU (32-bit) → Memory controller (128-bit)
```

2.6.2 Basic Downsize Converter (128-bit → 32-bit)

```
axi4_dwidth_converter #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (128),    // Wide slave
    .M_AXI_DATA_WIDTH   (32),     // Narrow master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),

    // Buffer depths
    .AW_FIFO_DEPTH      (4),
    .W_FIFO_DEPTH       (8),      // Moderate depth for unpacking
    .B_FIFO_DEPTH       (4),
    .AR_FIFO_DEPTH      (4),
    .R_FIFO_DEPTH       (32)      // Deeper for burst packing
) u_downsize_conv (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (wide) interface
    .s_axi_awid         (wide_awid),
    .s_axi_awaddr       (wide_awaddr),
    .s_axi_awlen        (wide_awlen),       // e.g., 3 (4 beats)
    .s_axi_awsize       (wide_awsize),      // e.g., 3'b100 (16 bytes)
    // ... rest of slave signals

    // Master (narrow) interface
    .m_axi_awid         (narrow_awid),
    .m_axi_awaddr       (narrow_awaddr),
    .m_axi_awlen        (narrow_awlen),     // e.g., 15 (16 beats)
    .m_axi_awsize       (narrow_awsize),    // e.g., 3'b010 (4 bytes)
    // ... rest of master signals

    // Status
    .busy                   (conv_busy),
```

```
    .wr_transactions_pending(wr_pend),
    .rd_transactions_pending(rd_pend)
);

// Typical downsize scenario: Interconnect (128-bit) → Peripheral (32-bit)
```

2.7 Design Notes

2.7.1 WIDTH_RATIO Constraints

Supported Ratios: 2, 4, 8, 16 - WIDTH_RATIO = 2: e.g., 32→64, 64→128, 128→256 - WIDTH_RATIO = 4: e.g., 32→128, 64→256 - WIDTH_RATIO = 8: e.g., 32→256, 64→512 - WIDTH_RATIO = 16: e.g., 32→512, 64→1024

Why Limited to 16? - Larger ratios require excessive buffering (512+ beats) - Address alignment becomes complex - Performance degrades significantly - Rare in real designs (use multi-stage conversion instead)

2.7.2 Address Alignment

Critical: Slave addresses must be aligned to master data width in upsize mode

```
// Upsize 32→128 (WIDTH_RATIO=4, M_STRB_WIDTH=16)
// Slave addresses must be 16-byte aligned (bottom 4 bits = 0)

✓ VALID:   AWADDR = 0x1000 (aligned to 16 bytes)
✓ VALID:   AWADDR = 0x2000
✗ INVALID: AWADDR = 0x1004 (not 16-byte aligned)
✗ INVALID: AWADDR = 0x2008 (not 16-byte aligned)
```

Downsize: No alignment restrictions (master can access any byte)

2.7.3 Burst Length Calculation

Upsize:

```
Master AWLEN = (Slave AWLEN + 1) / WIDTH_RATIO - 1
Master AWSIZE = Slave AWSIZE + $clog2(WIDTH_RATIO)

Example: 32→128 (WIDTH_RATIO=4)
Slave:  AWLEN=15, AWSIZE=2 (4 bytes)  → 16 beats × 4 bytes = 64 bytes
Master: AWLEN=3,  AWSIZE=4 (16 bytes) → 4 beats × 16 bytes = 64 bytes
```

Downsize:

```
Master AWLEN = (Slave AWLEN + 1) * WIDTH_RATIO - 1
Master AWSIZE = Slave AWSIZE - $clog2(WIDTH_RATIO)

Example: 128→32 (WIDTH_RATIO=4)
Slave:  AWLEN=3,  AWSIZE=4 (16 bytes) → 4 beats × 16 bytes = 64 bytes
Master: AWLEN=15, AWSIZE=2 (4 bytes)  → 16 beats × 4 bytes = 64 bytes
```

2.7.4 Buffer Depth Guidelines

Address Channels (AW/AR): - Default: 4 entries (sufficient for most cases) - Increase if high address command rate

Data Channels (W/R): - **Upsize:** Deeper buffers needed to accumulate narrow beats - W_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length - R_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length - **Downsize:** Moderate buffers for unpacking wide beats - W_FIFO_DEPTH ≥ max_burst_length (wide side) - R_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length (for packing)

Example (32→128, max burst=16 narrow beats):

```
.W_FIFO_DEPTH  (16),  // Accumulate 16 narrow beats → 4 wide beats
.R_FIFO_DEPTH  (16)   // Unpack 4 wide beats → 16 narrow beats
```

2.7.5 Performance Characteristics

Throughput (Upsize): - Accumulation latency: WIDTH_RATIO-1 cycles per wide beat - Example (WIDTH_RATIO=4): 3-cycle latency to accumulate first wide beat

Throughput (Downsize): - Unpacking latency: 1 cycle per narrow beat - Full wide beat can be unpacked in consecutive cycles

Backpressure Handling: - FIFOs provide elastic buffering - Prevents deadlock during width mismatch scenarios - Slave can stall independently of master

---

2.8 Related Modules

2.8.1 Specialized Converters

- **axi4_dwidth_converter_rd** - Read-only data width conversion
- **axi4_dwidth_converter_wr** - Write-only data width conversion

2.8.2 Core Modules

- **axi4_master_rd** - AXI4 read master
- **axi4_master_wr** - AXI4 write master
- **axi4_slave_rd** - AXI4 read slave
- **axi4_slave_wr** - AXI4 write slave

2.8.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
- **gaxi_fifo_sync** - Synchronous FIFOs

2.9 References

2.9.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Chapter 11: Data Width Conversion

2.9.2 Source Code

- RTL: `rtl/amba/axi4/axi4_dwidth_converter.sv`
- Tests: `val/amba/test_axi4_dwidth_converter.py`
- Framework: `bin/TBClasses/components/axi4/`

2.9.3 Documentation

- Architecture: RTLAmba Overview
- AXI4 Index: README.md

Last Updated: 2025-10-20

2.10 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

3 AXI4 Read Data Width Converter

Module: `axi4_dwidth_converter_rd.sv` **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

3.1 Overview

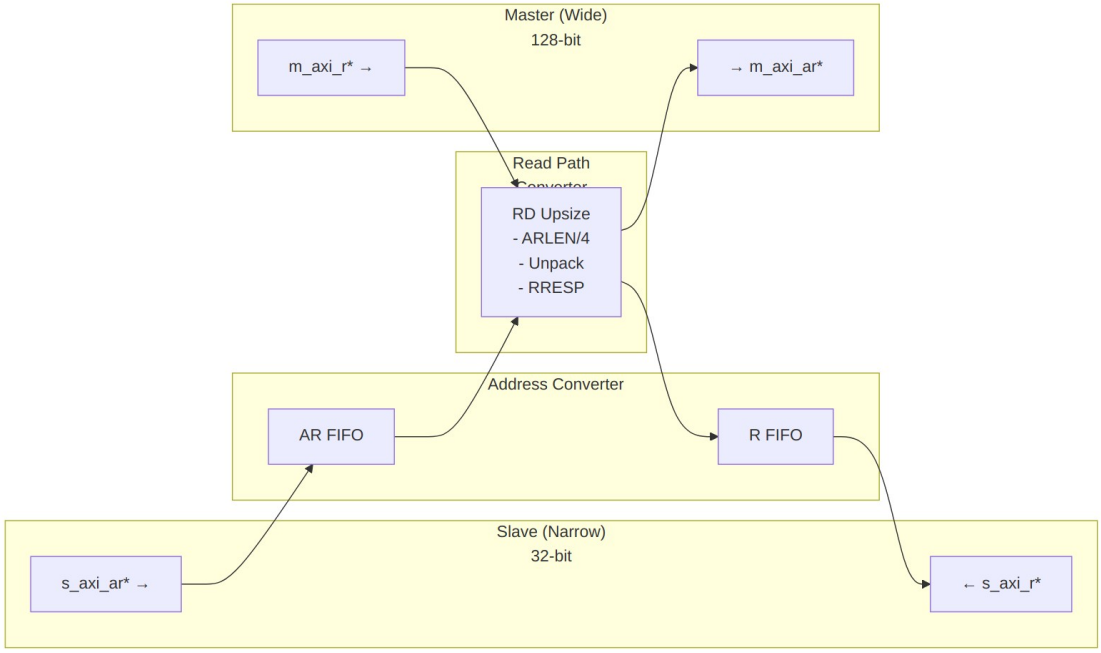
The AXI4 Read Data Width Converter provides data width conversion for read-only AXI4 interfaces (AR and R channels only). This specialized converter is optimized for unidirectional bridges where the read and write paths are separate, reducing resource usage compared to the full bidirectional converter.

3.1.1 Key Features

- √ **Read-Only:** AR and R channels only (no write channels)
- √ **Bidirectional Conversion:** Supports both upsize and downsize
- √ **Resource Optimized:** Smaller than bidirectional converter
- √ **Burst Preservation:** Maintains burst semantics across conversion
- √ **Error Propagation:** Correctly forwards RRESP error codes
- √ **Elastic Buffering:** Configurable FIFO depths for AR and R channels
- √ **Status Outputs:** Busy signal and pending transaction counter

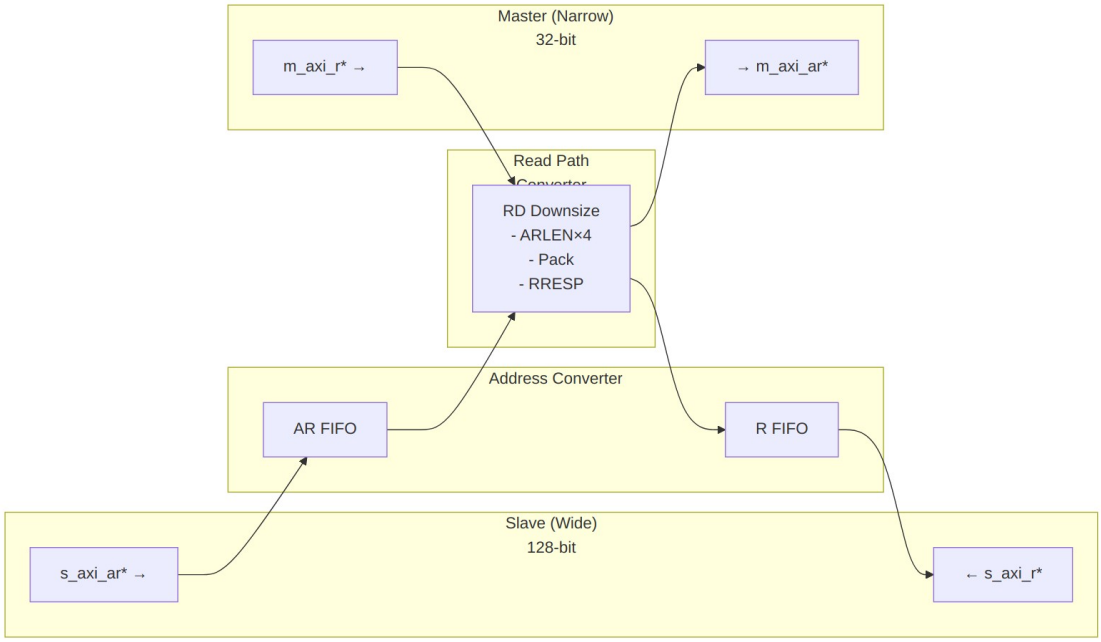
---

3.2 Module Architecture



Upsize Mode (Narrow → Wide Reads)

Example: 16 narrow R beats (32-bit) ← 4 wide R beats (128-bit)



Downsize Mode (Wide → Narrow Reads)

Example: 4 wide R beats (128-bit) ← 16 narrow R beats (32-bit)

3.3 Parameters

3.3.1 Width Configuration

Parameter	Type	Default	Range	Description
`S_AXI_DATA_WIDTH`	int	32	8-1024	Slave interface data width (power of 2)
`M_AXI_DATA_WIDTH`	int	128	8-1024	Master interface data width (power of 2)
`AXI_ID_WIDTH`	int	8	1-16	Transaction ID width
`AXI_ADDR_WIDTH`	int	32	12-64	Address bus width
`AXI_USER_WIDTH`	int	1	0-1024	User signal width

3.3.2 Buffer Depths

Parameter	Type	Default	Description
`AR_FIFO_DEPTH`	int	4	Read address

Parameter	Type	Default	Description
			FIFO depth (power of 2)
R_FIFO_DEPTH	int	8	Read data FIFO depth (power of 2)

3.3.3 Calculated Parameters (Auto)

Parameter	Calculation	Description
WIDTH_RATIO	MAX / MIN	Data width ratio (2-16)
UPSIZE	S < M	1 if upsizing, 0 if downsizing
DOWNSIZE	S > M	1 if downsizing, 0 if upsizing

3.4 Port Groups

3.4.1 AXI4 Slave Read Interface

Read Address Channel (AR): - `s_axi_arid, s_axi_araddr, s_axi_arlen, s_axi_arsize, s_axi_arburst, s_axi_arlock, s_axi_arcache, s_axi_arprot, s_axi_arqos, s_axi_arregion, s_axi_aruser, s_axi_arvalid, s_axi_arready`

Read Data Channel (R): - `s_axi_rid, s_axi_rdata[S_AXI_DATA_WIDTH], s_axi_rresp, s_axi_rlast, s_axi_ruser, s_axi_rvalid, s_axi_rready`

3.4.2 AXI4 Master Read Interface

Read Address Channel (AR): - `m_axi_arid, m_axi_araddr, m_axi_arlen, m_axi_arsize, m_axi_arburst, m_axi_arlock, m_axi_arcache, m_axi_arprot, m_axi_arqos, m_axi_arregion, m_axi_aruser, m_axi_arvalid, m_axi_arready`

Read Data Channel (R): - `m_axi_rid, m_axi_rdata[M_AXI_DATA_WIDTH], m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid, m_axi_rready`

3.4.3 Status/Debug Outputs

Port	Direction	Width	Description
busy	Output	1	Indicates active read conversions in progress

Port	Direction	Width	Description
rd_transactions_pending	Output	16	Number of pending read transactions

3.5 Read Conversion Mechanics

3.5.1 Upsize Read (Narrow → Wide)

```
Example: 32-bit → 128-bit (WIDTH_RATIO = 4)

Slave Read Request:            Master Read Request:
ARLEN = 15 (16 beats)          ARLEN = 3 (4 beats)
ARSIZE = 3'b010 (4 bytes)      ARSIZE = 3'b100 (16 bytes)
ARADDR = 0x2000                ARADDR = 0x2000 (aligned)

Master R beats (128-bit):      Slave R beats (32-bit):
Beat 0: rdata[127:0] ──────►    Beat 0: rdata[31:0]   (bits [31:0])
                                Beat 1: rdata[31:0]   (bits [63:32])
                                Beat 2: rdata[31:0]   (bits [95:64])
                                Beat 3: rdata[31:0]   (bits [127:96])
Beat 1: rdata[127:0] ──────►    Beat 4-7: ...
... (4 beats total)             ... (16 beats total)

RRESP handling:
Master: OKAY (all beats)        Slave: OKAY on each narrow beat
```

3.5.2 Downsize Read (Wide → Narrow)

```
Example: 128-bit → 32-bit (WIDTH_RATIO = 4)

Slave Read Request:            Master Read Request:
ARLEN = 3 (4 beats)            ARLEN = 15 (16 beats)
ARSIZE = 3'b100 (16 bytes)     ARSIZE = 3'b010 (4 bytes)
ARADDR = 0x4000                ARADDR = 0x4000 (aligned)

Master R beats (32-bit):       Slave R beats (128-bit):
Beat 0: rdata[31:0]  ──┐
Beat 1: rdata[31:0]  ──┤
Beat 2: rdata[31:0]  ──┼──►    Beat 0: {beat3, beat2, beat1, beat0}
Beat 3: rdata[31:0]  ──┘
... (16 beats total)            ... (4 beats total)

RRESP handling:
```

```
Master: OKAY, OKAY, SLVERR, OKAY
Slave:  Beat 0 → SLVERR (worst-case RRESP propagation)
```

3.5.3 RRESP Error Propagation

Upsize: Each narrow beat inherits RRESP from corresponding wide beat

```
Master R beats:          Slave R beats:
Beat 0: RRESP=OKAY  →  Beat 0-3: RRESP=OKAY
Beat 1: RRESP=SLVERR → Beat 4-7: RRESP=SLVERR
```

Downsize: Wide beat RRESP is worst-case of narrow beats

```
Master R beats:          Slave R beat:
Beat 0-2: RRESP=OKAY  ┐
Beat 3: RRESP=SLVERR  ┼→ Beat 0: RRESP=SLVERR (worst-case)
```

3.6 Usage Example

3.6.1 Basic Read Upsize Converter (32-bit → 128-bit)

```
axi4_dwidth_converter_rd #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (32),     // Narrow slave
    .M_AXI_DATA_WIDTH   (128),    // Wide master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_USER_WIDTH     (1),

    // Buffer depths
    .AR_FIFO_DEPTH      (4),
    .R_FIFO_DEPTH       (16)      // Deeper for burst unpacking
) u_rd_upsize (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (narrow) interface
    .s_axi_arid         (cpu_arid),
    .s_axi_araddr       (cpu_araddr),
    .s_axi_arlen        (cpu_arlen),     // e.g., 15 (16 beats)
    .s_axi_arsize       (cpu_arsize),    // e.g., 3'b010 (4 bytes)
    .s_axi_arvalid      (cpu_arvalid),
    .s_axi_arready      (cpu_arready),
    // ... rest of AR signals

    .s_axi_rid          (cpu_rid),
```

```
    .s_axi_rdata        (cpu_rdata),     // 32-bit
    .s_axi_rresp        (cpu_rresp),
    .s_axi_rlast        (cpu_rlast),
    .s_axi_rvalid       (cpu_rvalid),
    .s_axi_rready       (cpu_rready),

    // Master (wide) interface
    .m_axi_arid         (mem_arid),
    .m_axi_araddr       (mem_araddr),
    .m_axi_arlen        (mem_arlen),     // e.g., 3 (4 beats)
    .m_axi_arsize       (mem_arsize),    // e.g., 3'b100 (16 bytes)
    .m_axi_arvalid      (mem_arvalid),
    .m_axi_arready      (mem_arready),
    // ... rest of AR signals

    .m_axi_rid          (mem_rid),
    .m_axi_rdata        (mem_rdata),     // 128-bit
    .m_axi_rresp        (mem_rresp),
    .m_axi_rlast        (mem_rlast),
    .m_axi_rvalid       (mem_rvalid),
    .m_axi_rready       (mem_rready),

    // Status
    .busy                   (rd_busy),
    .rd_transactions_pending(rd_pend)
);

// Use case: CPU (32-bit read) ← Memory controller (128-bit interface)
```

3.6.2 Basic Read Downsize Converter (128-bit → 32-bit)

```
axi4_dwidth_converter_rd #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (128),    // Wide slave
    .M_AXI_DATA_WIDTH   (32),     // Narrow master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),

    // Buffer depths
    .AR_FIFO_DEPTH      (4),
    .R_FIFO_DEPTH       (32)      // Deeper for burst packing
) u_rd_downsize (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (wide) interface
    .s_axi_arid         (wide_arid),
```

```
    .s_axi_araddr       (wide_araddr),
    .s_axi_arlen        (wide_arlen),     // e.g., 3 (4 beats)
    .s_axi_arsize       (wide_arsize),    // e.g., 3'b100 (16 bytes)
    .s_axi_arvalid      (wide_arvalid),
    .s_axi_arready      (wide_arready),
    // ... rest of AR signals

    .s_axi_rid          (wide_rid),
    .s_axi_rdata        (wide_rdata),     // 128-bit
    .s_axi_rresp        (wide_rresp),
    .s_axi_rlast        (wide_rlast),
    .s_axi_rvalid       (wide_rvalid),
    .s_axi_rready       (wide_rready),

    // Master (narrow) interface
    .m_axi_arid         (periph_arid),
    .m_axi_araddr       (periph_araddr),
    .m_axi_arlen        (periph_arlen),   // e.g., 15 (16 beats)
    .m_axi_arsize       (periph_arsize),  // e.g., 3'b010 (4 bytes)
    .m_axi_arvalid      (periph_arvalid),
    .m_axi_arready      (periph_arready),
    // ... rest of AR signals

    .m_axi_rid          (periph_rid),
    .m_axi_rdata        (periph_rdata),   // 32-bit
    .m_axi_rresp        (periph_rresp),
    .m_axi_rlast        (periph_rlast),
    .m_axi_rvalid       (periph_rvalid),
    .m_axi_rready       (periph_rready),

    // Status
    .busy                   (rd_busy),
    .rd_transactions_pending(rd_pend)
);

// Use case: Interconnect (128-bit) → Peripheral (32-bit read-only)
```

3.7 Design Notes

3.7.1 Read-Only Use Cases

When to Use Read-Only Converter: - Unidirectional read-only bridges - Separate read and write data paths - ROM/Flash interfaces (read-only by nature) - Read-heavy systems with dedicated write path

Resource Savings vs. Full Converter: - No AW/W/B channel FIFOs (saves ~40% resources) - Simpler control logic - Lower latency (no write path dependencies)

3.7.2 Address Alignment

Same alignment requirements as axi4_dwidth_converter:

Upsize: Slave addresses must be aligned to master data width

```
32→128 upsize: ARADDR must be 16-byte aligned (bottom 4 bits = 0)
```

Downsize: No alignment restrictions

3.7.3 Burst Length Calculation

Upsize:

```
Master ARLEN = (Slave ARLEN + 1) / WIDTH_RATIO - 1
Master ARSIZE = Slave ARSIZE + $clog2(WIDTH_RATIO)
```

Downsize:

```
Master ARLEN = (Slave ARLEN + 1) * WIDTH_RATIO - 1
Master ARSIZE = Slave ARSIZE - $clog2(WIDTH_RATIO)
```

See axi4_dwidth_converter for detailed examples.

3.7.4 Buffer Depth Guidelines

AR Channel: - Default: 4 entries (sufficient for most cases) - Increase for high address command rate

R Channel: - **Upsize:** R_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length - Need to buffer unpacked narrow beats - **Downsize:** R_FIFO_DEPTH ≥ WIDTH_RATIO × max_burst_length - Need to accumulate narrow beats for packing

Example (32→128, max burst=16 narrow beats):

```
.AR_FIFO_DEPTH  (4),   // Address commands
.R_FIFO_DEPTH   (16)   // Unpack 4 wide → 16 narrow beats
```

3.7.5 Performance Characteristics

Throughput (Upsize): - Unpacking latency: 1 cycle per narrow beat - Full wide beat unpacked in consecutive cycles

Throughput (Downsize): - Packing latency: WIDTH_RATIO-1 cycles per wide beat - Need to accumulate all narrow beats before forwarding

Comparison to Full Converter: - ~20-30% lower latency (no write path overhead) - ~40% resource savings (no write channel FIFOs)

3.8 Related Modules

3.8.1 Companion Converters

- **axi4_dwidth_converter** - Full bidirectional converter (AW/W/B/AR/R)
- **axi4_dwidth_converter_wr** - Write-only data width conversion

3.8.2 Core Modules

- **axi4_master_rd** - AXI4 read master
- **axi4_slave_rd** - AXI4 read slave

3.8.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering
- **gaxi_fifo_sync** - Synchronous FIFOs

3.9 References

3.9.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Chapter 11: Data Width Conversion

3.9.2 Source Code

- RTL: `rtl/amba/axi4/axi4_dwidth_converter_rd.sv`
- Tests: `val/amba/test_axi4_dwidth_converter_rd.py`
- Framework: `bin/TBClasses/components/axi4/`

3.9.3 Documentation

- Architecture: RTLAmba Overview
- AXI4 Index: README.md

Last Updated: 2025-10-20

3.10 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

4 AXI4 Write Data Width Converter

Module: `axi4_dwidth_converter_wr.sv` **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

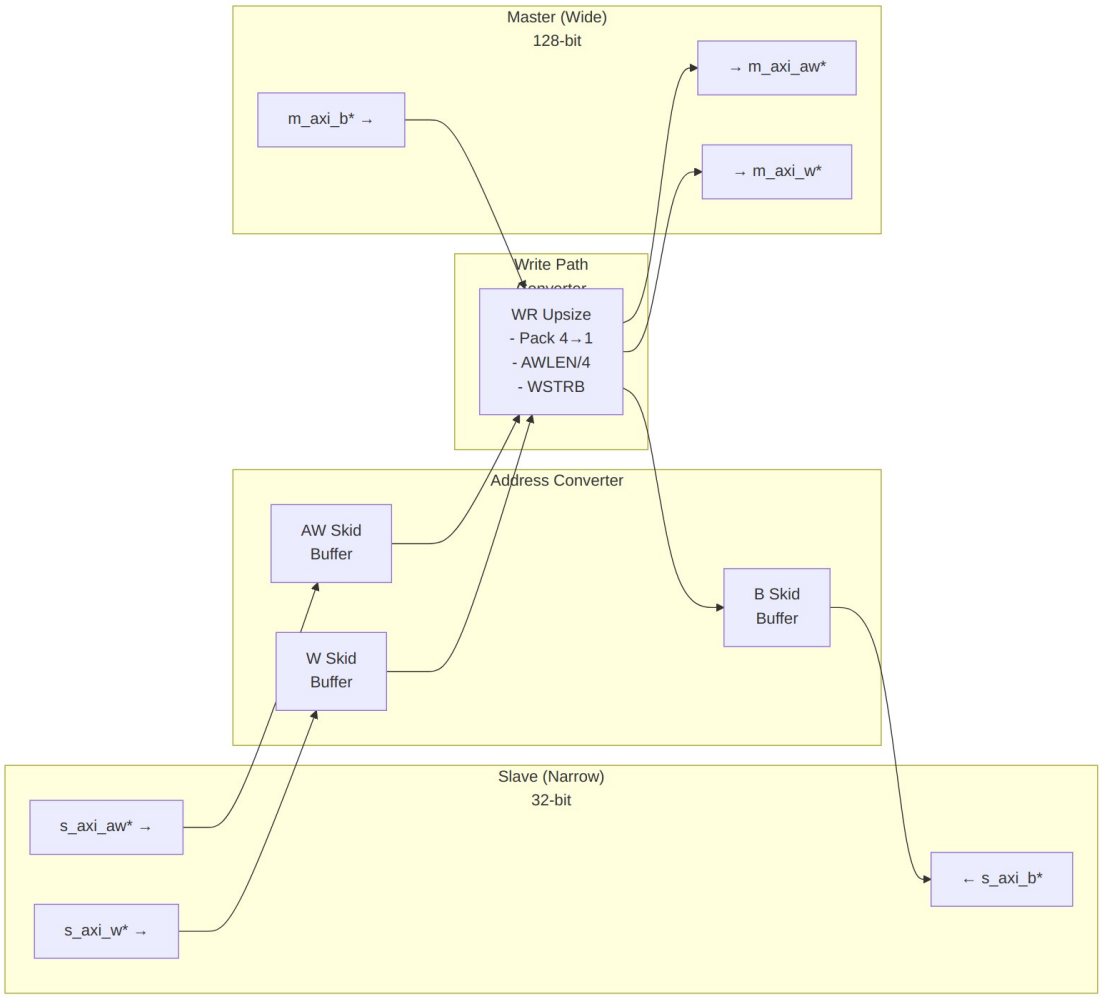
4.1 Overview

The AXI4 Write Data Width Converter provides data width conversion for write-only AXI4 interfaces (AW, W, and B channels only). This specialized converter is optimized for unidirectional bridges where the read and write paths are separate, using skid buffers for timing closure and reducing resource usage compared to the full bidirectional converter.

4.1.1 Key Features

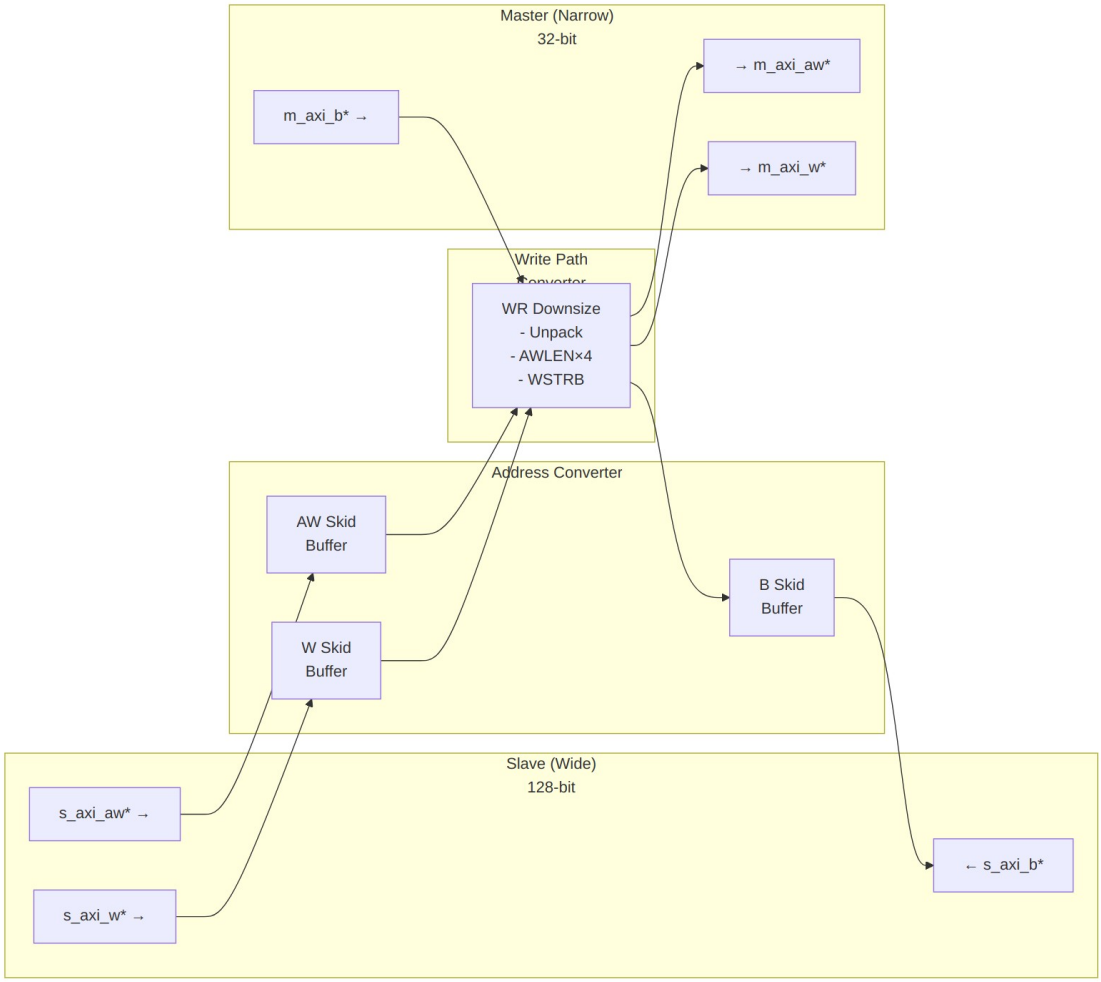
- √ **Write-Only:** AW, W, and B channels only (no read channels)
- √ **Bidirectional Conversion:** Supports both upsize and downsize
- √ **Timing Closure:** Uses gaxi_skid_buffer on all channels
- √ **Resource Optimized:** Smaller than bidirectional converter
- √ **Burst Preservation:** Maintains burst semantics across conversion
- √ **WSTRB Handling:** Correct packing/unpacking of write strobes
- √ **Error Propagation:** Correctly forwards BRESP codes
- √ **Elastic Buffering:** Configurable skid buffer depths

4.2 Module Architecture



Upsize Mode (Narrow → Wide Writes)

Example: 16 narrow W beats (32-bit) → 4 wide W beats (128-bit)



Downsize Mode (Wide → Narrow Writes)

Example: 4 wide W beats (128-bit) → 16 narrow W beats (32-bit)

4.3 Parameters

4.3.1 Width Configuration

Parameter	Type	Default	Range	Description
`S_AXI_DATA_WIDTH`	int	32	32-256	Slave interface data width (power of 2)
`M_AXI_DATA_WIDTH`	int	128	32-256	Master interface data width (power of 2)

Parameter	Type	Default	Range	Description
AXI_ID_WIDTH	int	8	1-16	Transaction ID width
AXI_ADDR_WIDTH	int	32	12-64	Address bus width
AXI_USER_WIDTH	int	1	0-1024	User signal width

4.3.2 Skid Buffer Depths

Parameter	Type	Default	Range	Description
SKID_DEPTH_AW	int	2	2-8	AW channel skid buffer depth
SKID_DEPTH_W	int	4	2-8	W channel skid buffer depth
SKID_DEPTH_B	int	2	2-8	B channel skid buffer depth

4.3.3 Calculated Parameters (Auto)

Parameter	Calculation	Description
S_STRB_WIDTH	S_AXI_DATA_WIDTH / 8	Slave write strobe width
M_STRB_WIDTH	M_AXI_DATA_WIDTH / 8	Master write strobe width
WIDTH_RATIO	MAX / MIN	Data width ratio (2-16)
UPSIZE	S < M	1 if upsizing, 0 if downsizing
DOWNSIZE	S > M	1 if downsizing, 0 if upsizing
PTR_WIDTH	$clog2(WIDTH_RATIO)	Beat pointer width

4.4 Port Groups

4.4.1 AXI4 Slave Write Interface

Write Address Channel (AW): - `s_axi_awid, s_axi_awaddr, s_axi_awlen, s_axi_awsize, s_axi_awburst, s_axi_awlock, s_axi_awcache, s_axi_awprot, s_axi_awqos, s_axi_awregion, s_axi_awuser, s_axi_awvalid, s_axi_awready`

Write Data Channel (W): - `s_axi_wdata[S_AXI_DATA_WIDTH], s_axi_wstrb[S_STRB_WIDTH], s_axi_wlast, s_axi_wuser, s_axi_wvalid, s_axi_wready`

Write Response Channel (B): - `s_axi_bid, s_axi_bresp, s_axi_buser, s_axi_bvalid, s_axi_bready`

4.4.2 AXI4 Master Write Interface

Write Address Channel (AW): - `m_axi_awid, m_axi_awaddr, m_axi_awlen, m_axi_awsize, m_axi_awburst, m_axi_awlock, m_axi_awcache, m_axi_awprot, m_axi_awqos, m_axi_awregion, m_axi_awuser, m_axi_awvalid, m_axi_awready`

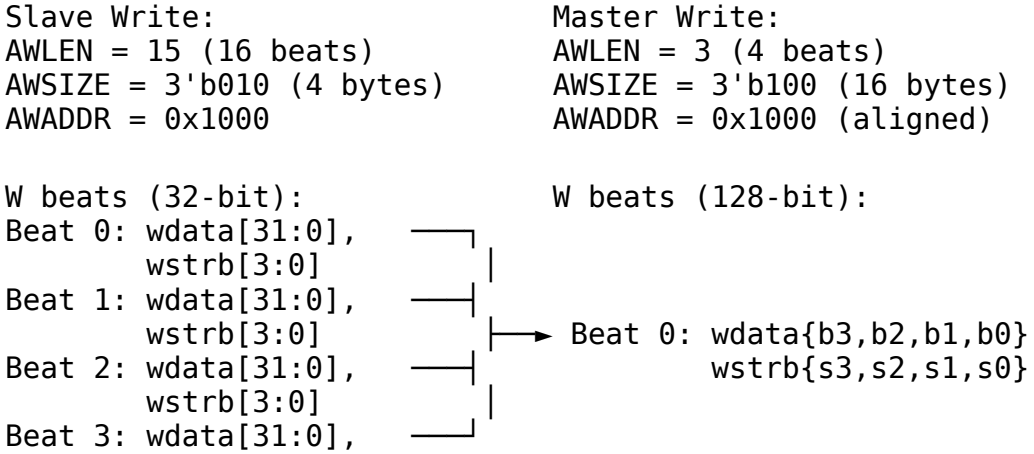
Write Data Channel (W): - `m_axi_wdata[M_AXI_DATA_WIDTH], m_axi_wstrb[M_STRB_WIDTH], m_axi_wlast, m_axi_wuser, m_axi_wvalid, m_axi_wready`

Write Response Channel (B): - `m_axi_bid, m_axi_bresp, m_axi_buser, m_axi_bvalid, m_axi_bready`

4.5 Write Conversion Mechanics

4.5.1 Upsize Write (Narrow → Wide)

Example: 32-bit → 128-bit (WIDTH_RATIO = 4)



```
        wstrb[3:0], WLAST=0

Beat 4-7: ...          ──┐
                         ├──► Beat 1: ...
Beat 8-11: ...         ──┤
                         ├──► Beat 2: ...
Beat 12-15: ...        ──┘
        WLAST=1 (beat 15)  ──► Beat 3: WLAST=1

BRESP: Single response from master → slave
```

4.5.2 Downsize Write (Wide → Narrow)

```
Example: 128-bit → 32-bit (WIDTH_RATIO = 4)

Slave Write:                    Master Write:
AWLEN = 3 (4 beats)             AWLEN = 15 (16 beats)
AWSIZE = 3'b100 (16 bytes)      AWSIZE = 3'b010 (4 bytes)
AWADDR = 0x3000                 AWADDR = 0x3000 (aligned)

W beats (128-bit):              W beats (32-bit):
Beat 0: wdata[127:0],  ──────►  Beat 0: [31:0],   wstrb[3:0]
        wstrb[15:0]             Beat 1: [63:32],  wstrb[7:4]
                                Beat 2: [95:64],  wstrb[11:8]
                                Beat 3: [127:96], wstrb[15:12]

Beat 1: wdata[127:0],  ──────►  Beat 4-7: ...
        wstrb[15:0]
Beat 2: wdata[127:0]   ──────►  Beat 8-11: ...
Beat 3: wdata[127:0],  ──────►  Beat 12-15: ...
        WLAST=1                         WLAST=1 (beat 15)

BRESP: Single response from master → slave
```

4.5.3 WSTRB Handling

Upsize: Packs narrow WSTRB beats into wide WSTRB

```
32-bit WSTRB (4 beats):
Beat 0: 4'b1111  ──┐
Beat 1: 4'b1111  ──┤
Beat 2: 4'b0011  ──┼──► 128-bit WSTRB: 16'b1111_1111_1111_0011
Beat 3: 4'b1100  ──┘
```

Downsize: Unpacks wide WSTRB into narrow WSTRB beats

```
128-bit WSTRB: 16'b1111_0000_1111_0000
                     ↓
```

```
32-bit WSTRB (4 beats):
Beat 0: 4'b0000  (bits [3:0])
Beat 1: 4'b1111  (bits [7:4])
Beat 2: 4'b0000  (bits [11:8])
Beat 3: 4'b1111  (bits [15:12])
```

4.5.4 BRESP Propagation

Both Modes: Single B channel response is forwarded unchanged

```
Master BRESP: OKAY → Slave BRESP: OKAY
Master BRESP: SLVERR → Slave BRESP: SLVERR
Master BRESP: DECERR → Slave BRESP: DECERR
```

4.6 Usage Example

4.6.1 Basic Write Upsize Converter (32-bit → 128-bit)

```
axi4_dwidth_converter_wr #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (32),     // Narrow slave
    .M_AXI_DATA_WIDTH   (128),    // Wide master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),
    .AXI_USER_WIDTH     (1),

    // Skid buffer depths
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),      // Moderate for burst accumulation
    .SKID_DEPTH_B       (2)
) u_wr_upsize (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (narrow) interface
    .s_axi_awid         (cpu_awid),
    .s_axi_awaddr       (cpu_awaddr),
    .s_axi_awlen        (cpu_awlen),     // e.g., 15 (16 beats)
    .s_axi_awsize       (cpu_awsize),    // e.g., 3'b010 (4 bytes)
    .s_axi_awvalid      (cpu_awvalid),
    .s_axi_awready      (cpu_awready),
    // ... rest of AW signals

    .s_axi_wdata        (cpu_wdata),     // 32-bit
    .s_axi_wstrb        (cpu_wstrb),     // 4-bit
    .s_axi_wlast        (cpu_wlast),
```

```
    .s_axi_wvalid       (cpu_wvalid),
    .s_axi_wready       (cpu_wready),
    // ... rest of W signals

    .s_axi_bid          (cpu_bid),
    .s_axi_bresp        (cpu_bresp),
    .s_axi_bvalid       (cpu_bvalid),
    .s_axi_bready       (cpu_bready),

    // Master (wide) interface
    .m_axi_awid         (mem_awid),
    .m_axi_awaddr       (mem_awaddr),
    .m_axi_awlen        (mem_awlen),     // e.g., 3 (4 beats)
    .m_axi_awsize       (mem_awsize),    // e.g., 3'b100 (16 bytes)
    .m_axi_awvalid      (mem_awvalid),
    .m_axi_awready      (mem_awready),
    // ... rest of AW signals

    .m_axi_wdata        (mem_wdata),     // 128-bit
    .m_axi_wstrb        (mem_wstrb),     // 16-bit
    .m_axi_wlast        (mem_wlast),
    .m_axi_wvalid       (mem_wvalid),
    .m_axi_wready       (mem_wready),
    // ... rest of W signals

    .m_axi_bid          (mem_bid),
    .m_axi_bresp        (mem_bresp),
    .m_axi_bvalid       (mem_bvalid),
    .m_axi_bready       (mem_bready)
);

// Use case: CPU (32-bit write) → Memory controller (128-bit
interface)
```

4.6.2 Basic Write Downsize Converter (128-bit → 32-bit)

```
axi4_dwidth_converter_wr #(
    // Width configuration
    .S_AXI_DATA_WIDTH   (128),    // Wide slave
    .M_AXI_DATA_WIDTH   (32),     // Narrow master
    .AXI_ID_WIDTH       (4),
    .AXI_ADDR_WIDTH     (32),

    // Skid buffer depths
    .SKID_DEPTH_AW      (2),
    .SKID_DEPTH_W       (4),      // Moderate for unpacking
    .SKID_DEPTH_B       (2)
```

```
) u_wr_downsize (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),

    // Slave (wide) interface
    .s_axi_awid         (wide_awid),
    .s_axi_awaddr       (wide_awaddr),
    .s_axi_awlen        (wide_awlen),     // e.g., 3 (4 beats)
    .s_axi_awsize       (wide_awsize),    // e.g., 3'b100 (16 bytes)
    // ... rest of slave signals

    .s_axi_wdata        (wide_wdata),     // 128-bit
    .s_axi_wstrb        (wide_wstrb),     // 16-bit
    // ... rest of slave W signals

    // Master (narrow) interface
    .m_axi_awid         (periph_awid),
    .m_axi_awaddr       (periph_awaddr),
    .m_axi_awlen        (periph_awlen),   // e.g., 15 (16 beats)
    .m_axi_awsize       (periph_awsize),  // e.g., 3'b010 (4 bytes)
    // ... rest of master signals

    .m_axi_wdata        (periph_wdata),   // 32-bit
    .m_axi_wstrb        (periph_wstrb),   // 4-bit
    // ... rest of master W signals
);

// Use case: Interconnect (128-bit) → Peripheral (32-bit write-only)
```

4.7 Design Notes

4.7.1 Write-Only Use Cases

When to Use Write-Only Converter: - Unidirectional write-only bridges - Separate read and write data paths - Write-through caches - Write-only peripherals (configuration registers, command FIFOs)

Resource Savings vs. Full Converter: - No AR/R channel buffers (saves ~40% resources) - Simpler control logic - Lower latency (no read path dependencies)

4.7.2 Skid Buffer vs. FIFO

Why Skid Buffers: - Optimized for timing closure (single-cycle latency) - Smaller area than FIFOs for shallow depths - Sufficient for write path elasticity

Depth Guidelines: - **SKID_DEPTH_AW:** 2 (default) - sufficient for most cases - **SKID_DEPTH_W:** 4 (default) - handles moderate bursts - **SKID_DEPTH_B:** 2 (default) - single-beat responses

Increase SKID_DEPTH_W for: - Large burst lengths (AWLEN > 16) - High backpressure from master - Width ratio > 4

4.7.3 Address Alignment

Same alignment requirements as axi4_dwidth_converter:

Upsize: Slave addresses must be aligned to master data width

```
32→128 upsize: AWADDR must be 16-byte aligned (bottom 4 bits = 0)
```

Downsize: No alignment restrictions

4.7.4 Burst Length Calculation

Upsize:

```
Master AWLEN = (Slave AWLEN + 1) / WIDTH_RATIO - 1
Master AWSIZE = Slave AWSIZE + $clog2(WIDTH_RATIO)
```

Downsize:

```
Master AWLEN = (Slave AWLEN + 1) * WIDTH_RATIO - 1
Master AWSIZE = Slave AWSIZE - $clog2(WIDTH_RATIO)
```

See axi4_dwidth_converter for detailed examples.

4.7.5 Performance Characteristics

Throughput (Upsize): - Accumulation latency: WIDTH_RATIO-1 cycles per wide beat - Example (WIDTH_RATIO=4): 3-cycle latency for first wide W beat

Throughput (Downsize): - Unpacking latency: 1 cycle per narrow beat - Full wide beat unpacked in consecutive cycles

Comparison to Full Converter: - ~20-30% lower latency (no read path overhead) - ~40% resource savings (no read channel buffers)

4.7.6 WSTRB Validation

Critical: Ensure WSTRB is valid for all write beats

```
// Upsize: Each narrow beat must have valid WSTRB
always_comb begin
    if (s_axi_wvalid) begin
        assert (s_axi_wstrb != '0) else
            $error("WSTRB cannot be all-zeros when WVALID=1");
```

```
    end
end
```

Partial Writes: WSTRB controls byte lane enables

```
32-bit write with WSTRB=4'b0011:
  - Byte lanes [1:0] written
  - Byte lanes [3:2] unchanged
```

4.8 Related Modules

4.8.1 Companion Converters

- **axi4_dwidth_converter** - Full bidirectional converter (AW/W/B/AR/R)
- **axi4_dwidth_converter_rd** - Read-only data width conversion

4.8.2 Core Modules

- **axi4_master_wr** - AXI4 write master
- **axi4_slave_wr** - AXI4 write slave

4.8.3 Used Components

- **gaxi_skid_buffer** - Elastic buffering with timing closure

---

4.9 References

4.9.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- Chapter 11: Data Width Conversion

4.9.2 Source Code

- RTL: `rtl/amba/axi4/axi4_dwidth_converter_wr.sv`
- Tests: `val/amba/test_axi4_dwidth_converter_wr.py`
- Framework: `bin/TBClasses/components/axi4/`

4.9.3 Documentation

- Architecture: RTLAmba Overview
- AXI4 Index: README.md

Last Updated: 2025-10-20

4.10 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

5 AXI4 Master Read Interface (Clock-Gated)

Module: `axi4_master_rd_cg.sv` **Base Module:** axi4_master_rd **Location:** `rtl/amba/axi4/`
Status: √ Production Ready

5.1 Quick Reference

This is the **clock-gated variant** of axi4_master_rd.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **Clock-Gated Variants Guide**

5.2 Summary

The `axi4_master_rd_cg` module adds power optimization to `axi4_master_rd` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization
- √ **Configurable:** Idle threshold, gating domains, enable/disable
- √ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

5.3 Common Parameters

In addition to all axi4_master_rd parameters:

Parameter	Default	Description
ENABLE_CLOCK_GATING	1	Master enable (0=disable, identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables

5.4 Quick Usage

```
axi4_master_rd_cg #(
    // Base module parameters (see axi4_master_rd.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axi4_master_rd
);
```

5.5 Documentation

- **Base Module Functionality:** axi4_master_rd.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

5.6 Navigation

- ← Back to AXI4 Index
- ← Back to RTLAmba Index
- ← Back to Main Documentation Index

6 axi4_master_rd

An AXI4 read master module that provides high-performance read transaction processing with dual skid buffer architecture for optimal throughput and pipeline efficiency in AXI4 memory-mapped systems.

6.1 Overview

The `axi4_master_rd` module implements a complete AXI4 read master interface with integrated address and data channel buffering using GAXI skid buffers. It acts as a bridge between upstream AXI4 read requestors and downstream AXI4 memory interfaces, providing transaction buffering, pipeline optimization, and flow control to maximize memory subsystem performance.

6.2 Module Declaration

```
module axi4_master_rd #(
    parameter int SKID_DEPTH_AR     = 2,
    parameter int SKID_DEPTH_R      = 4,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1,
    parameter int AXI_WSTRB_WIDTH   = AXI_DATA_WIDTH / 8,
    // Short and calculated params
    parameter int AW       = AXI_ADDR_WIDTH,
    parameter int DW       = AXI_DATA_WIDTH,
    parameter int IW       = AXI_ID_WIDTH,
    parameter int SW       = AXI_WSTRB_WIDTH,
    parameter int UW       = AXI_USER_WIDTH,
    parameter int ARSize   = IW+AW+8+3+2+1+4+3+4+4+UW,
    parameter int RSize    = IW+DW+2+1+UW
) (
    // Global Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (Input Side) - FUB
    // Read address channel (AR)
    input  logic [IW-1:0]               fub_axi_arid,
```

```
input  logic [AW-1:0]               fub_axi_araddr,
input  logic [7:0]                  fub_axi_arlen,
input  logic [2:0]                  fub_axi_arsize,
input  logic [1:0]                  fub_axi_arburst,
input  logic                        fub_axi_arlock,
input  logic [3:0]                  fub_axi_arcache,
input  logic [2:0]                  fub_axi_arprot,
input  logic [3:0]                  fub_axi_arqos,
input  logic [3:0]                  fub_axi_arregion,
input  logic [UW-1:0]               fub_axi_aruser,
input  logic                        fub_axi_arvalid,
output logic                        fub_axi_arready,

// Read data channel (R)
output logic [IW-1:0]               fub_axi_rid,
output logic [DW-1:0]               fub_axi_rdata,
output logic [1:0]                  fub_axi_rresp,
output logic                        fub_axi_rlast,
output logic [UW-1:0]               fub_axi_ruser,
output logic                        fub_axi_rvalid,
input  logic                        fub_axi_rready,

// Master AXI Interface (Output Side)
// Read address channel (AR)
output logic [IW-1:0]               m_axi_arid,
output logic [AW-1:0]               m_axi_araddr,
output logic [7:0]                  m_axi_arlen,
output logic [2:0]                  m_axi_arsize,
output logic [1:0]                  m_axi_arburst,
output logic                        m_axi_arlock,
output logic [3:0]                  m_axi_arcache,
output logic [2:0]                  m_axi_arprot,
output logic [3:0]                  m_axi_arqos,
output logic [3:0]                  m_axi_arregion,
output logic [UW-1:0]               m_axi_aruser,
output logic                        m_axi_arvalid,
input  logic                        m_axi_arready,

// Read data channel (R)
input  logic [IW-1:0]               m_axi_rid,
input  logic [DW-1:0]               m_axi_rdata,
input  logic [1:0]                  m_axi_rresp,
input  logic                        m_axi_rlast,
input  logic [UW-1:0]               m_axi_ruser,
input  logic                        m_axi_rvalid,
output logic                        m_axi_rready,
```

```
    // Status outputs for clock gating
    output logic                        busy
);
```

6.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Address channel skid buffer depth (2^N entries)
SKID_DEPTH_R	int	4	Read data channel skid buffer depth (2^N entries)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	AXI write strobe width (calculated)

6.4 Ports

6.4.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI active-low reset

6.4.2 Slave AXI Interface (Input Side - FUB)

6.4.2.1 Read Address Channel (AR)

Port	Width	Direction	Description
fub_axi_arid	AXI_ID_WIDTH	Input	Read address ID
fub_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
fub_axi_arlen	8	Input	Burst length (0-

Port	Width	Direction	Description
			255)
fub_axi_arsize	3	Input	Transfer size (bytes per beat)
fub_axi_arburst	2	Input	Burst type (FIXED, INCR, WRAP)
fub_axi_arlock	1	Input	Lock type (atomic access)
fub_axi_arcache	4	Input	Cache attributes
fub_axi_arprot	3	Input	Protection attributes
fub_axi_arqos	4	Input	Quality of Service
fub_axi_arregion	4	Input	Region identifier
fub_axi_aruser	AXI_USER_WIDTH	Input	User-defined signals
fub_axi_arvalid	1	Input	Read address valid
fub_axi_arready	1	Output	Read address ready

6.4.2.2 Read Data Channel (R)

Port	Width	Direction	Description
fub_axi_rid	AXI_ID_WIDTH	Output	Read data ID
fub_axi_rdata	AXI_DATA_WIDTH	Output	Read data
fub_axi_rresp	2	Output	Read response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_rla	1	Output	Last read transfer in burst

Port	Width	Direction	Description
st			
fub_axi_ruser	AXI_USER_WIDTH	Output	User-defined signals
fub_axi_rvalid	1	Output	Read data valid
fub_axi_rready	1	Input	Read data ready

6.4.3 Master AXI Interface (Output Side)

6.4.3.1 Read Address Channel (AR)

Port	Width	Direction	Description
m_axi_arid	AXI_ID_WIDTH	Output	Read address ID
m_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
m_axi_arlen	8	Output	Burst length (0-255)
m_axi_arsize	3	Output	Transfer size (bytes per beat)
m_axi_arburst	2	Output	Burst type (FIXED, INCR, WRAP)
m_axi_arlock	1	Output	Lock type (atomic access)
m_axi_arcache	4	Output	Cache attributes
m_axi_arprot	3	Output	Protection attributes
m_axi_arqos	4	Output	Quality of Service
m_axi_arregion	4	Output	Region identifier

Port	Width	Direction	Description
m_axi_aruser	AXI_USER_WIDTH	Output	User-defined signals
m_axi_arvalid	1	Output	Read address valid
m_axi_arready	1	Input	Read address ready

6.4.3.2 Read Data Channel (R)

Port	Width	Direction	Description
m_axi_rid	AXI_ID_WIDTH	Input	Read data ID
m_axi_rdata	AXI_DATA_WIDTH	Input	Read data
m_axi_rresp	2	Input	Read response (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_rlast	1	Input	Last read transfer in burst
m_axi_ruser	AXI_USER_WIDTH	Input	User-defined signals
m_axi_rvalid	1	Input	Read data valid
m_axi_rready	1	Output	Read data ready

6.4.4 Status Interface

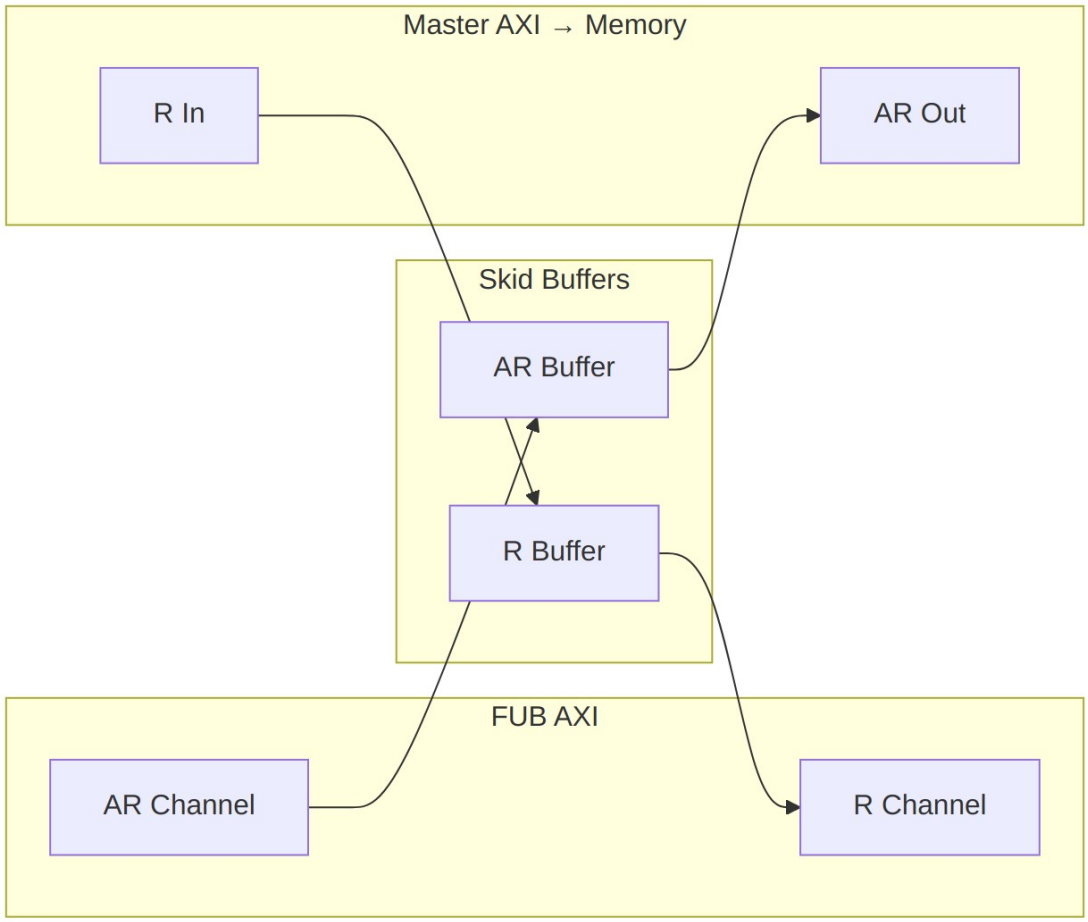
Port	Width	Direction	Description
busy	1	Output	Module activity indicator for clock gating

6.5 Architecture

6.5.1 Dual Buffer Design

The module employs a dual skid buffer architecture:

1. **AR Channel Buffer**: Buffers read address transactions
2. **R Channel Buffer**: Buffers read data responses



Data Flow

6.5.2 Key Features

- **Pipeline Optimization**: Dual buffering eliminates pipeline stalls
- **AXI4 Compliance**: Full AXI4 protocol support including all signals
- **Flow Control**: Independent buffering for address and data channels
- **Clock Gating Support**: Busy signal for power optimization
- **Burst Support**: Full burst transaction handling (1-256 beats)

- **ID Preservation**: Transaction ID forwarding and matching
- **Error Handling**: Proper error response propagation

6.6 Functionality

6.6.1 Address Channel Processing

1. **Address Capture**: Incoming address transactions buffered in AR skid buffer
2. **Address Forward**: Buffered addresses forwarded to master interface
3. **Flow Control**: Ready/valid handshaking manages buffer flow

6.6.2 Data Channel Processing

1. **Data Reception**: Read data from master interface buffered in R skid buffer
2. **Data Forward**: Buffered data forwarded to FUB interface
3. **Transaction Matching**: ID-based transaction correlation maintained

6.6.3 Busy Signal Generation

The busy output indicates module activity:

```
busy = (ar_buffer_count > 0) || (r_buffer_count > 0) ||
       fub_axi_arvalid || m_axi_rvalid;
```

6.7 Timing Characteristics

6.7.1 Buffer Latency

Buffer	Default Depth	Latency	Description
AR Channel	4 entries (DEPTH=2)	1 cycle	Address buffering
R Channel	16 entries (DEPTH=4)	1 cycle	Data buffering

6.7.2 Performance Metrics

Metric	Value	Conditions
Maximum Frequency	300-600 MHz	Technology dependent
Address Throughput	1 transaction/cycle	When buffers not full

Metric	Value	Conditions
Data Throughput	1 beat/cycle	When buffers not empty
Pipeline Efficiency	95-99%	With appropriate buffer sizing

6.7.3 Transaction Latency

Transaction Type	Latency	Description
Single Read	2-3 cycles	Address → Data (minimum)
Burst Read	2+N cycles	Address + N data beats
Back-to-back	1 cycle/txn	Sustained rate with buffering

6.8 Usage Examples

6.8.1 Basic AXI4 Read Master Configuration

```
axi4_master_rd #(
    .SKID_DEPTH_AR(2),        // 4-entry address buffer
    .SKID_DEPTH_R(4),         // 16-entry data buffer
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi4_rd_master (
    .aclk            (axi_clk),
    .aresetn         (axi_resetn),

    // Slave interface (from CPU/DMA)
    .fub_axi_arid      (cpu_axi_arid),
    .fub_axi_araddr    (cpu_axi_araddr),
    .fub_axi_arlen     (cpu_axi_arlen),
    .fub_axi_arsize    (cpu_axi_arsize),
    .fub_axi_arburst   (cpu_axi_arburst),
    .fub_axi_arlock    (cpu_axi_arlock),
    .fub_axi_arcache   (cpu_axi_arcache),
    .fub_axi_arprot    (cpu_axi_arprot),
    .fub_axi_arqos     (cpu_axi_arqos),
    .fub_axi_arregion  (cpu_axi_arregion),
    .fub_axi_aruser    (cpu_axi_aruser),
    .fub_axi_arvalid   (cpu_axi_arvalid),
```

```
    .fub_axi_arready   (cpu_axi_arready),

    .fub_axi_rid       (cpu_axi_rid),
    .fub_axi_rdata     (cpu_axi_rdata),
    .fub_axi_rresp     (cpu_axi_rresp),
    .fub_axi_rlast     (cpu_axi_rlast),
    .fub_axi_ruser     (cpu_axi_ruser),
    .fub_axi_rvalid    (cpu_axi_rvalid),
    .fub_axi_rready    (cpu_axi_rready),

    // Master interface (to memory)
    .m_axi_arid        (mem_axi_arid),
    .m_axi_araddr      (mem_axi_araddr),
    .m_axi_arlen       (mem_axi_arlen),
    .m_axi_arsize      (mem_axi_arsize),
    .m_axi_arburst     (mem_axi_arburst),
    .m_axi_arlock      (mem_axi_arlock),
    .m_axi_arcache     (mem_axi_arcache),
    .m_axi_arprot      (mem_axi_arprot),
    .m_axi_arqos       (mem_axi_arqos),
    .m_axi_arregion    (mem_axi_arregion),
    .m_axi_aruser      (mem_axi_aruser),
    .m_axi_arvalid     (mem_axi_arvalid),
    .m_axi_arready     (mem_axi_arready),

    .m_axi_rid         (mem_axi_rid),
    .m_axi_rdata       (mem_axi_rdata),
    .m_axi_rresp       (mem_axi_rresp),
    .m_axi_rlast       (mem_axi_rlast),
    .m_axi_ruser       (mem_axi_ruser),
    .m_axi_rvalid      (mem_axi_rvalid),
    .m_axi_rready      (mem_axi_rready),

    // Status for clock gating
    .busy              (rd_master_busy)
);
```

6.8.2 High-Performance Memory Interface

```
// High-performance configuration for DDR interface
axi4_master_rd #(
    .SKID_DEPTH_AR(3),        // 8-entry address buffer
    .SKID_DEPTH_R(5),         // 32-entry data buffer
    .AXI_ID_WIDTH(4),         // Reduced ID width for efficiency
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(128),     // Wide data path
    .AXI_USER_WIDTH(1)
) u_ddr_rd_master (
```

```
    .aclk            (ddr_clk),
    .aresetn         (ddr_resetn),

    // Connect to multiple read requestors via AXI interconnect
    .fub_axi_*(cpu_cluster_axi_*),

    // Connect to DDR controller
    .m_axi_*(ddr_ctrl_axi_*),

    .busy            (ddr_rd_busy)
);
```

6.8.3 Multi-Master System Integration

```
module axi_memory_system (
    input logic axi_clk,
    input logic axi_resetn,

    // CPU interface
    axi4_if.slave   cpu_axi,
    // DMA interface
    axi4_if.slave   dma_axi,
    // Memory interface
    axi4_if.master  mem_axi
);

    // Read masters for independent buffering
    axi4_master_rd #(
        .SKID_DEPTH_AR(2),
        .SKID_DEPTH_R(3),
        .AXI_ID_WIDTH(4),
        .AXI_DATA_WIDTH(64)
    ) u_cpu_rd_master (
        .aclk(axi_clk),
        .aresetn(axi_resetn),
        .fub_axi_*(cpu_axi.*),
        .m_axi_*(cpu_rd_axi.*),
        .busy(cpu_rd_busy)
    );

    axi4_master_rd #(
        .SKID_DEPTH_AR(3),
        .SKID_DEPTH_R(4),
        .AXI_ID_WIDTH(4),
        .AXI_DATA_WIDTH(64)
    ) u_dma_rd_master (
        .aclk(axi_clk),
```

```
        .aresetn(axi_resetn),
        .fub_axi_*(dma_axi.*),
        .m_axi_*(dma_rd_axi.*),
        .busy(dma_rd_busy)
    );

    // AXI interconnect for arbitration
    axi4_interconnect #(
        .NUM_MASTERS(2),
        .NUM_SLAVES(1)
    ) u_interconnect (
        .aclk(axi_clk),
        .aresetn(axi_resetn),
        .s_axi({cpu_rd_axi, dma_rd_axi}),
        .m_axi(mem_axi)
    );

endmodule
```

6.8.4 Clock Gating Integration

```
// Clock gated version for power optimization
axi4_master_rd_cg #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_DATA_WIDTH(32)
) u_rd_master_cg (
    .aclk            (axi_clk),
    .aresetn         (axi_resetn),

    // Standard AXI interfaces
    .fub_axi_*(fub_axi_*),
    .m_axi_*(m_axi_*),

    // Clock gating control
    .cg_enable       (rd_enable),
    .cg_test_enable  (scan_mode),
    .busy            (rd_busy)
);

// Use busy signal for system-level power management
always_ff @(posedge axi_clk) begin
    rd_power_gate <= !rd_busy && idle_counter > IDLE_THRESHOLD;
end
```

6.9 Performance Optimization

6.9.1 Buffer Depth Selection

Choose buffer depths based on system characteristics:

System Type	AR Depth	R Depth	Rationale
Low Latency CPU	2 (4 entries)	3 (8 entries)	Minimize latency
High Throughput DMA	3 (8 entries)	5 (32 entries)	Maximize bandwidth
DDR4 Interface	4 (16 entries)	6 (64 entries)	Handle refresh cycles
PCIe Interface	3 (8 entries)	4 (16 entries)	Variable latency tolerance

6.9.2 Burst Optimization

```
// Optimize for burst efficiency
localparam BURST_LENGTH = 16; // Optimize for cache line size

// Generate efficient burst addresses
always_comb begin
    axi_araddr = base_addr & ~(BURST_LENGTH * DATA_BYTES - 1); //
Align
    axi_arlen  = BURST_LENGTH - 1;                              //
Length
    axi_arsize = $clog2(DATA_BYTES);                            // Size
    axi_arburst = 2'b01;                                        // INCR
end
```

6.9.3 Quality of Service (QoS)

```
// QoS-aware read master configuration
always_comb begin
    case (requestor_type)
        CPU_CRITICAL:  axi_arqos = 4'hF;    // Highest priority
        CPU_NORMAL:    axi_arqos = 4'h8;    // Medium priority
        DMA_BULK:      axi_arqos = 4'h4;    // Lower priority
        BACKGROUND:    axi_arqos = 4'h1;    // Lowest priority
        default:       axi_arqos = 4'h0;
    endcase
end
```

6.10 Advanced Features

6.10.1 Transaction Monitoring

```
// Performance monitoring integration
logic [31:0] read_transaction_count;
logic [31:0] read_burst_total;
logic [31:0] read_latency_accumulator;

always_ff @(posedge axi_clk) begin
    // Count transactions
    if (fub_axi_arvalid && fub_axi_arready) begin
        read_transaction_count <= read_transaction_count + 1;
        read_burst_total <= read_burst_total + fub_axi_arlen + 1;
    end

    // Measure latency (simplified)
    if (fub_axi_rvalid && fub_axi_rready && fub_axi_rlast) begin
        read_latency_accumulator <= read_latency_accumulator +
measured_latency;
    end
end

// Average burst length and latency calculations
assign avg_burst_length = read_burst_total / read_transaction_count;
assign avg_latency = read_latency_accumulator /
read_transaction_count;
```

6.10.2 Error Handling and Recovery

```
// Enhanced error handling
logic error_detected;
logic [7:0] error_count;

always_ff @(posedge axi_clk) begin
    error_detected <= fub_axi_rvalid && fub_axi_rready &&
                      (fub_axi_rresp == 2'b10 || fub_axi_rresp ==
2'b11);

    if (error_detected) begin
        error_count <= error_count + 1;
        // Log error details for debugging
        error_log <= {fub_axi_rid, fub_axi_rresp, timestamp};
    end
end
```

6.11 Synthesis Considerations

6.11.1 Area Optimization

- Reduce buffer depths for area-constrained designs
- Use smaller ID widths when possible
- Share buffers across multiple masters if timing allows

6.11.2 Timing Optimization

- Register all interface signals for timing closure
- Use appropriate buffer depths to break critical paths
- Consider pipeline stages for very high-frequency operation

6.11.3 Power Optimization

- Use clock gating variant (`axi4_master_rd_cg`) when available
- Implement QoS-based power scaling
- Size buffers appropriately to minimize switching activity

6.12 Verification Notes

6.12.1 Protocol Compliance

- Verify AXI4 handshaking on all channels
- Check burst handling and RLAST generation
- Validate ID preservation through the pipeline

6.12.2 Buffer Verification

- Test buffer overflow/underflow protection
- Verify data integrity through buffers
- Check flow control under various load conditions

6.12.3 Performance Verification

- Measure sustained throughput with various patterns
- Verify buffer efficiency and utilization
- Check latency under different loading conditions

6.13 Related Modules

- **axi4_master_rd_cg**: Clock-gated version for power optimization

- **axi4_master_wr**: Complementary AXI4 write master
- **gaxi_skid_buffer**: Underlying buffer infrastructure
- **axi4_interconnect**: AXI4 interconnect for multi-master systems
- **axi_monitor_base**: AXI4 protocol monitoring and analysis

The `axi4_master_rd` module provides a complete, high-performance solution for AXI4 read master functionality with advanced buffering, full protocol compliance, and comprehensive system integration capabilities.

6.14 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

7 AXI4 Master Read Stub

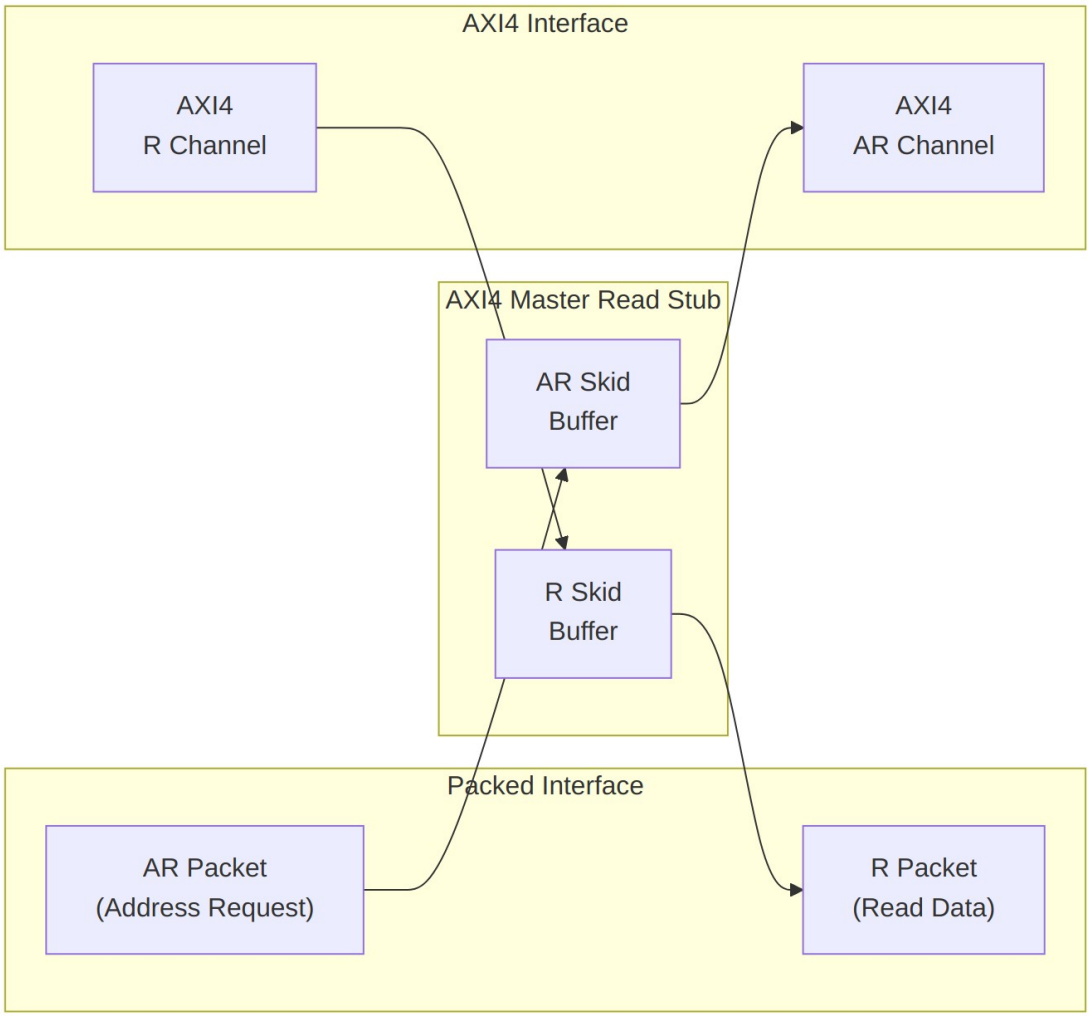
Module: `axi4_master_rd_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

7.1 Overview

The AXI4 Master Read Stub provides a simplified packed-data interface for driving AXI4 read transactions. It uses skid buffers to pack/unpack AXI4 AR (read address) and R (read data) channels into simple packet interfaces, making it ideal for testbenches and integration scenarios where a simplified interface is preferred.

7.1.1 Key Features

- Packed packet interface for AR and R channels
- Configurable skid buffer depths for each channel
- Full AXI4 read transaction support
- Burst, ID, user signal support
- Parameterized data widths



Module Architecture

7.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel skid buffer depth (log2)
SKID_DEPTH_R	int	4	R channel skid buffer depth (log2)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width

Parameter	Type	Default	Description
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (unused)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

7.3 Ports

7.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI reset (active low)

7.3.2 AXI4 Read Address Channel (AR)

Port	Width	Direction	Description
m_axi_arid	AXI_ID_WIDTH	Output	Read address ID
m_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
m_axi_arlen	8	Output	Burst length

Port	Width	Direction	Description
m_axi_arsize	3	Output	Burst size
m_axi_arburst	2	Output	Burst type
m_axi_arlock	1	Output	Lock type
m_axi_arcache	4	Output	Cache type
m_axi_arprot	3	Output	Protection type
m_axi_arqos	4	Output	Quality of service
m_axi_arregion	4	Output	Region identifier
m_axi_aruser	AXI_USER_WIDTH	Output	User signal
m_axi_arvalid	1	Output	Read address valid
m_axi_arready	1	Input	Read address ready

7.3.3 AXI4 Read Data Channel (R)

Port	Width	Direction	Description
m_axi_rid	AXI_ID_WIDTH	Input	Read data ID
m_axi_rdata	AXI_DATA_WIDTH	Input	Read data
m_axi_rresp	2	Input	Read response
m_axi_rlast	1	Input	Read last
m_axi_ruser	AXI_USER_WIDTH	Input	User signal
m_axi_rvalid	1	Input	Read data valid
m_axi_rready	1	Output	Read data ready

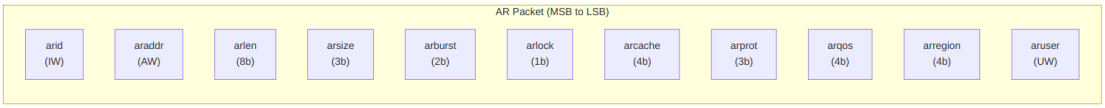
7.3.4 AR Packet Interface

Port	Width	Direction	Description
fub_axi_arvalid	1	Input	AR packet valid
fub_axi_arready	1	Output	Ready to accept AR packet
fub_axi_ar_count	3	Output	AR buffer occupancy
fub_axi_ar_pkt	ARSize	Input	Packed AR packet data

7.3.5 R Packet Interface

Port	Width	Direction	Description
fub_axi_rvalid	1	Output	R packet valid
fub_axi_rready	1	Input	Ready to accept R packet
fub_axi_r_pkt	RSize	Output	Packed R packet data

7.4 Packet Formats

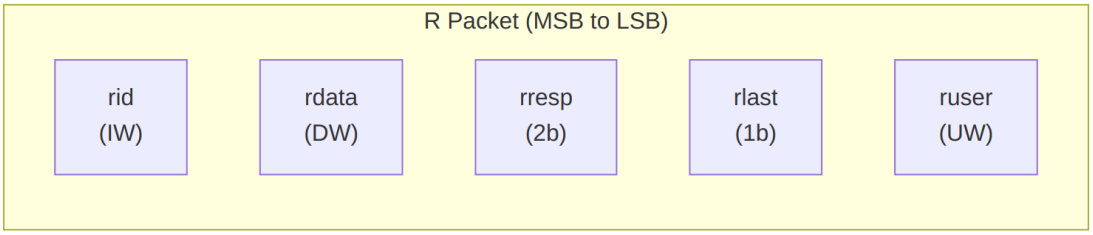


AR Packet Structure (Read Address)

Bit Positions:

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}

Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```



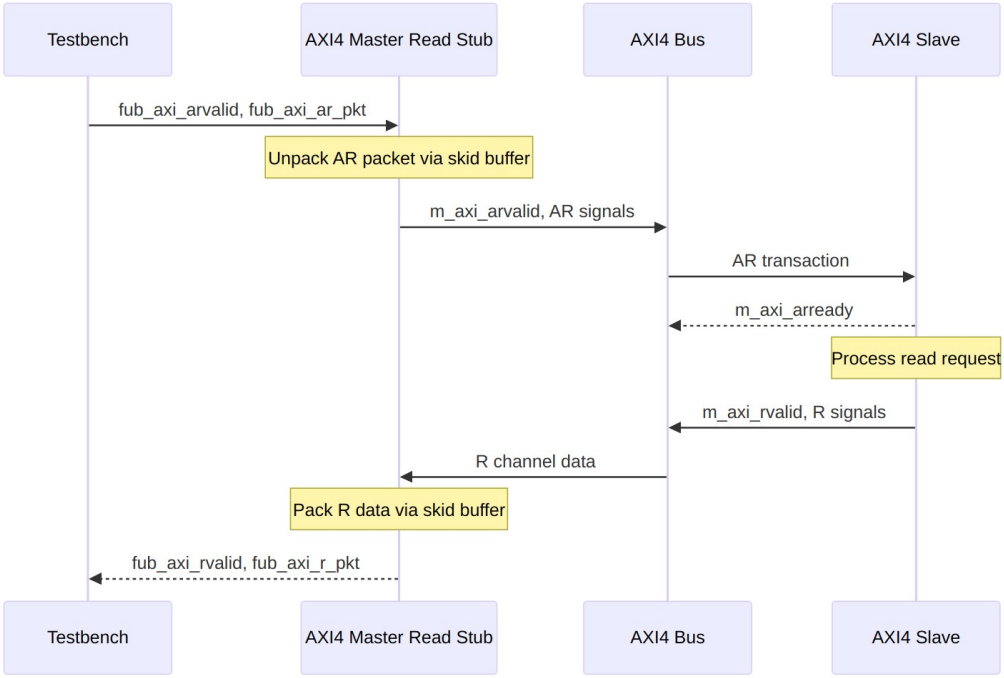
R Packet Structure (Read Data)

Bit Positions:

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}

Width = IW + DW + 2 + 1 + UW
```

7.5 Transaction Flow



Read Transaction

7.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- aclk
- fub_axi_arvalid, fub_axi_arready, fub_axi_ar_pkt
```

```
- AXI AR signals (m_axi_arvalid, m_axi_araddr, m_axi_arlen, etc.)
- AXI R signals (m_axi_rvalid, m_axi_rdata, m_axi_rlast, etc.)
- fub_axi_rvalid, fub_axi_rready, fub_axi_r_pkt
- Packet-to-AXI timing relationship with skid buffer operation
```

7.6 Usage Example

```
axi4_master_rd_stub #(
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_master_rd_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 master read interface
    .m_axi_arid      (m_axi_arid),
    .m_axi_araddr    (m_axi_araddr),
    .m_axi_arlen     (m_axi_arlen),
    .m_axi_arsize    (m_axi_arsize),
    .m_axi_arburst   (m_axi_arburst),
    .m_axi_arlock    (m_axi_arlock),
    .m_axi_arcache   (m_axi_arcache),
    .m_axi_arprot    (m_axi_arprot),
    .m_axi_arqos     (m_axi_arqos),
    .m_axi_arregion  (m_axi_arregion),
    .m_axi_aruser    (m_axi_aruser),
    .m_axi_arvalid   (m_axi_arvalid),
    .m_axi_arready   (m_axi_arready),

    .m_axi_rid       (m_axi_rid),
    .m_axi_rdata     (m_axi_rdata),
    .m_axi_rresp     (m_axi_rresp),
    .m_axi_rlast     (m_axi_rlast),
    .m_axi_ruser     (m_axi_ruser),
    .m_axi_rvalid    (m_axi_rvalid),
    .m_axi_rready    (m_axi_rready),

    // Packed AR interface
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
```

```
    .fub_axi_ar_pkt  (tb_ar_pkt),

    // Packed R interface
    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);

// Build AR packet (single beat read at address 0x1000)
localparam ARSize = 8 + 32 + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + 4;  //
Calculate size
assign tb_ar_pkt = {
    8'd0,           // arid
    32'h0000_1000,  // araddr
    8'd0,           // arlen (1 beat)
    3'b011,         // arsize (8 bytes)
    2'b01,          // arburst (INCR)
    1'b0,           // arlock
    4'b0011,        // arcache
    3'b000,         // arprot
    4'b0000,        // arqos
    4'b0000,        // arregion
    4'h0            // aruser
};

// Parse R packet
wire [7:0]  r_id   = tb_r_pkt[RSize-1:RSize-8];
wire [63:0] r_data = tb_r_pkt[RSize-9:RSize-72];
wire [1:0]  r_resp = tb_r_pkt[6:5];
wire        r_last = tb_r_pkt[4];
wire [3:0]  r_user = tb_r_pkt[3:0];
```

7.7 Design Notes

7.7.1 Skid Buffer Operation

The stub uses `gaxi_skid_buffer` modules to: - Decouple timing between testbench and AXI bus - Provide configurable buffering depth per channel - Handle backpressure gracefully - Support burst transactions without stalling

Recommended Depths: - **AR Channel:** 2-4 (address transactions) - **R Channel:** 4-8 (data beats for bursts)

7.7.2 Packet Packing Order

AR and R packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet creation - Matches common concatenation order - Efficient for burst transaction handling

7.7.3 Internal Architecture

The stub instantiates two `gaxi_skid_buffer` modules: - **AR Skid Buffer:** Unpacks AR packets to AXI AR channel - **R Skid Buffer:** Packs AXI R channel to R packets

All AXI protocol handling is done by the skid buffers and downstream modules.

7.8 Related Documentation

- **AXI4 Master Read** - Full AXI4 master read module (if wrapping one)
- **AXI4 Master Write Stub** - Corresponding write stub
- **AXI4 Master Stub** - Combined read/write stub
- **AXI4 Slave Read Stub** - Slave-side read stub

7.9 Navigation

- **<- Back to AXI4 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

8 AXI4 Master Stub

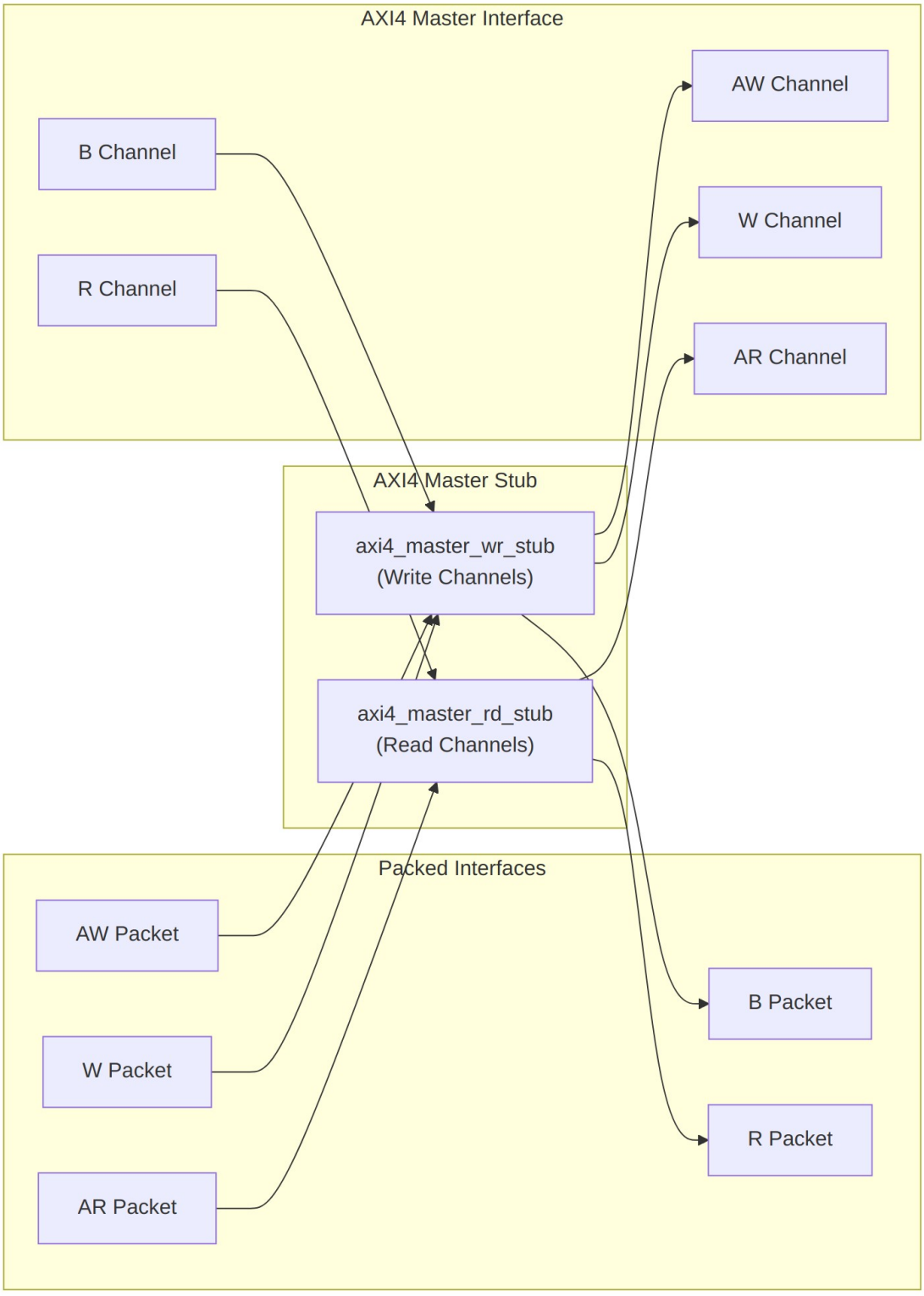
Module: `axi4_master_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

8.1 Overview

The AXI4 Master Stub provides a simplified packed-data interface for driving both AXI4 read and write transactions. It combines the read and write stub functionalities into a single module, internally instantiating `axi4_master_rd_stub` and `axi4_master_wr_stub`. This provides a complete AXI4 master interface with simplified packet-based control, ideal for testbenches and integration scenarios.

8.1.1 Key Features

- Combined read and write channel support
- Packed packet interfaces for all channels (AW, W, B, AR, R)
- Configurable skid buffer depths per channel
- Full AXI4 protocol support (bursts, IDs, user signals)
- Simplified testbench integration
- Parameterized data widths



Module Architecture

8.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth (log2)
SKID_DEPTH_W	int	4	W channel skid buffer depth (log2)
SKID_DEPTH_B	int	2	B channel skid buffer depth (log2)
SKID_DEPTH_AR	int	2	AR channel skid buffer depth (log2)
SKID_DEPTH_R	int	4	R channel skid buffer depth (log2)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)

Parameter	Type	Default	Description
BSize	int	IW+2+UW	B packet size (calculated)
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

8.3 Ports

8.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI reset (active low)

8.3.2 AXI4 Write Channels

Port	Width	Direction	Description
m_axi_awid	AXI_ID_WIDTH	Output	Write address ID
m_axi_awaddr	AXI_ADDR_WIDTH	Output	Write address
m_axi_awlen	8	Output	Burst length
m_axi_awsize	3	Output	Burst size
m_axi_awburst	2	Output	Burst type
m_axi_awlock	1	Output	Lock type
m_axi_awcache	4	Output	Cache type
m_axi_awprot	3	Output	Protection type
m_axi_awqos	4	Output	Quality of service
m_axi_awregion	4	Output	Region identifier
m_axi_awuser	AXI_USER_WID	Output	User signal

Port	Width	Direction	Description
	TH		
m_axi_awvalid	1	Output	Write address valid
m_axi_awready	1	Input	Write address ready
m_axi_wdata	AXI_DATA_WIDTH	Output	Write data
m_axi_wstrb	AXI_WSTRB_WIDTH	Output	Write strobes
m_axi_wlast	1	Output	Write last
m_axi_wuser	AXI_USER_WIDTH	Output	User signal
m_axi_wvalid	1	Output	Write data valid
m_axi_wready	1	Input	Write data ready
m_axi_bid	AXI_ID_WIDTH	Input	Response ID
m_axi_bresp	2	Input	Write response
m_axi_buser	AXI_USER_WIDTH	Input	User signal
m_axi_bvalid	1	Input	Write response valid
m_axi_bready	1	Output	Write response ready

8.3.3 AXI4 Read Channels

Port	Width	Direction	Description
m_axi_arid	AXI_ID_WIDTH	Output	Read address ID
m_axi_araddr	AXI_ADDR_WIDTH	Output	Read address
m_axi_arlen	8	Output	Burst length
m_axi_arsize	3	Output	Burst size

Port	Width	Direction	Description
m_axi_arburst	2	Output	Burst type
m_axi_arlock	1	Output	Lock type
m_axi_arcache	4	Output	Cache type
m_axi_arprot	3	Output	Protection type
m_axi_arqos	4	Output	Quality of service
m_axi_arregion	4	Output	Region identifier
m_axi_aruser	AXI_USER_WIDTH	Output	User signal
m_axi_arvalid	1	Output	Read address valid
m_axi_arready	1	Input	Read address ready
m_axi_rid	AXI_ID_WIDTH	Input	Read data ID
m_axi_rdata	AXI_DATA_WIDTH	Input	Read data
m_axi_rresp	2	Input	Read response
m_axi_rlast	1	Input	Read last
m_axi_ruser	AXI_USER_WIDTH	Input	User signal
m_axi_rvalid	1	Input	Read data valid
m_axi_rready	1	Output	Read data ready

8.3.4 Write Packet Interfaces

Port	Width	Direction	Description
fub_axi_awvalid	1	Input	AW packet valid
fub_axi_awready	1	Output	Ready to accept AW packet

Port	Width	Direction	Description
fub_axi_aw_count	4	Output	AW buffer occupancy
fub_axi_aw_pkt	AWSize	Input	Packed AW packet data
fub_axi_wvalid	1	Input	W packet valid
fub_axi_wready	1	Output	Ready to accept W packet
fub_axi_w_pkt	WSize	Input	Packed W packet data
fub_axi_bvalid	1	Output	B packet valid
fub_axi_bready	1	Input	Ready to accept B packet
fub_axi_b_pkt	BSize	Output	Packed B packet data

8.3.5 Read Packet Interfaces

Port	Width	Direction	Description
fub_axi_arvalid	1	Input	AR packet valid
fub_axi_arready	1	Output	Ready to accept AR packet
fub_axi_ar_count	3	Output	AR buffer occupancy
fub_axi_ar_pkt	ARSize	Input	Packed AR packet data
fub_axi_rvalid	1	Output	R packet valid
fub_axi_rready	1	Input	Ready to accept R packet
fub_axi_r_pkt	RSize	Output	Packed R

Port	Width	Direction	Description
			packet data

8.4 Packet Formats

8.4.1 Write Channel Packets

AW Packet (Write Address):

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

W Packet (Write Data):

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}
Width = DW + SW + 1 + UW
```

B Packet (Write Response):

```
fub_axi_b_pkt = {bid, bresp, buser}
Width = IW + 2 + UW
```

8.4.2 Read Channel Packets

AR Packet (Read Address):

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

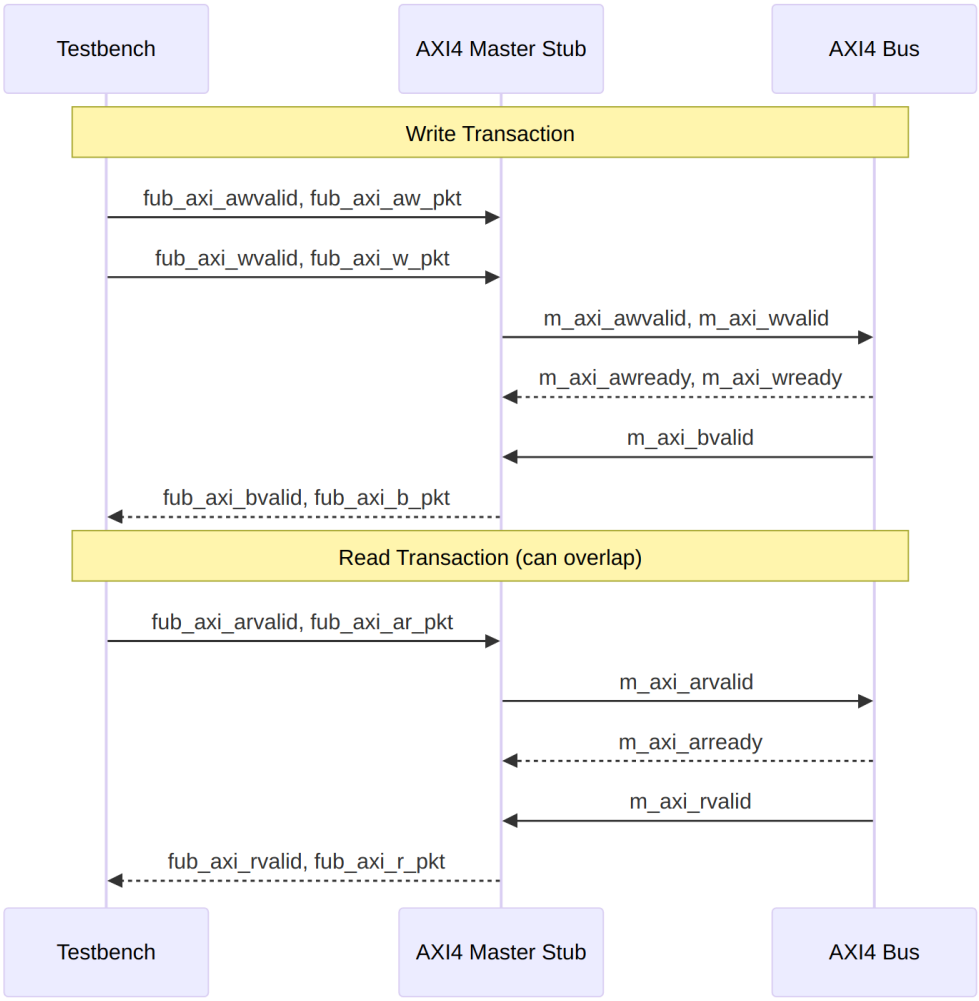
R Packet (Read Data):

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}
Width = IW + DW + 2 + 1 + UW
```

Complete packet format details: See AXI4 Master Write Stub and AXI4 Master Read Stub

---

8.5 Transaction Flow



Combined Read and Write Operations

8.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- aclk
- Write packet interfaces (fub_axi_aw*, fub_axi_w*, fub_axi_b*)
- Read packet interfaces (fub_axi_ar*, fub_axi_r*)
- AXI write channels (m_axi_aw*, m_axi_w*, m_axi_b*)
- AXI read channels (m_axi_ar*, m_axi_r*)
- Overlapping read/write operations
```

8.6 Usage Example

```
axi4_master_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_master_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 master interface (all channels)
    .m_axi_awid      (m_axi_awid),
    .m_axi_awaddr    (m_axi_awaddr),
    .m_axi_awlen     (m_axi_awlen),
    .m_axi_awsize    (m_axi_awsize),
    .m_axi_awburst   (m_axi_awburst),
    .m_axi_awlock    (m_axi_awlock),
    .m_axi_awcache   (m_axi_awcache),
    .m_axi_awprot    (m_axi_awprot),
    .m_axi_awqos     (m_axi_awqos),
    .m_axi_awregion  (m_axi_awregion),
    .m_axi_awuser    (m_axi_awuser),
    .m_axi_awvalid   (m_axi_awvalid),
    .m_axi_awready   (m_axi_awready),

    .m_axi_wdata     (m_axi_wdata),
    .m_axi_wstrb     (m_axi_wstrb),
    .m_axi_wlast     (m_axi_wlast),
    .m_axi_wuser     (m_axi_wuser),
    .m_axi_wvalid    (m_axi_wvalid),
    .m_axi_wready    (m_axi_wready),

    .m_axi_bid       (m_axi_bid),
    .m_axi_bresp     (m_axi_bresp),
    .m_axi_buser     (m_axi_buser),
    .m_axi_bvalid    (m_axi_bvalid),
    .m_axi_bready    (m_axi_bready),

    .m_axi_arid      (m_axi_arid),
    .m_axi_araddr    (m_axi_araddr),
    .m_axi_arlen     (m_axi_arlen),
```

```
    .m_axi_arsize    (m_axi_arsize),
    .m_axi_arburst   (m_axi_arburst),
    .m_axi_arlock    (m_axi_arlock),
    .m_axi_arcache   (m_axi_arcache),
    .m_axi_arprot    (m_axi_arprot),
    .m_axi_arqos     (m_axi_arqos),
    .m_axi_arregion  (m_axi_arregion),
    .m_axi_aruser    (m_axi_aruser),
    .m_axi_arvalid   (m_axi_arvalid),
    .m_axi_arready   (m_axi_arready),

    .m_axi_rid       (m_axi_rid),
    .m_axi_rdata     (m_axi_rdata),
    .m_axi_rresp     (m_axi_rresp),
    .m_axi_rlast     (m_axi_rlast),
    .m_axi_ruser     (m_axi_ruser),
    .m_axi_rvalid    (m_axi_rvalid),
    .m_axi_rready    (m_axi_rready),

    // Write packet interfaces
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt),

    // Read packet interfaces
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);

// Example: Build write transaction packets
assign tb_aw_pkt = {
```

```
    8'd1,           // awid
    32'h1000_0000,  // awaddr
    8'd0,           // awlen (1 beat)
    3'b011,         // awsize (8 bytes)
    2'b01,          // awburst (INCR)
    1'b0,           // awlock
    4'b0011,        // awcache
    3'b000,         // awprot
    4'b0000,        // awqos
    4'b0000,        // awregion
    4'h0            // awuser
};

assign tb_w_pkt = {
    64'hDEAD_BEEF_CAFE_BABE,  // wdata
    8'hFF,                    // wstrb
    1'b1,                     // wlast
    4'h0                      // wuser
};

// Example: Build read transaction packet
assign tb_ar_pkt = {
    8'd2,           // arid
    32'h2000_0000,  // araddr
    8'd3,           // arlen (4 beats)
    3'b011,         // arsize (8 bytes)
    2'b01,          // arburst (INCR)
    1'b0,           // arlock
    4'b0011,        // arcache
    3'b000,         // arprot
    4'b0000,        // arqos
    4'b0000,        // arregion
    4'h0            // aruser
};

// Parse responses
wire [7:0] b_id   = tb_b_pkt[BSize-1:BSize-8];
wire [1:0] b_resp = tb_b_pkt[5:4];

wire [7:0]  r_id   = tb_r_pkt[RSize-1:RSize-8];
wire [63:0] r_data = tb_r_pkt[RSize-9:RSize-72];
wire [1:0]  r_resp = tb_r_pkt[6:5];
wire        r_last = tb_r_pkt[4];
```

8.7 Design Notes

8.7.1 Internal Architecture

The stub instantiates two sub-modules: - **`axi4_master_wr_stub`** - Handles AW, W, and B channels - **`axi4_master_rd_stub`** - Handles AR and R channels

This hierarchical design: - Reuses proven read/write stub modules - Maintains clear channel separation - Simplifies verification and testing - Allows independent read/write operation

8.7.2 Read/Write Independence

Read and write channels are completely independent: - Can overlap in time (simultaneous read and write) - Each has independent skid buffers - No ordering constraints between reads and writes - Testbench can drive at different rates

8.7.3 Skid Buffer Depths

Recommended configurations:

Low latency (fast memory):

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(2), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(2)
```

Typical system:

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(4), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(4)
```

High throughput bursts:

```
.SKID_DEPTH_AW(4), .SKID_DEPTH_W(8), .SKID_DEPTH_B(4),
.SKID_DEPTH_AR(4), .SKID_DEPTH_R(8)
```

8.8 Related Documentation

- **AXI4 Master Read Stub** - Read-only stub (instantiated internally)
- **AXI4 Master Write Stub** - Write-only stub (instantiated internally)
- **AXI4 Slave Stub** - Corresponding combined slave stub
- **AXI4 Master Read** - Full AXI4 master read module
- **AXI4 Master Write** - Full AXI4 master write module

8.9 Navigation

- **<- Back to AXI4 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

9 AXI4 Master Write Interface (Clock-Gated)

Module: `axi4_master_wr_cg.sv` **Base Module:** axi4_master_wr **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

---

9.1 Quick Reference

This is the **clock-gated variant** of axi4_master_wr.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **Clock-Gated Variants Guide**

9.2 Summary

The `axi4_master_wr_cg` module adds power optimization to `axi4_master_wr` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization
- √ **Configurable:** Idle threshold, gating domains, enable/disable
- √ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

9.3 Common Parameters

In addition to all axi4_master_wr parameters:

Parameter	Default	Description
`ENABLE_CLOCK_GATING`	1	Master enable (0=disable,

Parameter	Default	Description
		identical to base)
CG_IDLE_CYCLES	8	Cycles before clock gating activates
CG_GATE_*	1	Domain-specific gating enables

9.4 Quick Usage

```
axi4_master_wr_cg #(
    // Base module parameters (see axi4_master_wr.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axi4_master_wr
);
```

9.5 Documentation

- **Base Module Functionality:** axi4_master_wr.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

9.6 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**

- ← Back to Main Documentation Index

10 AXI4 Master Write

Module: `axi4_master_wr.sv` **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

10.1 Overview

The AXI4 Master Write module provides a buffered AXI4 master write interface with configurable depth skid buffers on each write channel. This module serves as an elastic buffer between a write initiator and an AXI4 interconnect, decoupling timing and providing backpressure handling.

10.1.1 Key Features

- √ **Full AXI4 Write Support:** Complete AW, W, and B channel implementation
- √ **Independent Channel Buffering:** Separate configurable depth buffers for each channel
- √ **Elastic Buffering:** Decouples upstream and downstream timing domains
- √ **Burst Support:** Full burst transaction handling with WLAST tracking
- √ **User Signal Support:** Carries AWUSER, WUSER, and BUSER signals
- √ **Clock Gating Support:** Busy signal for dynamic power management

---

10.2 Module Interface

```
module axi4_master_wr #(
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
    parameter int SKID_DEPTH_B      = 2,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1
) (
    // Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Frontend AXI Interface (fub_axi_*)
    // Write address channel (AW)
    input  logic [AXI_ID_WIDTH-1:0]   fub_axi_awid,
```

```
input  logic [AXI_ADDR_WIDTH-1:0] fub_axi_awaddr,
input  logic [7:0]                fub_axi_awlen,
input  logic [2:0]                fub_axi_awsize,
input  logic [1:0]                fub_axi_awburst,
input  logic                      fub_axi_awlock,
input  logic [3:0]                fub_axi_awcache,
input  logic [2:0]                fub_axi_awprot,
input  logic [3:0]                fub_axi_awqos,
input  logic [3:0]                fub_axi_awregion,
input  logic [AXI_USER_WIDTH-1:0] fub_axi_awuser,
input  logic                      fub_axi_awvalid,
output logic                      fub_axi_awready,

// Write data channel (W)
input  logic [AXI_DATA_WIDTH-1:0] fub_axi_wdata,
input  logic [AXI_DATA_WIDTH/8-1:0] fub_axi_wstrb,
input  logic                      fub_axi_wlast,
input  logic [AXI_USER_WIDTH-1:0] fub_axi_wuser,
input  logic                      fub_axi_wvalid,
output logic                      fub_axi_wready,

// Write response channel (B)
output logic [AXI_ID_WIDTH-1:0]   fub_axi_bid,
output logic [1:0]                fub_axi_bresp,
output logic [AXI_USER_WIDTH-1:0] fub_axi_buser,
output logic                      fub_axi_bvalid,
input  logic                      fub_axi_bready,

// Master AXI Interface (m_axi_*)
// Write address channel (AW)
output logic [AXI_ID_WIDTH-1:0]   m_axi_awid,
output logic [AXI_ADDR_WIDTH-1:0] m_axi_awaddr,
output logic [7:0]                m_axi_awlen,
output logic [2:0]                m_axi_awsize,
output logic [1:0]                m_axi_awburst,
output logic                      m_axi_awlock,
output logic [3:0]                m_axi_awcache,
output logic [2:0]                m_axi_awprot,
output logic [3:0]                m_axi_awqos,
output logic [3:0]                m_axi_awregion,
output logic [AXI_USER_WIDTH-1:0] m_axi_awuser,
output logic                      m_axi_awvalid,
input  logic                      m_axi_awready,

// Write data channel (W)
output logic [AXI_DATA_WIDTH-1:0] m_axi_wdata,
output logic [AXI_DATA_WIDTH/8-1:0] m_axi_wstrb,
```

```
    output logic                          m_axi_wlast,
    output logic [AXI_USER_WIDTH-1:0] m_axi_wuser,
    output logic                          m_axi_wvalid,
    input  logic                          m_axi_wready,

    // Write response channel (B)
    input  logic [AXI_ID_WIDTH-1:0]   m_axi_bid,
    input  logic [1:0]                    m_axi_bresp,
    input  logic [AXI_USER_WIDTH-1:0] m_axi_buser,
    input  logic                          m_axi_bvalid,
    output logic                          m_axi_bready,

    // Status
    output logic                          busy
);
```

10.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	Depth of write address (AW) channel skid buffer
SKID_DEPTH_W	int	4	Depth of write data (W) channel skid buffer
SKID_DEPTH_B	int	2	Depth of write response (B) channel skid buffer
AXI_ID_WIDTH	int	8	Width of transaction ID signals (AWID, BID)
AXI_ADDR_WIDTH	int	32	Width of address bus (AWADDR)
AXI_DATA_WIDTH	int	32	Width of data bus (WDATA), must be 8, 16, 32, 64, 128, 256, 512, or 1024
AXI_USER_WIDTH	int	1	Width of user-defined signals (AWUSER, WUSER, BUSER)

10.4 Port Groups

10.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock - all signals sampled on rising edge
aresetn	Input	1	Active-low asynchronous reset

10.4.2 Frontend AXI Interface (fub_axi_*)

Write Address Channel (AW)

Port	Direction	Width	Description
fub_axi_awid	Input	AXI_ID_WIDTH	Write transaction ID
fub_axi_awaddr	Input	AXI_ADDR_WIDTH	Write address
fub_axi_awlen	Input	8	Burst length (0-255 beats)
fub_axi_awsize	Input	3	Burst size (bytes per beat)
fub_axi_awburst	Input	2	Burst type (FIXED, INCR, WRAP)
fub_axi_awlock	Input	1	Lock type (atomic access support)
fub_axi_awcache	Input	4	Cache attributes
fub_axi_awprot	Input	3	Protection attributes
fub_axi_awqos	Input	4	Quality of

Port	Direction	Width	Description
			Service identifier
fub_axi_awregion	Input	4	Region identifier
fub_axi_awuser	Input	AXI_USER_WIDTH	User-defined signal
fub_axi_awvalid	Input	1	Write address valid
fub_axi_awready	Output	1	Write address ready

Write Data Channel (W)

Port	Direction	Width	Description
fub_axi_wdata	Input	AXI_DATA_WIDTH	Write data
fub_axi_wstrb	Input	AXI_DATA_WIDTH/8	Write strobes (byte enables)
fub_axi_wlast	Input	1	Last beat of burst indicator
fub_axi_wuser	Input	AXI_USER_WIDTH	User-defined signal
fub_axi_wvalid	Input	1	Write data valid
fub_axi_wready	Output	1	Write data ready

Write Response Channel (B)

Port	Direction	Width	Description
fub_axi_bid	Output	AXI_ID_WIDTH	Response transaction ID
fub_axi_bresp	Output	2	Write response (OKAY, EXOKAY, SLVERR, DECERR)
fub_axi_buser	Output	AXI_USER_WIDTH	User-defined signal
fub_axi_b	Output	1	Write response valid

Port	Direction	Width	Description
`valid`			
`fub_axi_bready`	Input	1	Write response ready

10.4.3 Master AXI Interface (m_axi_*)

Mirrors the frontend interface but in the opposite direction (output on AW/W, input on B).

10.4.4 Status Outputs

Port	Direction	Width	Description
`busy`	Output	1	Indicates active transactions in buffers (for clock gating)

10.5 Functional Description

10.5.1 Architecture

The AXI4 Master Write module uses three independent `gaxi_skid_buffer` instances to provide elastic buffering on each write channel:

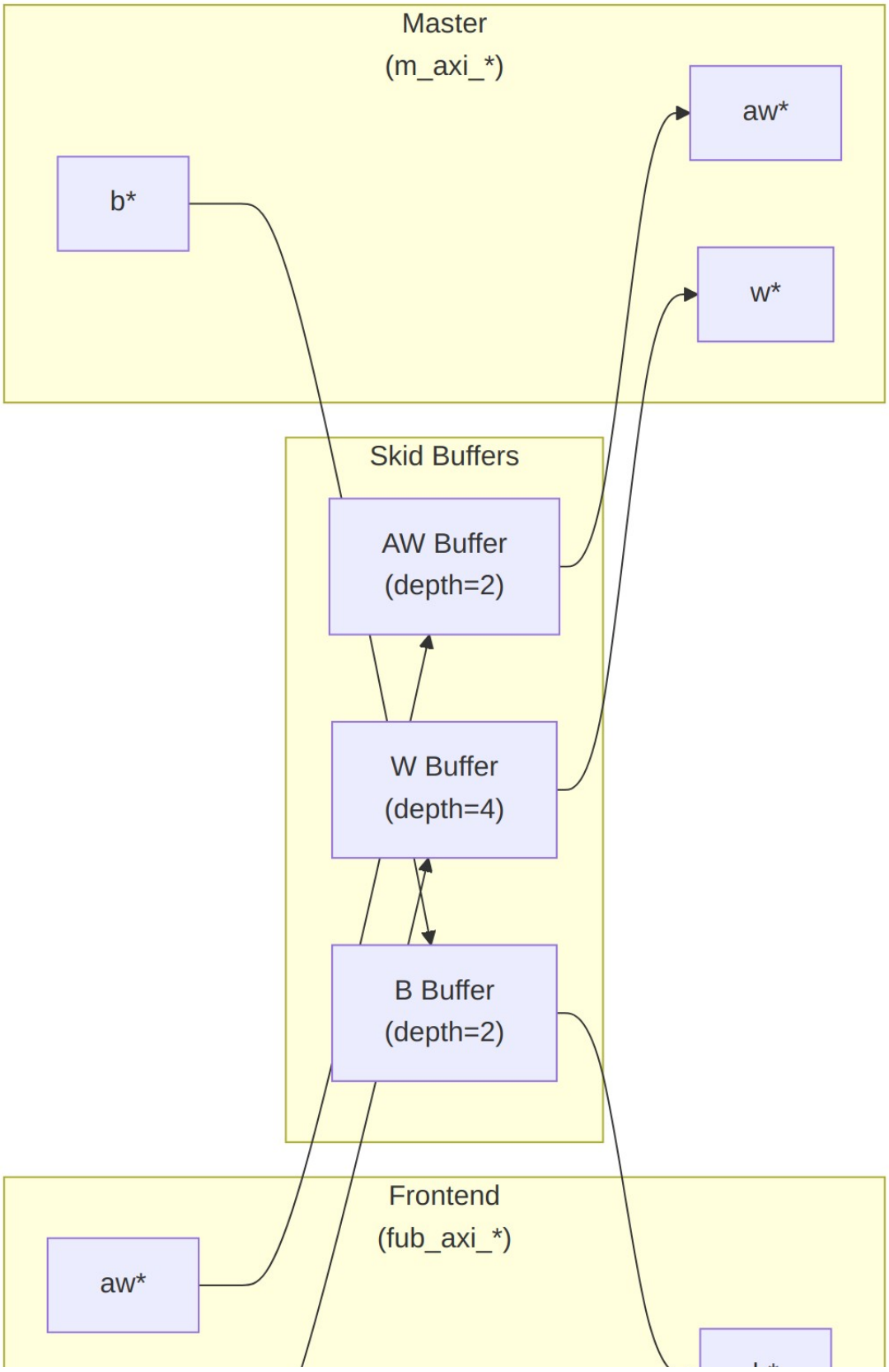

Master
(m_axi_*)
aw*
b*
w*
Skid Buffers
AW Buffer
(depth=2)
W Buffer
(depth=4)
B Buffer
(depth=2)
Frontend
(fub_axi_*)
aw*

Diagram 15

10.5.2 Channel Operations

10.5.2.1 Write Address (AW) Channel

1. Frontend presents write address and attributes
2. AW skid buffer accepts when space available (`fub_axi_awready` high)
3. Buffered address presented to master interface when ready
4. Configurable depth (`SKID_DEPTH_AW`) smooths timing variations

10.5.2.2 Write Data (W) Channel

1. Frontend presents write data with strobes
2. W skid buffer provides deeper buffering (default depth=4)
3. WLAST signal preserved to indicate burst boundaries
4. Independent from AW channel (AXI4 allows W before AW)

10.5.2.3 Write Response (B) Channel

1. Master returns write response with transaction ID
2. B skid buffer captures response when frontend busy
3. Frontend receives response when ready to accept
4. ID matching handled by AXI interconnect (not this module)

10.5.3 Busy Signal

The `busy` output indicates active transactions:

```
busy = (aw_count > 0) || (w_count > 0) || (b_count > 0) ||
       fub_axi_awvalid || fub_axi_wvalid || m_axi_bvalid
```

Use cases: - **Clock gating:** Disable clock when `busy` is low - **Power management:** Enter low-power mode when idle - **Synchronization:** Wait for idle before configuration changes

---

10.6 Configuration Guidelines

10.6.1 Buffer Depth Selection

Write Address (SKID_DEPTH_AW): - Default: 2 (sufficient for most cases) - Increase if: - High-latency address decode paths - Frequent address channel backpressure - Multiple outstanding bursts needed

Write Data (SKID_DEPTH_W): - Default: 4 (deeper than AW for burst data) - Increase if: - Large burst sizes (AWLEN > 4) - High-bandwidth streaming writes - Significant W channel backpressure

Write Response (SKID_DEPTH_B): - Default: 2 (responses are typically single-beat) - Increase if: - Frontend slow to accept responses - Many concurrent outstanding writes - Response channel backpressure observed

10.6.2 Recommended Configurations

Low-Latency System (single outstanding write):

```
axi4_master_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_master_wr ( ... );
```

High-Throughput Streaming (burst writes):

```
axi4_master_wr #(
    .SKID_DEPTH_AW(4),
    .SKID_DEPTH_W(16),   // Deep for burst data
    .SKID_DEPTH_B(4)
) u_master_wr ( ... );
```

Multiple Outstanding Writes:

```
axi4_master_wr #(
    .SKID_DEPTH_AW(8),
    .SKID_DEPTH_W(16),
    .SKID_DEPTH_B(8)
) u_master_wr ( ... );
```

10.7 Usage Example

10.7.1 Basic Integration

```
// Instantiate AXI4 master write buffer
axi4_master_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi_master_wr (
    .aclk               (axi_aclk),
```

```
    .aresetn            (axi_aresetn),

    // Frontend interface (from write initiator)
    .fub_axi_awid       (write_awid),
    .fub_axi_awaddr     (write_awaddr),
    .fub_axi_awlen      (write_awlen),
    .fub_axi_awsize     (write_awsize),
    .fub_axi_awburst    (write_awburst),
    .fub_axi_awlock     (1'b0),
    .fub_axi_awcache    (4'b0010),
    .fub_axi_awprot     (3'b000),
    .fub_axi_awqos      (4'b0000),
    .fub_axi_awregion   (4'b0000),
    .fub_axi_awuser     (1'b0),
    .fub_axi_awvalid    (write_awvalid),
    .fub_axi_awready    (write_awready),

    .fub_axi_wdata      (write_wdata),
    .fub_axi_wstrb      (write_wstrb),
    .fub_axi_wlast      (write_wlast),
    .fub_axi_wuser      (1'b0),
    .fub_axi_wvalid     (write_wvalid),
    .fub_axi_wready     (write_wready),

    .fub_axi_bid        (write_bid),
    .fub_axi_bresp      (write_bresp),
    .fub_axi_buser      (write_buser),
    .fub_axi_bvalid     (write_bvalid),
    .fub_axi_bready     (write_bready),

    // Master interface (to AXI interconnect)
    .m_axi_awid         (m_axi_awid),
    .m_axi_awaddr       (m_axi_awaddr),
    .m_axi_awlen        (m_axi_awlen),
    .m_axi_awsize       (m_axi_awsize),
    .m_axi_awburst      (m_axi_awburst),
    .m_axi_awlock       (m_axi_awlock),
    .m_axi_awcache      (m_axi_awcache),
    .m_axi_awprot       (m_axi_awprot),
    .m_axi_awqos        (m_axi_awqos),
    .m_axi_awregion     (m_axi_awregion),
    .m_axi_awuser       (m_axi_awuser),
    .m_axi_awvalid      (m_axi_awvalid),
    .m_axi_awready      (m_axi_awready),

    .m_axi_wdata        (m_axi_wdata),
    .m_axi_wstrb        (m_axi_wstrb),
```

```
    .m_axi_wlast        (m_axi_wlast),
    .m_axi_wuser        (m_axi_wuser),
    .m_axi_wvalid       (m_axi_wvalid),
    .m_axi_wready       (m_axi_wready),

    .m_axi_bid          (m_axi_bid),
    .m_axi_bresp        (m_axi_bresp),
    .m_axi_buser        (m_axi_buser),
    .m_axi_bvalid       (m_axi_bvalid),
    .m_axi_bready       (m_axi_bready),

    // Status
    .busy               (wr_master_busy)
);
```

10.7.2 Clock Gating Example

```
// Use busy signal for clock gating
logic axi_clk_gated;

clock_gate_ctrl u_cg (
    .i_clk          (axi_clk),
    .i_enable       (wr_master_busy),
    .o_clk_gated    (axi_clk_gated)
);

// Connect module to gated clock
axi4_master_wr #( ... ) u_master_wr (
    .aclk (axi_clk_gated),
    ...
);
```

10.8 Design Notes

10.8.1 Buffer Independence

The three skid buffers operate independently: - AW and W channels can progress at different rates - AXI4 allows W data before AW address - B responses return asynchronously based on slave timing

10.8.2 Packet Preservation

All signal groups packed and unpacked atomically: - AW channel: `{awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser}` - W channel: `{wdata, wstrb, wlast, wuser}` - B channel: `{bid, bresp, buser}`

10.8.3 Backpressure Handling

Ready signals propagate backpressure: - Frontend sees `awready/wready` low when buffers full - Master backpressure captured in buffers - Prevents data loss while decoupling timing

10.8.4 Reset Behavior

On `aresetn` assertion (active-low): - All skid buffers flush - Valid signals deasserted - Busy signal goes low - No data retained across reset

---

10.9 Related Modules

10.9.1 Companion Modules

- **axi4_master_rd** - AXI4 master read with buffering
- **axi4_master_wr_cg** - Clock-gated variant with additional CG logic
- **axi4_master_wr_mon** - Write transaction monitor for verification

10.9.2 Used Components

- **gaxi_skid_buffer** - Elastic buffer with valid/ready handshake
- **clock_gate_ctrl** - Clock gating control (example)

10.9.3 Related Infrastructure

- **axi4_interconnect** - Multi-master/multi-slave crossbar
- **axi4_slave_wr** - Corresponding AXI4 write slave module

10.10 References

10.10.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)

10.10.2 Source Code

- RTL: `rtl/amba/axi4/axi4_master_wr.sv`
- Tests: `val/amba/test_axi4_master_wr.py`
- Framework: `bin/TBClasses/components/axi4/`

10.10.3 Documentation

- Architecture: RTLAmba Overview

- AXI4 Index: axi4/README.md
- GAXI Buffers: gaxi/README.md

Last Updated: 2025-10-20

10.11 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

11 AXI4 Master Write Stub

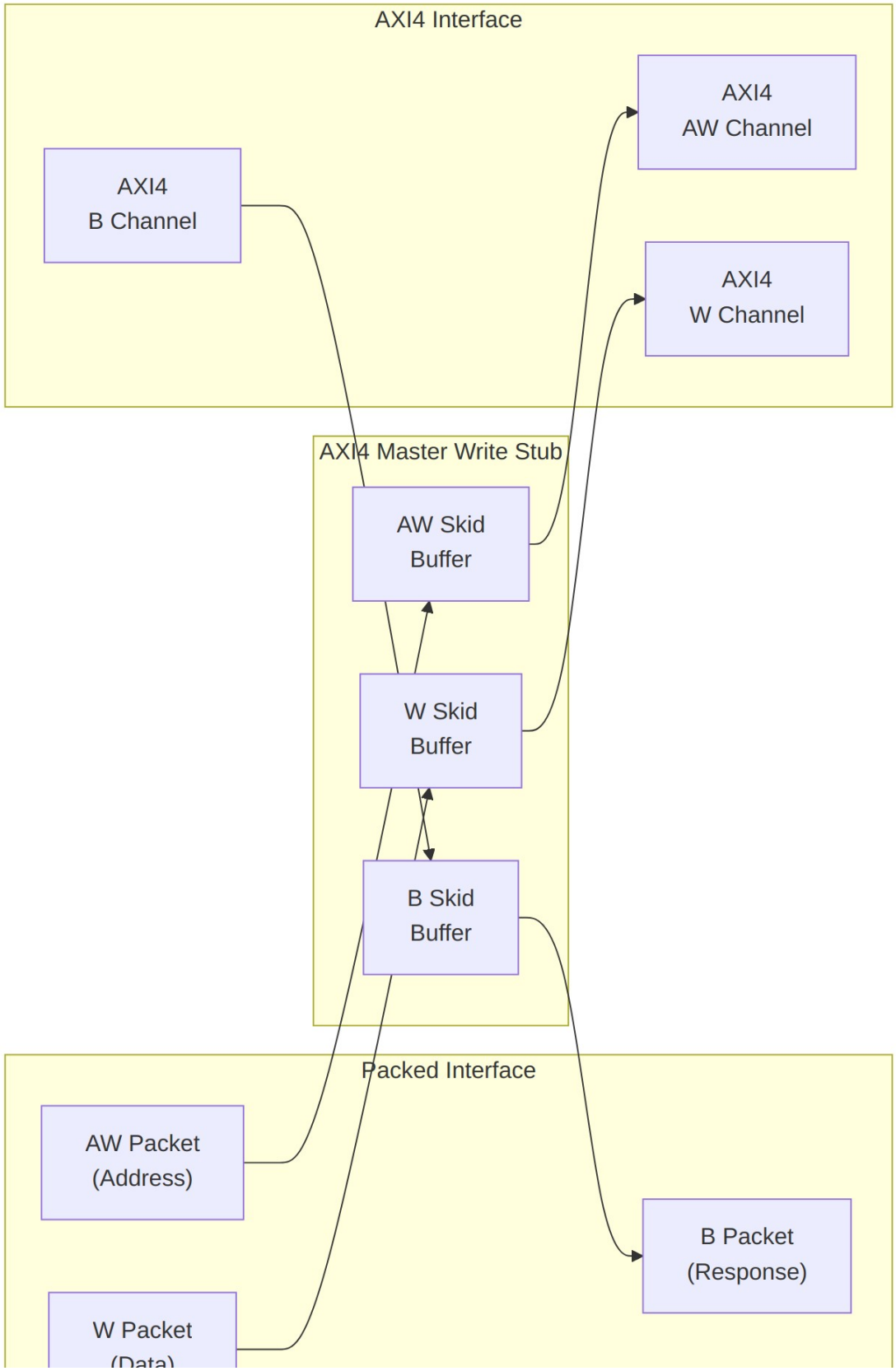
Module: `axi4_master_wr_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

11.1 Overview

The AXI4 Master Write Stub provides a simplified packed-data interface for driving AXI4 write transactions. It uses skid buffers to pack/unpack AXI4 AW (write address), W (write data), and B (write response) channels into simple packet interfaces, making it ideal for testbenches and integration scenarios where a simplified interface is preferred.

11.1.1 Key Features

- Packed packet interface for AW, W, and B channels
- Configurable skid buffer depths for each channel
- Full AXI4 write transaction support
- Burst, ID, strobe, user signal support
- Parameterized data widths


AXI4 Interface
AXI4
AW Channel
AXI4
B Channel
AXI4
W Channel
AXI4 Master Write Stub
AW Skid
Buffer
W Skid
Buffer
B Skid
Buffer
Packed Interface
AW Packet
(Address)
B Packet
(Response)
W Packet
(Data)

11.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth (log2)
SKID_DEPTH_W	int	4	W channel skid buffer depth (log2)
SKID_DEPTH_B	int	2	B channel skid buffer depth (log2)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)
BSize	int	IW+2+UW	B packet size (calculated)

11.3 Ports

11.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI reset (active low)

11.3.2 AXI4 Write Address Channel (AW)

Port	Width	Direction	Description
m_axi_awid	IW	Output	Write address ID
m_axi_awaddr	AW	Output	Write address
m_axi_awlen	8	Output	Burst length
m_axi_awsize	3	Output	Burst size
m_axi_awburst	2	Output	Burst type
m_axi_awlock	1	Output	Lock type
m_axi_awcache	4	Output	Cache type
m_axi_awprot	3	Output	Protection type
m_axi_awqos	4	Output	Quality of service
m_axi_awregion	4	Output	Region identifier
m_axi_awuser	UW	Output	User signal
m_axi_awvalid	1	Output	Write address valid
m_axi_awready	1	Input	Write address ready

11.3.3 AXI4 Write Data Channel (W)

Port	Width	Direction	Description
m_axi_wdata	DW	Output	Write data
m_axi_wstrb	SW	Output	Write strobes

Port	Width	Direction	Description
m_axi_wlast	1	Output	Write last
m_axi_wuser	UW	Output	User signal
m_axi_wvalid	1	Output	Write data valid
m_axi_wready	1	Input	Write data ready

11.3.4 AXI4 Write Response Channel (B)

Port	Width	Direction	Description
m_axi_bid	IW	Input	Response ID
m_axi_bresp	2	Input	Write response
m_axi_buser	UW	Input	User signal
m_axi_bvalid	1	Input	Write response valid
m_axi_bready	1	Output	Write response ready

11.3.5 AW Packet Interface

Port	Width	Direction	Description
fub_axi_awvalid	1	Input	AW packet valid
fub_axi_awready	1	Output	Ready to accept AW packet
fub_axi_aw_count	4	Output	AW buffer occupancy
fub_axi_aw_pkt	AWSize	Input	Packed AW packet data

11.3.6 W Packet Interface

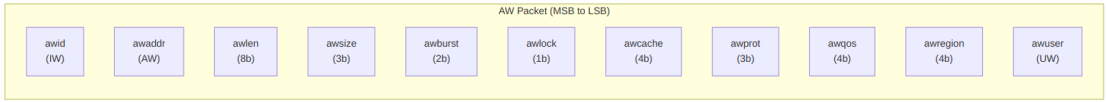
Port	Width	Direction	Description
fub_axi_wvalid	1	Input	W packet valid
fub_axi_wready	1	Output	Ready to accept W

Port	Width	Direction	Description
			packet
fub_axi_w_pkt	WSize	Input	Packed W packet data

11.3.7 B Packet Interface

Port	Width	Direction	Description
fub_axi_bvalid	1	Output	B packet valid
fub_axi_bready	1	Input	Ready to accept B packet
fub_axi_b_pkt	BSize	Output	Packed B packet data

11.4 Packet Formats

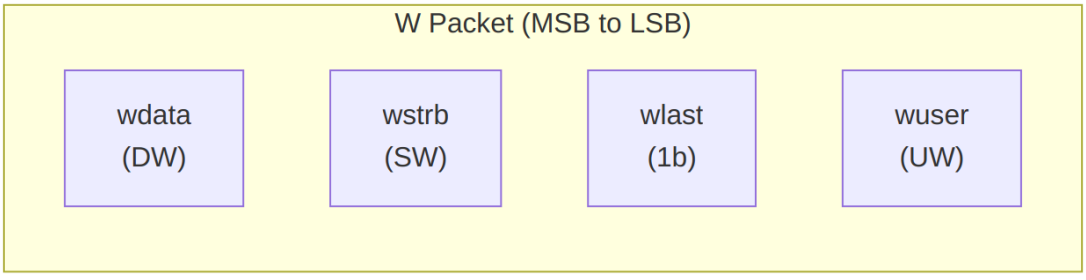


AW Packet Structure (Write Address)

Bit Positions:

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}

Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

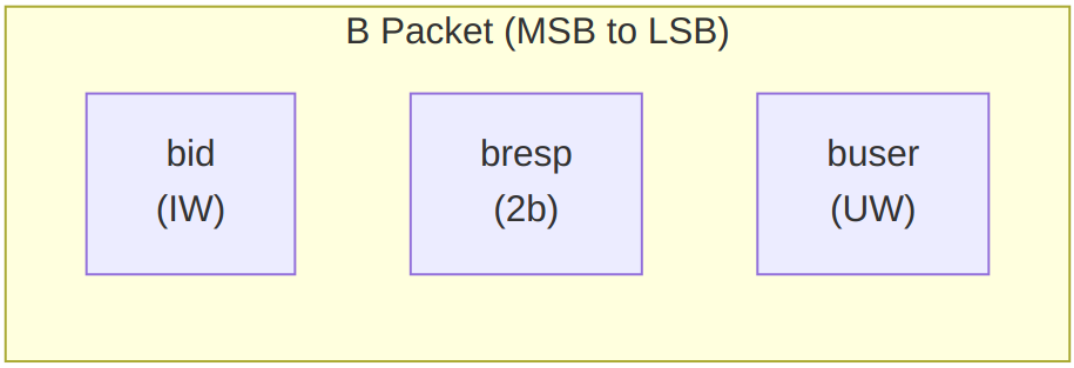


W Packet Structure (Write Data)

Bit Positions:

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}

Width = DW + SW + 1 + UW
```



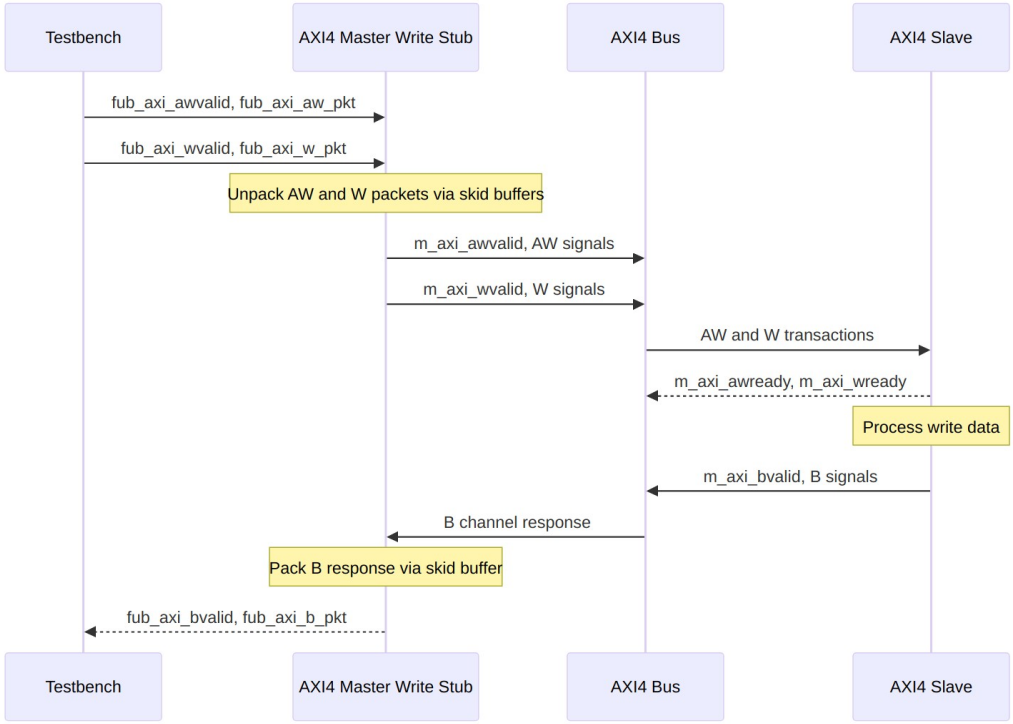
B Packet Structure (Write Response)

Bit Positions:

```
fub_axi_b_pkt = {bid, bresp, buser}

Width = IW + 2 + UW
```

11.5 Transaction Flow



Write Transaction

11.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- aclk
- fub_axi_awvalid, fub_axi_awready, fub_axi_aw_pkt
- fub_axi_wvalid, fub_axi_wready, fub_axi_w_pkt
- AXI AW signals (m_axi_awvalid, m_axi_awaddr, m_axi_awlen, etc.)
- AXI W signals (m_axi_wvalid, m_axi_wdata, m_axi_wstrb, m_axi_wlast,
etc.)
- AXI B signals (m_axi_bvalid, m_axi_bid, m_axi_bresp, etc.)
- fub_axi_bvalid, fub_axi_bready, fub_axi_b_pkt
- Packet-to-AXI timing relationship with skid buffer operation
```

11.6 Usage Example

```
axi4_master_wr_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .AXI_ID_WIDTH    (8),
```

```
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_master_wr_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 master write interface
    .m_axi_awid      (m_axi_awid),
    .m_axi_awaddr    (m_axi_awaddr),
    .m_axi_awlen     (m_axi_awlen),
    .m_axi_awsize    (m_axi_awsize),
    .m_axi_awburst   (m_axi_awburst),
    .m_axi_awlock    (m_axi_awlock),
    .m_axi_awcache   (m_axi_awcache),
    .m_axi_awprot    (m_axi_awprot),
    .m_axi_awqos     (m_axi_awqos),
    .m_axi_awregion  (m_axi_awregion),
    .m_axi_awuser    (m_axi_awuser),
    .m_axi_awvalid   (m_axi_awvalid),
    .m_axi_awready   (m_axi_awready),

    .m_axi_wdata     (m_axi_wdata),
    .m_axi_wstrb     (m_axi_wstrb),
    .m_axi_wlast     (m_axi_wlast),
    .m_axi_wuser     (m_axi_wuser),
    .m_axi_wvalid    (m_axi_wvalid),
    .m_axi_wready    (m_axi_wready),

    .m_axi_bid       (m_axi_bid),
    .m_axi_bresp     (m_axi_bresp),
    .m_axi_buser     (m_axi_buser),
    .m_axi_bvalid    (m_axi_bvalid),
    .m_axi_bready    (m_axi_bready),

    // Packed AW interface
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    // Packed W interface
    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    // Packed B interface
```

```
    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt)
);

// Build AW packet (single beat write at address 0x1000)
localparam AWSize = 8 + 32 + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + 4;  //
Calculate size
assign tb_aw_pkt = {
    8'd0,           // awid
    32'h0000_1000,  // awaddr
    8'd0,           // awlen (1 beat)
    3'b011,         // awsize (8 bytes)
    2'b01,          // awburst (INCR)
    1'b0,           // awlock
    4'b0011,        // awcache
    3'b000,         // awprot
    4'b0000,        // awqos
    4'b0000,        // awregion
    4'h0            // awuser
};

// Build W packet
localparam WSize = 64 + 8 + 1 + 4;  // Calculate size
assign tb_w_pkt = {
    64'hDEAD_BEEF_CAFE_BABE,  // wdata
    8'hFF,                    // wstrb (all bytes)
    1'b1,                     // wlast
    4'h0                      // wuser
};

// Parse B packet
wire [7:0] b_id   = tb_b_pkt[BSize-1:BSize-8];
wire [1:0] b_resp = tb_b_pkt[5:4];
wire [3:0] b_user = tb_b_pkt[3:0];
```

11.7 Design Notes

11.7.1 Skid Buffer Operation

The stub uses `gaxi_skid_buffer` modules to: - Decouple timing between testbench and AXI bus - Provide configurable buffering depth per channel - Handle backpressure gracefully - Support burst transactions without stalling

Recommended Depths: - **AW Channel:** 2-4 (address transactions) - **W Channel:** 4-8 (data beats for bursts) - **B Channel:** 2-4 (responses)

11.7.2 Channel Independence

The AW, W, and B channels are independent: - AW and W can be driven in any order - Stub handles proper AXI timing - B responses arrive asynchronously

11.7.3 Packet Packing Order

AW, W, and B packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet creation - Matches common concatenation order - Efficient for burst transaction handling

11.7.4 Internal Architecture

The stub instantiates three `gaxi_skid_buffer` modules: - **AW Skid Buffer:** Unpacks AW packets to AXI AW channel - **W Skid Buffer:** Unpacks W packets to AXI W channel - **B Skid Buffer:** Packs AXI B channel to B packets

All AXI protocol handling is done by the skid buffers and downstream modules.

11.8 Related Documentation

- **AXI4 Master Write** - Full AXI4 master write module (if wrapping one)
- **AXI4 Master Read Stub** - Corresponding read stub
- **AXI4 Master Stub** - Combined read/write stub
- **AXI4 Slave Write Stub** - Slave-side write stub

11.9 Navigation

- **<- Back to AXI4 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

12 AXI4 Slave Read Interface (Clock-Gated)

Module: `axi4_slave_rd_cg.sv` **Base Module:** axi4_slave_rd **Location:** `rtl/amba/axi4/`
Status: √ Production Ready

12.1 Quick Reference

This is the **clock-gated variant** of axi4_slave_rd.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ Clock-Gated Variants Guide

12.2 Summary

The `axi4_slave_rd_cg` module adds power optimization to `axi4_slave_rd` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization
- √ **Configurable:** Idle threshold, gating domains, enable/disable
- √ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

12.3 Common Parameters

In addition to all axi4_slave_rd parameters:

Parameter	Default	Description
`ENABLE_CLOCK_GATING`	1	Master enable (0=disable, identical to base)
`CG_IDLE_CYCLES`	8	Cycles before clock gating activates
`CG_GATE_*`	1	Domain-specific gating enables

12.4 Quick Usage

```
axi4_slave_rd_cg #(
    // Base module parameters (see axi4_slave_rd.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
```

```
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axi4_slave_rd
);
```

12.5 Documentation

- **Base Module Functionality:** axi4_slave_rd.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**
  - axi4_master_rd_mon_cg.md (AXI4 monitor)
  - axil4_master_rd_mon_cg.md (AXIL4 monitor)
  - apb_slave_cg.md (APB interface)

12.6 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

13 AXI4 Slave Read

Module: `axi4_slave_rd.sv` **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

13.1 Overview

The AXI4 Slave Read module provides a buffered AXI4 slave read interface with configurable depth skid buffers on the AR and R channels. This module serves as an elastic buffer between an AXI4 interconnect and a memory or backend processing element, decoupling timing and providing backpressure handling.

13.1.1 Key Features

- ✓ **Full AXI4 Read Support:** Complete AR and R channel implementation
- ✓ **Independent Channel Buffering:** Separate configurable depth buffers for each channel
- ✓ **Elastic Buffering:** Decouples interconnect and backend timing domains
- ✓ **Burst Support:** Full burst transaction handling with RLAST tracking
- ✓ **User Signal Support:** Carries ARUSER and RUSER signals
- ✓ **Clock Gating Support:** Busy signal for dynamic power management

---

13.2 Module Interface

```
module axi4_slave_rd #(
    parameter int SKID_DEPTH_AR     = 2,
    parameter int SKID_DEPTH_R      = 4,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1
) (
    // Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (s_axi_*)
    // Read address channel (AR)
    input  logic [AXI_ID_WIDTH-1:0]   s_axi_arid,
    input  logic [AXI_ADDR_WIDTH-1:0] s_axi_araddr,
    input  logic [7:0]                 s_axi_arlen,
    input  logic [2:0]                 s_axi_arsize,
    input  logic [1:0]                 s_axi_arburst,
    input  logic                       s_axi_arlock,
    input  logic [3:0]                 s_axi_arcache,
    input  logic [2:0]                 s_axi_arprot,
    input  logic [3:0]                 s_axi_arqos,
    input  logic [3:0]                 s_axi_arregion,
    input  logic [AXI_USER_WIDTH-1:0] s_axi_aruser,
    input  logic                       s_axi_arvalid,
    output logic                       s_axi_arready,

    // Read data channel (R)
    output logic [AXI_ID_WIDTH-1:0]   s_axi_rid,
    output logic [AXI_DATA_WIDTH-1:0] s_axi_rdata,
    output logic [1:0]                 s_axi_rresp,
```

```
    output logic                          s_axi_rlast,
    output logic [AXI_USER_WIDTH-1:0] s_axi_ruser,
    output logic                          s_axi_rvalid,
    input  logic                          s_axi_rready,

    // Backend Interface (fub_axi_* - to memory/backend)
    // Read address channel (AR)
    output logic [AXI_ID_WIDTH-1:0]   fub_axi_arid,
    output logic [AXI_ADDR_WIDTH-1:0] fub_axi_araddr,
    output logic [7:0]                 fub_axi_arlen,
    output logic [2:0]                 fub_axi_arsize,
    output logic [1:0]                 fub_axi_arburst,
    output logic                       fub_axi_arlock,
    output logic [3:0]                 fub_axi_arcache,
    output logic [2:0]                 fub_axi_arprot,
    output logic [3:0]                 fub_axi_arqos,
    output logic [3:0]                 fub_axi_arregion,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_aruser,
    output logic                       fub_axi_arvalid,
    input  logic                       fub_axi_arready,

    // Read data channel (R)
    input  logic [AXI_ID_WIDTH-1:0]   fub_axi_rid,
    input  logic [AXI_DATA_WIDTH-1:0] fub_axi_rdata,
    input  logic [1:0]                 fub_axi_rresp,
    input  logic                       fub_axi_rlast,
    input  logic [AXI_USER_WIDTH-1:0] fub_axi_ruser,
    input  logic                       fub_axi_rvalid,
    output logic                       fub_axi_rready,

    // Status
    output logic                       busy
);
```

13.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	Depth of read address (AR) channel skid buffer
SKID_DEPTH_R	int	4	Depth of read data (R) channel skid buffer
AXI_ID_WIDTH	int	8	Width of transaction ID signals (ARID, RID)

Parameter	Type	Default	Description
AXI_ADDR_WIDTH	int	32	Width of address bus (ARADDR)
AXI_DATA_WIDTH	int	32	Width of data bus (RDATA), must be 8, 16, 32, 64, 128, 256, 512, or 1024
AXI_USER_WIDTH	int	1	Width of user-defined signals (ARUSER, RUSER)

13.4 Port Groups

13.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock - all signals sampled on rising edge
aresetn	Input	1	Active-low asynchronous reset

13.4.2 Slave AXI Interface (s_axi_*)

Read Address Channel (AR)

Port	Direction	Width	Description
s_axi_arid	Input	AXI_ID_WIDTH	Read transaction ID
s_axi_araddr	Input	AXI_ADDR_WIDTH	Read address
s_axi_arlen	Input	8	Burst length (0-255 beats)
s_axi_arsize	Input	3	Burst size (bytes per beat)

Port	Direction	Width	Description
s_axi_arburst	Input	2	Burst type (FIXED, INCR, WRAP)
s_axi_arlock	Input	1	Lock type (atomic access support)
s_axi_arcache	Input	4	Cache attributes
s_axi_arprot	Input	3	Protection attributes
s_axi_arqos	Input	4	Quality of Service identifier
s_axi_arregion	Input	4	Region identifier
s_axi_aruser	Input	AXI_USER_WIDTH	User-defined signal
s_axi_arvalid	Input	1	Read address valid
s_axi_arready	Output	1	Read address ready

Read Data Channel (R)

Port	Direction	Width	Description
s_axi_rid	Output	AXI_ID_WIDTH	Read transaction ID
s_axi_rdata	Output	AXI_DATA_WIDTH	Read data
s_axi_rresp	Output	2	Read response (OKAY, EXOKAY, SLVERR, DECERR)
s_axi_rlast	Output	1	Last beat of burst indicator
s_axi_ruser	Output	AXI_USER_WIDTH	User-defined signal

Port	Direction	Width	Description
s_axi_rvalid	Output	1	Read data valid
s_axi_rready	Input	1	Read data ready

13.4.3 Backend Interface (fub_axi_*)

Mirrors the slave interface but in the opposite direction (output on AR, input on R).

13.4.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Indicates active transactions in buffers (for clock gating)

13.5 Functional Description

13.5.1 Architecture

The AXI4 Slave Read module uses two independent gaxi_skid_buffer instances to provide elastic buffering on each read channel:

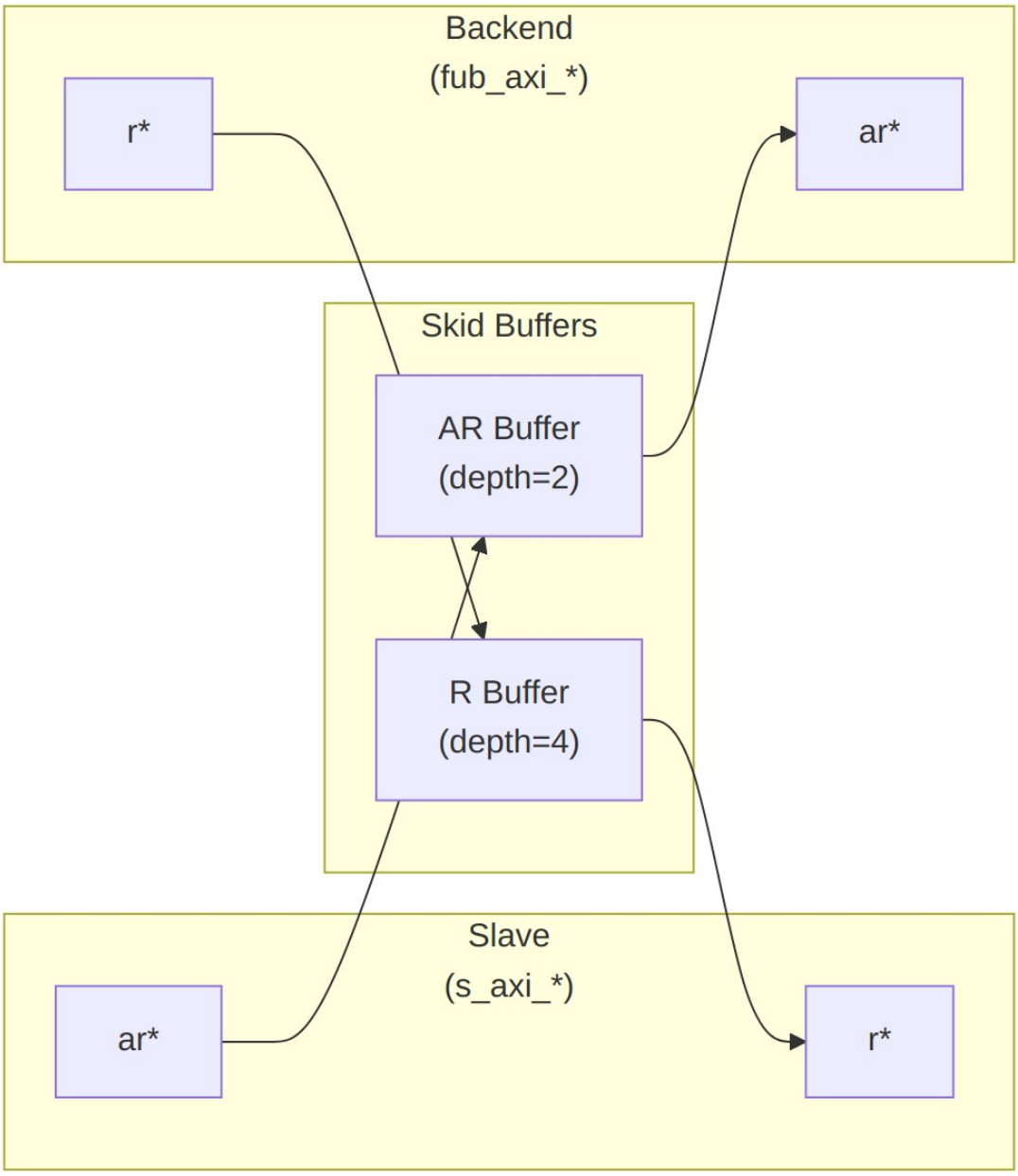


Diagram 21

13.5.2 Channel Operations

13.5.2.1 Read Address (AR) Channel

1. Master presents read address and attributes via interconnect
2. AR skid buffer accepts when space available (`s_axi_arready` high)
3. Buffered address presented to backend when ready

4. Configurable depth (SKID_DEPTH_AR) smooths timing variations

13.5.2.2 Read Data (R) Channel

1. Backend returns read data with transaction ID
2. R skid buffer provides deeper buffering (default depth=4)
3. RLAST signal preserved to indicate burst boundaries
4. Data forwarded to master when ready to accept

13.5.3 Busy Signal

The busy output indicates active transactions:

```
busy = (ar_count > 0) || (r_count > 0) ||
       s_axi_arvalid || fub_axi_rvalid
```

Use cases: - **Clock gating:** Disable clock when busy is low - **Power management:** Enter low-power mode when idle - **Synchronization:** Wait for idle before configuration changes

---

13.6 Configuration Guidelines

13.6.1 Buffer Depth Selection

Read Address (SKID_DEPTH_AR): - Default: 2 (sufficient for most cases) - Increase if: - High-latency backend address processing - Frequent address channel backpressure - Multiple outstanding bursts needed

Read Data (SKID_DEPTH_R): - Default: 4 (deeper than AR for burst data) - Increase if: - Large burst sizes (ARLEN > 4) - High-bandwidth streaming reads - Variable backend read latency

13.6.2 Recommended Configurations

Low-Latency Memory (single outstanding read):

```
axi4_slave_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(2)
) u_slave_rd ( ... );
```

High-Throughput Streaming (burst reads):

```
axi4_slave_rd #(
    .SKID_DEPTH_AR(4),
    .SKID_DEPTH_R(16)    // Deep for burst data
) u_slave_rd ( ... );
```

Variable Latency Backend:

```
axi4_slave_rd #(
    .SKID_DEPTH_AR(8),
    .SKID_DEPTH_R(16)
) u_slave_rd ( ... );
```

13.7 Usage Example

13.7.1 Basic Integration with Memory

```
// Instantiate AXI4 slave read buffer
axi4_slave_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi_slave_rd (
    .aclk               (axi_aclk),
    .aresetn            (axi_aresetn),

    // Slave interface (from AXI interconnect)
    .s_axi_arid         (s_axi_arid),
    .s_axi_araddr       (s_axi_araddr),
    .s_axi_arlen        (s_axi_arlen),
    .s_axi_arsize       (s_axi_arsize),
    .s_axi_arburst      (s_axi_arburst),
    .s_axi_arlock       (s_axi_arlock),
    .s_axi_arcache      (s_axi_arcache),
    .s_axi_arprot       (s_axi_arprot),
    .s_axi_arqos        (s_axi_arqos),
    .s_axi_arregion     (s_axi_arregion),
    .s_axi_aruser       (s_axi_aruser),
    .s_axi_arvalid      (s_axi_arvalid),
    .s_axi_arready      (s_axi_arready),

    .s_axi_rid          (s_axi_rid),
    .s_axi_rdata        (s_axi_rdata),
    .s_axi_rresp        (s_axi_rresp),
    .s_axi_rlast        (s_axi_rlast),
    .s_axi_ruser        (s_axi_ruser),
    .s_axi_rvalid       (s_axi_rvalid),
    .s_axi_rready       (s_axi_rready),

    // Backend interface (to memory controller)
```

```
    .fub_axi_arid       (mem_arid),
    .fub_axi_araddr     (mem_araddr),
    .fub_axi_arlen      (mem_arlen),
    .fub_axi_arsize     (mem_arsize),
    .fub_axi_arburst    (mem_arburst),
    .fub_axi_arlock     (mem_arlock),
    .fub_axi_arcache    (mem_arcache),
    .fub_axi_arprot     (mem_arprot),
    .fub_axi_arqos      (mem_arqos),
    .fub_axi_arregion   (mem_arregion),
    .fub_axi_aruser     (mem_aruser),
    .fub_axi_arvalid    (mem_arvalid),
    .fub_axi_arready    (mem_arready),

    .fub_axi_rid        (mem_rid),
    .fub_axi_rdata      (mem_rdata),
    .fub_axi_rresp      (mem_rresp),
    .fub_axi_rlast      (mem_rlast),
    .fub_axi_ruser      (mem_ruser),
    .fub_axi_rvalid     (mem_rvalid),
    .fub_axi_rready     (mem_rready),

    // Status
    .busy               (rd_slave_busy)
);

// Memory controller backend
memory_controller u_mem_ctrl (
    .axi_aclk       (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_arid       (mem_arid),
    .axi_araddr     (mem_araddr),
    // ... rest of signals
);
```

13.7.2 Clock Gating Example

```
// Use busy signal for clock gating
logic axi_clk_gated;

clock_gate_ctrl u_cg (
    .i_clk          (axi_clk),
    .i_enable       (rd_slave_busy),
    .o_clk_gated    (axi_clk_gated)
);

// Connect module to gated clock
```

```
axi4_slave_rd #( ... ) u_slave_rd (
    .aclk (axi_clk_gated),
    ...
);
```

13.8 Design Notes

13.8.1 Buffer Independence

The two skid buffers operate independently: - AR channel can accept new addresses while R channel returns data - Burst reads can pipeline - next burst starts before previous completes - Backend can have variable read latency without stalling interconnect

13.8.2 Packet Preservation

All signal groups packed and unpacked atomically: - AR channel: `{arid, araddr, arlen, arsize, arburst, arlock, arcache, arprot, arqos, arregion, aruser}` - R channel: `{rid, rdata, rresp, rlast, ruser}`

13.8.3 Backpressure Handling

Ready signals propagate backpressure: - Interconnect sees `arready` low when AR buffer full - Backend sees `rready` low when R buffer full - Prevents data loss while decoupling timing

13.8.4 ID and Burst Handling

This module is a pass-through buffer: - Does NOT track transaction IDs - Does NOT enforce burst ordering - Relies on backend to: - Match RID to ARID - Return correct number of beats (ARLEN+1) - Assert RLAST on final beat

13.8.5 Reset Behavior

On `aresetn` assertion (active-low): - All skid buffers flush - Valid signals deasserted - Busy signal goes low - No data retained across reset

---

13.9 Related Modules

13.9.1 Companion Modules

- **axi4_slave_wr** - AXI4 slave write with buffering
- **axi4_slave_rd_cg** - Clock-gated variant with additional CG logic
- **axi4_slave_rd_mon** - Read transaction monitor for verification

13.9.2 Used Components

- **gaxi_skid_buffer** - Elastic buffer with valid/ready handshake
- **clock_gate_ctrl** - Clock gating control (example)

13.9.3 Related Infrastructure

- **axi4_master_rd** - Corresponding AXI4 read master module
- **axi4_interconnect** - Multi-master/multi-slave crossbar

13.10 References

13.10.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)

13.10.2 Source Code

- RTL: `rtl/amba/axi4/axi4_slave_rd.sv`
- Tests: `val/amba/test_axi4_slave_rd.py`
- Framework: `bin/TBClasses/components/axi4/`

13.10.3 Documentation

- Architecture: RTLAmba Overview
- AXI4 Index: axi4/README.md
- GAXI Buffers: gaxi/README.md

Last Updated: 2025-10-20

13.11 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

14 AXI4 Slave Read Stub

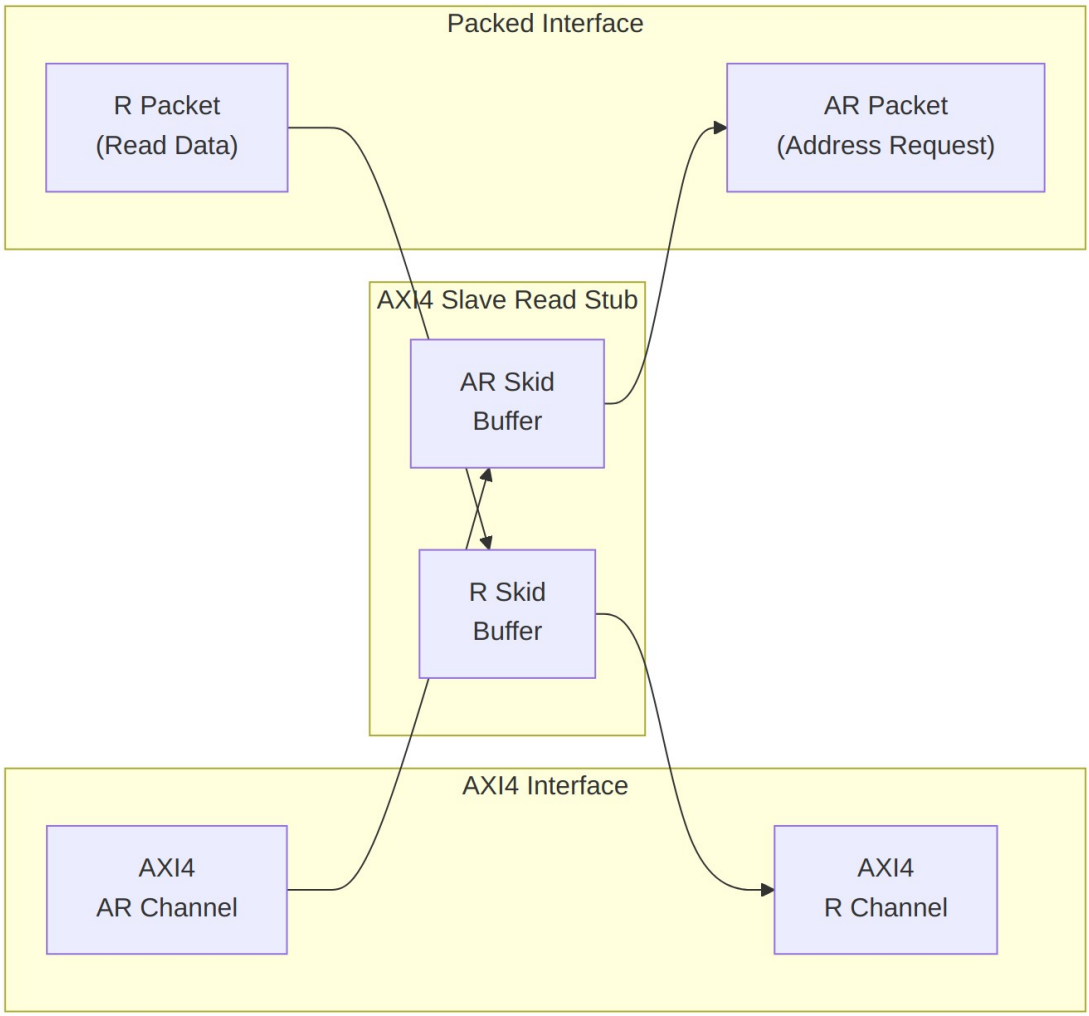
Module: `axi4_slave_rd_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

14.1 Overview

The AXI4 Slave Read Stub provides a simplified packed-data interface for receiving AXI4 read transactions from a master. It uses skid buffers to pack/unpack AXI4 AR (read address) and R (read data) channels into simple packet interfaces, making it ideal for testbenches and integration scenarios where a simplified slave interface is needed.

14.1.1 Key Features

- Packed packet interface for AR and R channels
- Configurable skid buffer depths for each channel
- Full AXI4 read transaction support
- Burst, ID, user signal support
- Parameterized data widths



Module Architecture

14.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR channel skid buffer depth (log2)
SKID_DEPTH_R	int	4	R channel skid buffer depth (log2)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width

Parameter	Type	Default	Description
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (unused)
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

14.3 Ports

14.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI reset (active low)

14.3.2 AXI4 Read Address Channel (AR)

Port	Width	Direction	Description
s_axi_arid	IW	Input	Read address ID
s_axi_araddr	AW	Input	Read address
s_axi_arlen	8	Input	Burst length

Port	Width	Direction	Description
s_axi_arsize	3	Input	Burst size
s_axi_arburst	2	Input	Burst type
s_axi_arlock	1	Input	Lock type
s_axi_arcache	4	Input	Cache type
s_axi_arprot	3	Input	Protection type
s_axi_arqos	4	Input	Quality of service
s_axi_arregion	4	Input	Region identifier
s_axi_aruser	UW	Input	User signal
s_axi_arvalid	1	Input	Read address valid
s_axi_arready	1	Output	Read address ready

14.3.3 AXI4 Read Data Channel (R)

Port	Width	Direction	Description
s_axi_rid	IW	Output	Read data ID
s_axi_rdata	DW	Output	Read data
s_axi_rresp	2	Output	Read response
s_axi_rlast	1	Output	Read last
s_axi_ruser	UW	Output	User signal
s_axi_rvalid	1	Output	Read data valid
s_axi_rready	1	Input	Read data ready

14.3.4 AR Packet Interface

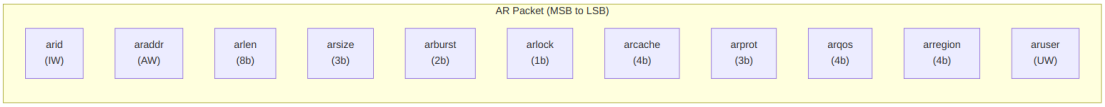
Port	Width	Direction	Description
fub_axi_arvalid	1	Output	AR packet valid
fub_axi_arready	1	Input	Ready to accept AR

Port	Width	Direction	Description
			packet
fub_axi_ar_count	4	Output	AR buffer occupancy
fub_axi_ar_pkt	ARSize	Output	Packed AR packet data

14.3.5 R Packet Interface

Port	Width	Direction	Description
fub_axi_rvalid	1	Input	R packet valid
fub_axi_rready	1	Output	Ready to accept R packet
fub_axi_r_pkt	RSize	Input	Packed R packet data

14.4 Packet Formats

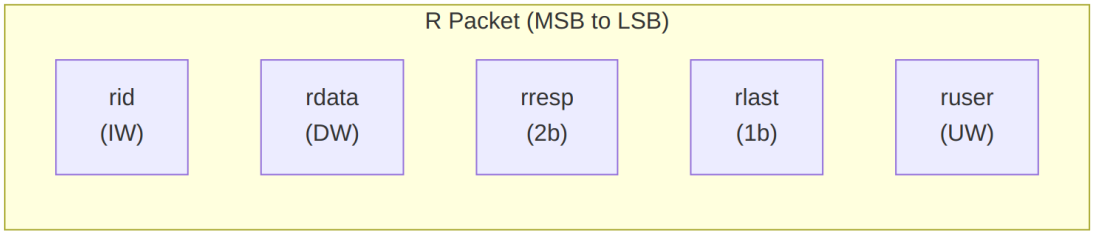


AR Packet Structure (Read Address)

Bit Positions:

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}

Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```



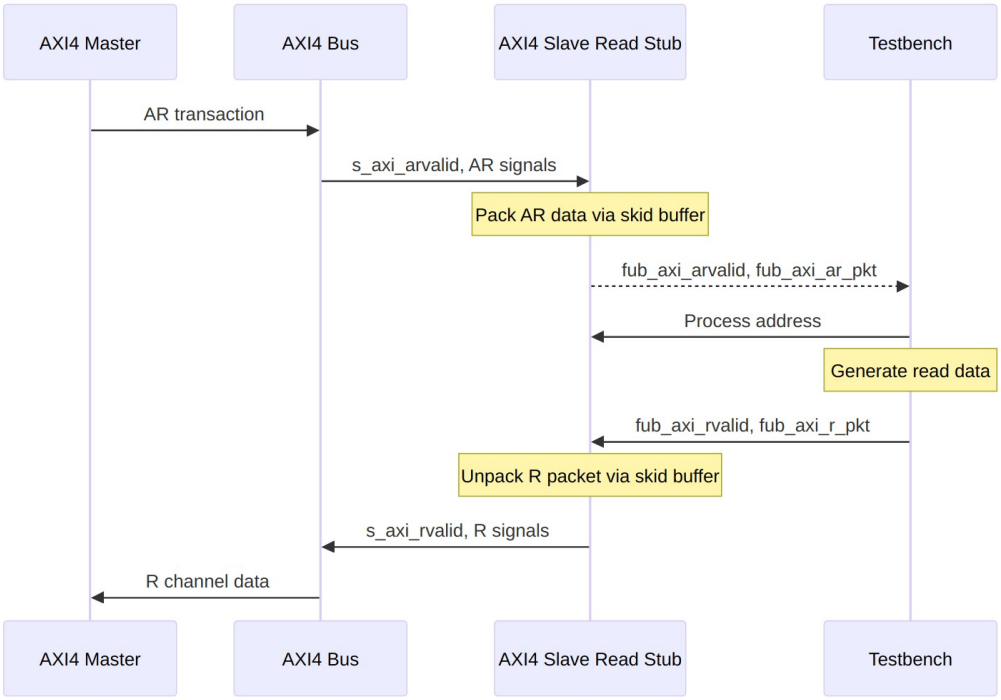
R Packet Structure (Read Data)

Bit Positions:

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}

Width = IW + DW + 2 + 1 + UW
```

14.5 Transaction Flow



Read Transaction

14.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- aclk
- AXI AR signals (s_axi_arvalid, s_axi_araddr, s_axi_arlen, etc.)
- fub_axi_arvalid, fub_axi_arready, fub_axi_ar_pkt
- fub_axi_rvalid, fub_axi_rready, fub_axi_r_pkt
- AXI R signals (s_axi_rvalid, s_axi_rdata, s_axi_rlast, etc.)
- Packet-to-AXI timing relationship with skid buffer operation
```

14.6 Usage Example

```
axi4_slave_rd_stub #(
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_slave_rd_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 slave read interface
    .s_axi_arid      (s_axi_arid),
    .s_axi_araddr    (s_axi_araddr),
    .s_axi_arlen     (s_axi_arlen),
    .s_axi_arsize    (s_axi_arsize),
    .s_axi_arburst   (s_axi_arburst),
    .s_axi_arlock    (s_axi_arlock),
    .s_axi_arcache   (s_axi_arcache),
    .s_axi_arprot    (s_axi_arprot),
    .s_axi_arqos     (s_axi_arqos),
    .s_axi_arregion  (s_axi_arregion),
    .s_axi_aruser    (s_axi_aruser),
    .s_axi_arvalid   (s_axi_arvalid),
    .s_axi_arready   (s_axi_arready),

    .s_axi_rid       (s_axi_rid),
    .s_axi_rdata     (s_axi_rdata),
    .s_axi_rresp     (s_axi_rresp),
    .s_axi_rlast     (s_axi_rlast),
    .s_axi_ruser     (s_axi_ruser),
    .s_axi_rvalid    (s_axi_rvalid),
    .s_axi_rready    (s_axi_rready),

    // Packed AR interface
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    // Packed R interface
    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);
```

```
// Parse AR packet
wire [7:0]  ar_id     = tb_ar_pkt[ARSize-1:ARSize-8];
wire [31:0] ar_addr   = tb_ar_pkt[ARSize-9:ARSize-40];
wire [7:0]  ar_len    = tb_ar_pkt[ARSize-41:ARSize-48];
wire [2:0]  ar_size   = tb_ar_pkt[ARSize-49:ARSize-51];
wire [1:0]  ar_burst  = tb_ar_pkt[ARSize-52:ARSize-53];
// ... additional fields as needed

// Build R packet (single beat response with data 0xDEADBEEFCAFEBABE)
localparam RSize = 8 + 64 + 2 + 1 + 4;  // Calculate size
assign tb_r_pkt = {
    ar_id,                      // rid (match request ID)
    64'hDEAD_BEEF_CAFE_BABE,   // rdata
    2'b00,                      // rresp (OKAY)
    1'b1,                       // rlast
    4'h0                        // ruser
};
```

14.7 Design Notes

14.7.1 Skid Buffer Operation

The stub uses `gaxi_skid_buffer` modules to: - Decouple timing between AXI bus and testbench - Provide configurable buffering depth per channel - Handle backpressure gracefully - Support burst transactions without stalling

Recommended Depths: - **AR Channel:** 2-4 (address transactions) - **R Channel:** 4-8 (data beats for bursts)

14.7.2 Packet Packing Order

AR and R packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet parsing - Matches common concatenation order - Efficient for burst transaction handling

14.7.3 Internal Architecture

The stub instantiates two `gaxi_skid_buffer` modules: - **AR Skid Buffer:** Packs AXI AR channel to AR packets - **R Skid Buffer:** Unpacks R packets to AXI R channel

All AXI protocol handling is done by the skid buffers and upstream modules.

14.8 Related Documentation

- **AXI4 Slave Read** - Full AXI4 slave read module (if wrapping one)
- **AXI4 Slave Write Stub** - Corresponding write stub
- **AXI4 Slave Stub** - Combined read/write stub
- **AXI4 Master Read Stub** - Master-side read stub

14.9 Navigation

- **<- Back to AXI4 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

15 AXI4 Slave Stub

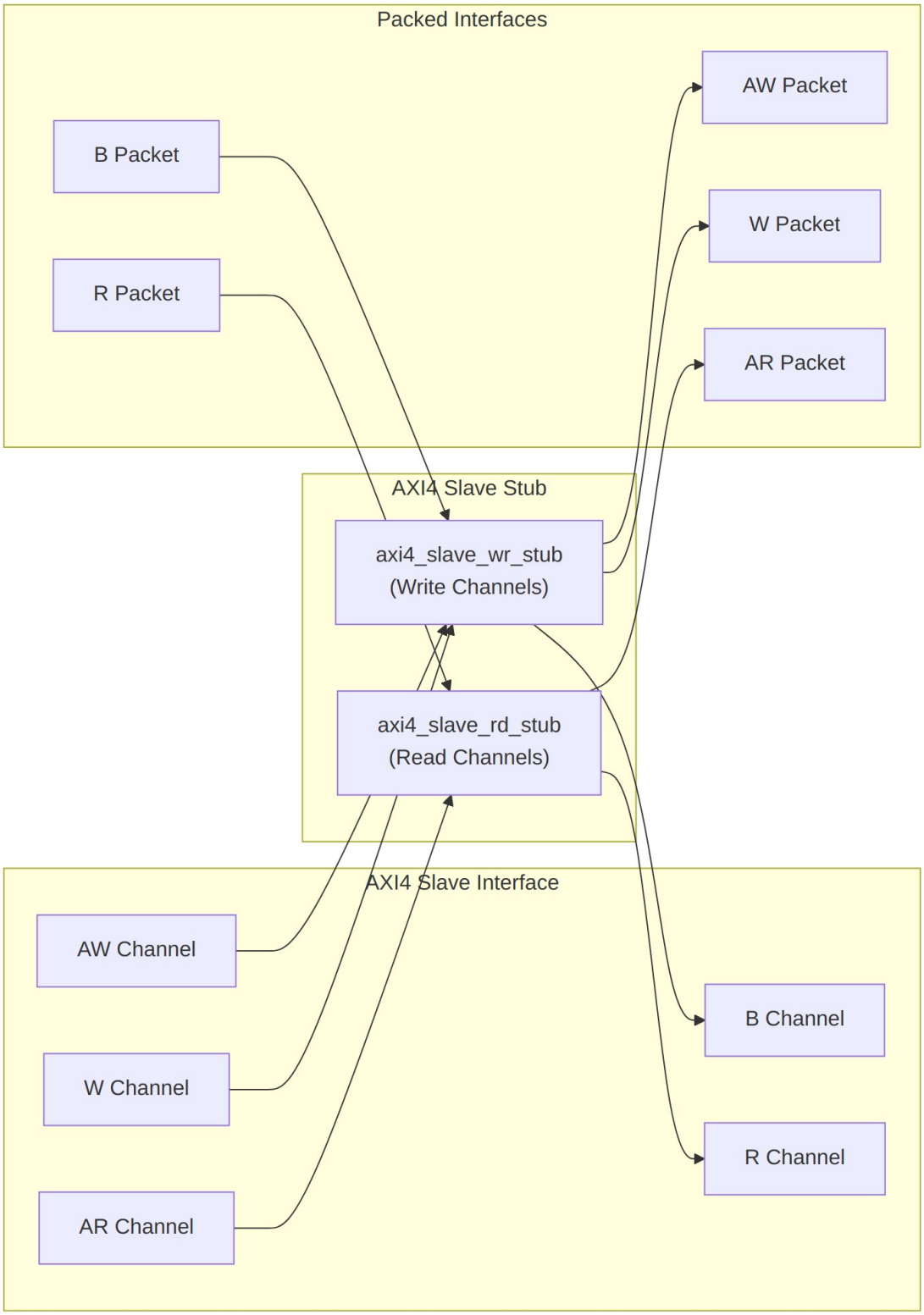
Module: `axi4_slave_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

15.1 Overview

The AXI4 Slave Stub provides a simplified packed-data interface for receiving both AXI4 read and write transactions from a master. It combines the read and write slave stub functionalities into a single module, internally instantiating `axi4_slave_rd_stub` and `axi4_slave_wr_stub`. This provides a complete AXI4 slave interface with simplified packet-based control, ideal for testbenches and integration scenarios.

15.1.1 Key Features

- Combined read and write channel support
- Packed packet interfaces for all channels (AW, W, B, AR, R)
- Configurable skid buffer depths per channel
- Full AXI4 protocol support (bursts, IDs, user signals)
- Simplified testbench integration
- Parameterized data widths



Module Architecture

15.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth (log2)
SKID_DEPTH_W	int	4	W channel skid buffer depth (log2)
SKID_DEPTH_B	int	2	B channel skid buffer depth (log2)
SKID_DEPTH_AR	int	2	AR channel skid buffer depth (log2)
SKID_DEPTH_R	int	4	R channel skid buffer depth (log2)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)

Parameter	Type	Default	Description
BSize	int	IW+2+UW	B packet size (calculated)
ARSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AR packet size (calculated)
RSize	int	IW+DW+2+1+UW	R packet size (calculated)

15.3 Ports

15.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI reset (active low)

15.3.2 AXI4 Write Channels

Port	Width	Direction	Description
s_axi_awid	AXI_ID_WIDTH	Input	Write address ID
s_axi_awaddr	AXI_ADDR_WIDTH	Input	Write address
s_axi_awlen	8	Input	Burst length
s_axi_awsize	3	Input	Burst size
s_axi_awburst	2	Input	Burst type
s_axi_awlock	1	Input	Lock type
s_axi_awcache	4	Input	Cache type
s_axi_awprot	3	Input	Protection type
s_axi_awqos	4	Input	Quality of service
s_axi_awregion	4	Input	Region identifier
s_axi_awuser	AXI_USER_WIDTH	Input	User signal

Port	Width	Direction	Description
s_axi_awvalid	1	Input	Write address valid
s_axi_awready	1	Output	Write address ready
s_axi_wdata	AXI_DATA_WIDTH	Input	Write data
s_axi_wstrb	AXI_WSTRB_WIDTH	Input	Write strobes
s_axi_wlast	1	Input	Write last
s_axi_wuser	AXI_USER_WIDTH	Input	User signal
s_axi_wvalid	1	Input	Write data valid
s_axi_wready	1	Output	Write data ready
s_axi_bid	AXI_ID_WIDTH	Output	Response ID
s_axi_bresp	2	Output	Write response
s_axi_buser	AXI_USER_WIDTH	Output	User signal
s_axi_bvalid	1	Output	Write response valid
s_axi_bready	1	Input	Write response ready

15.3.3 AXI4 Read Channels

Port	Width	Direction	Description
s_axi_arid	AXI_ID_WIDTH	Input	Read address ID
s_axi_araddr	AXI_ADDR_WIDTH	Input	Read address
s_axi_arlen	8	Input	Burst length
s_axi_arsize	3	Input	Burst size
s_axi_arburst	2	Input	Burst type

Port	Width	Direction	Description
s_axi_arlock	1	Input	Lock type
s_axi_arcache	4	Input	Cache type
s_axi_arprot	3	Input	Protection type
s_axi_arqos	4	Input	Quality of service
s_axi_arregion	4	Input	Region identifier
s_axi_aruser	AXI_USER_WIDTH	Input	User signal
s_axi_arvalid	1	Input	Read address valid
s_axi_arready	1	Output	Read address ready
s_axi_rid	AXI_ID_WIDTH	Output	Read data ID
s_axi_rdata	AXI_DATA_WIDTH	Output	Read data
s_axi_rresp	2	Output	Read response
s_axi_rlast	1	Output	Read last
s_axi_ruser	AXI_USER_WIDTH	Output	User signal
s_axi_rvalid	1	Output	Read data valid
s_axi_rready	1	Input	Read data ready

15.3.4 Write Packet Interfaces

Port	Width	Direction	Description
fub_axi_awvalid	1	Output	AW packet valid
fub_axi_awready	1	Input	Ready to accept AW packet
fub_axi_aw_co	4	Output	AW buffer

Port	Width	Direction	Description
unt			occupancy
fub_axi_aw_pkt	AWSize	Output	Packed AW packet data
fub_axi_wvalid	1	Output	W packet valid
fub_axi_wready	1	Input	Ready to accept W packet
fub_axi_w_pkt	WSize	Output	Packed W packet data
fub_axi_bvalid	1	Input	B packet valid
fub_axi_bready	1	Output	Ready to accept B packet
fub_axi_b_pkt	BSize	Input	Packed B packet data

15.3.5 Read Packet Interfaces

Port	Width	Direction	Description
fub_axi_arvalid	1	Output	AR packet valid
fub_axi_arready	1	Input	Ready to accept AR packet
fub_axi_ar_count	4	Output	AR buffer occupancy
fub_axi_ar_pkt	ARSize	Output	Packed AR packet data
fub_axi_rvalid	1	Input	R packet valid
fub_axi_rready	1	Output	Ready to accept R packet
fub_axi_r_pkt	RSize	Input	Packed R packet data

15.4 Packet Formats

15.4.1 Write Channel Packets

AW Packet (Write Address):

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

W Packet (Write Data):

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}
Width = DW + SW + 1 + UW
```

B Packet (Write Response):

```
fub_axi_b_pkt = {bid, bresp, buser}
Width = IW + 2 + UW
```

15.4.2 Read Channel Packets

AR Packet (Read Address):

```
fub_axi_ar_pkt = {arid, araddr, arlen, arsize, arburst, arlock,
arcache, arprot, arqos, arregion, aruser}
Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

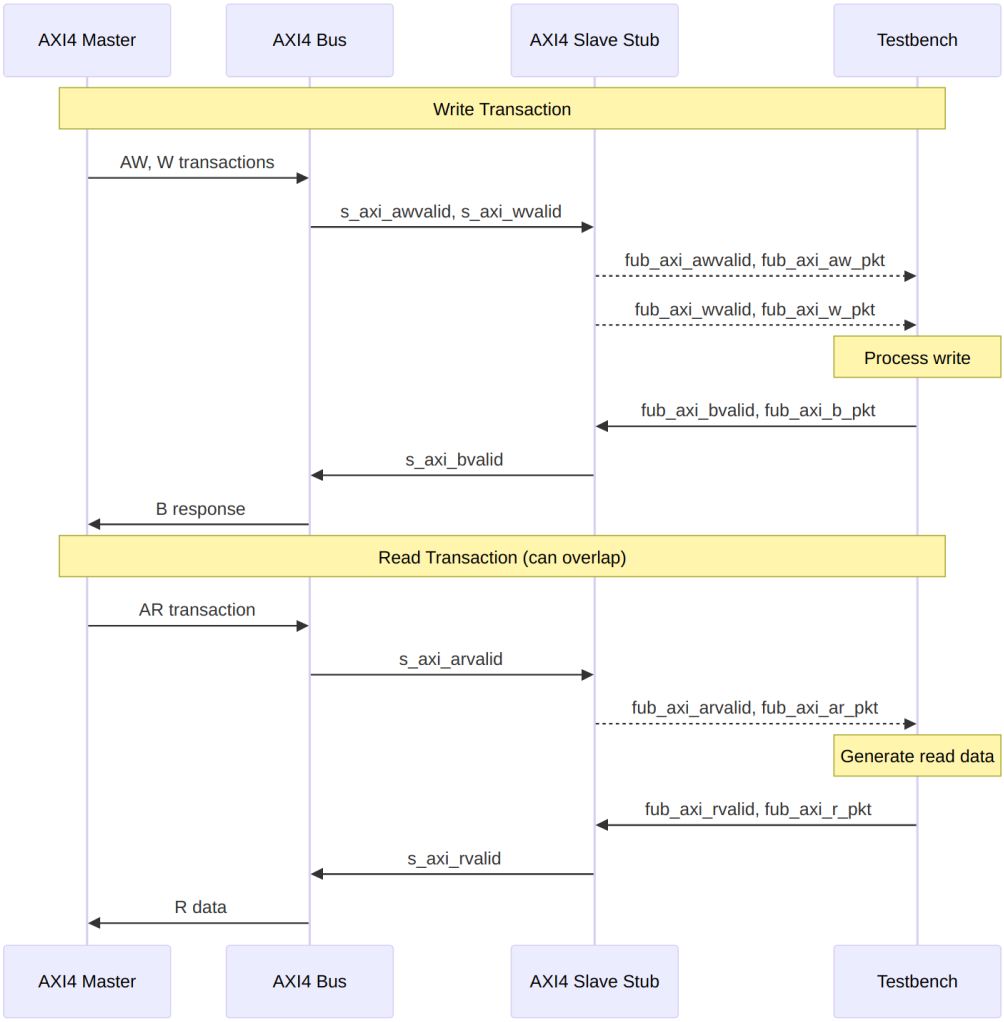
R Packet (Read Data):

```
fub_axi_r_pkt = {rid, rdata, rresp, rlast, ruser}
Width = IW + DW + 2 + 1 + UW
```

Complete packet format details: See AXI4 Slave Write Stub and AXI4 Slave Read Stub

---

15.5 Transaction Flow



Combined Read and Write Operations

15.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- aclk
- AXI write channels (s_axi_aw*, s_axi_w*, s_axi_b*)
- Write packet interfaces (fub_axi_aw*, fub_axi_w*, fub_axi_b*)
- AXI read channels (s_axi_ar*, s_axi_r*)
- Read packet interfaces (fub_axi_ar*, fub_axi_r*)
- Overlapping read/write operations
```

15.6 Usage Example

```
axi4_slave_stub #(
    .SKID_DEPTH_AW   (2),
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .SKID_DEPTH_AR   (2),
    .SKID_DEPTH_R    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_slave_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 slave interface (all channels)
    .s_axi_awid      (s_axi_awid),
    .s_axi_awaddr    (s_axi_awaddr),
    .s_axi_awlen     (s_axi_awlen),
    .s_axi_awsize    (s_axi_awsize),
    .s_axi_awburst   (s_axi_awburst),
    .s_axi_awlock    (s_axi_awlock),
    .s_axi_awcache   (s_axi_awcache),
    .s_axi_awprot    (s_axi_awprot),
    .s_axi_awqos     (s_axi_awqos),
    .s_axi_awregion  (s_axi_awregion),
    .s_axi_awuser    (s_axi_awuser),
    .s_axi_awvalid   (s_axi_awvalid),
    .s_axi_awready   (s_axi_awready),

    .s_axi_wdata     (s_axi_wdata),
    .s_axi_wstrb     (s_axi_wstrb),
    .s_axi_wlast     (s_axi_wlast),
    .s_axi_wuser     (s_axi_wuser),
    .s_axi_wvalid    (s_axi_wvalid),
    .s_axi_wready    (s_axi_wready),

    .s_axi_bid       (s_axi_bid),
    .s_axi_bresp     (s_axi_bresp),
    .s_axi_buser     (s_axi_buser),
    .s_axi_bvalid    (s_axi_bvalid),
    .s_axi_bready    (s_axi_bready),

    .s_axi_arid      (s_axi_arid),
    .s_axi_araddr    (s_axi_araddr),
    .s_axi_arlen     (s_axi_arlen),
```

```
    .s_axi_arsize    (s_axi_arsize),
    .s_axi_arburst   (s_axi_arburst),
    .s_axi_arlock    (s_axi_arlock),
    .s_axi_arcache   (s_axi_arcache),
    .s_axi_arprot    (s_axi_arprot),
    .s_axi_arqos     (s_axi_arqos),
    .s_axi_arregion  (s_axi_arregion),
    .s_axi_aruser    (s_axi_aruser),
    .s_axi_arvalid   (s_axi_arvalid),
    .s_axi_arready   (s_axi_arready),

    .s_axi_rid       (s_axi_rid),
    .s_axi_rdata     (s_axi_rdata),
    .s_axi_rresp     (s_axi_rresp),
    .s_axi_rlast     (s_axi_rlast),
    .s_axi_ruser     (s_axi_ruser),
    .s_axi_rvalid    (s_axi_rvalid),
    .s_axi_rready    (s_axi_rready),

    // Write packet interfaces
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
    .fub_axi_w_pkt   (tb_w_pkt),

    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt),

    // Read packet interfaces
    .fub_axi_arvalid (tb_ar_valid),
    .fub_axi_arready (tb_ar_ready),
    .fub_axi_ar_count(tb_ar_count),
    .fub_axi_ar_pkt  (tb_ar_pkt),

    .fub_axi_rvalid  (tb_r_valid),
    .fub_axi_rready  (tb_r_ready),
    .fub_axi_r_pkt   (tb_r_pkt)
);

// Parse incoming write address
wire [7:0]  aw_id     = tb_aw_pkt[AWSize-1:AWSize-8];
```

```
wire [31:0] aw_addr   = tb_aw_pkt[AWSize-9:AWSize-40];
wire [7:0]  aw_len    = tb_aw_pkt[AWSize-41:AWSize-48];
// ... additional fields

// Parse incoming write data
wire [63:0] w_data = tb_w_pkt[WSize-1:WSize-64];
wire [7:0]  w_strb = tb_w_pkt[WSize-65:WSize-72];
wire        w_last = tb_w_pkt[WSize-73];

// Generate write response (OKAY)
assign tb_b_pkt = {
    aw_id,    // bid (match request ID)
    2'b00,    // bresp (OKAY)
    4'h0      // buser
};

// Parse incoming read address
wire [7:0]  ar_id   = tb_ar_pkt[ARSize-1:ARSize-8];
wire [31:0] ar_addr = tb_ar_pkt[ARSize-9:ARSize-40];
wire [7:0]  ar_len  = tb_ar_pkt[ARSize-41:ARSize-48];
// ... additional fields

// Generate read data response
reg [63:0] read_data;  // From memory model
assign tb_r_pkt = {
    ar_id,      // rid (match request ID)
    read_data,  // rdata
    2'b00,      // rresp (OKAY)
    1'b1,       // rlast (single beat example)
    4'h0        // ruser
};
```

15.7 Design Notes

15.7.1 Internal Architecture

The stub instantiates two sub-modules: - **axi4_slave_wr_stub** - Handles AW, W, and B channels - **axi4_slave_rd_stub** - Handles AR and R channels

This hierarchical design: - Reuses proven read/write stub modules - Maintains clear channel separation - Simplifies verification and testing - Allows independent read/write operation

15.7.2 Read/Write Independence

Read and write channels are completely independent: - Can overlap in time (simultaneous read and write) - Each has independent skid buffers - No ordering constraints between reads and writes - Testbench can respond at different rates

15.7.3 Skid Buffer Depths

Recommended configurations:

Low latency (fast response):

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(2), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(2)
```

Typical system:

```
.SKID_DEPTH_AW(2), .SKID_DEPTH_W(4), .SKID_DEPTH_B(2),
.SKID_DEPTH_AR(2), .SKID_DEPTH_R(4)
```

High throughput bursts:

```
.SKID_DEPTH_AW(4), .SKID_DEPTH_W(8), .SKID_DEPTH_B(4),
.SKID_DEPTH_AR(4), .SKID_DEPTH_R(8)
```

15.7.4 Memory Model Integration

The slave stub is ideal for connecting to memory models:

```
// Simple memory model
reg [63:0] memory [0:1023];

// Handle write requests
always @(posedge aclk) begin
    if (tb_aw_valid && tb_aw_ready) begin
        // Capture write address
    end
    if (tb_w_valid && tb_w_ready) begin
        // Write data to memory
        memory[write_addr] <= w_data;
    end
end

// Handle read requests
always @(posedge aclk) begin
    if (tb_ar_valid && tb_ar_ready) begin
        // Generate read data from memory
        read_data <= memory[ar_addr];
    end
end
```

15.8 Related Documentation

- **AXI4 Slave Read Stub** - Read-only stub (instantiated internally)
- **AXI4 Slave Write Stub** - Write-only stub (instantiated internally)
- **AXI4 Master Stub** - Corresponding combined master stub
- **AXI4 Slave Read** - Full AXI4 slave read module
- **AXI4 Slave Write** - Full AXI4 slave write module

15.9 Navigation

- **<- Back to AXI4 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

16 AXI4 Slave Write Interface (Clock-Gated)

Module: `axi4_slave_wr_cg.sv` **Base Module:** axi4_slave_wr **Location:** `rtl/amba/axi4/`
Status: √ Production Ready

16.1 Quick Reference

This is the **clock-gated variant** of axi4_slave_wr.

For complete clock-gating documentation, usage examples, and configuration guidelines, see:

→ **Clock-Gated Variants Guide**

16.2 Summary

The `axi4_slave_wr_cg` module adds power optimization to `axi4_slave_wr` through activity-based clock gating:

- √ **Same Functionality:** 100% equivalent to base module
- √ **Power Savings:** 25-70% depending on traffic utilization

- ✓ **Configurable:** Idle threshold, gating domains, enable/disable
- ✓ **Zero Overhead When Disabled:** `ENABLE_CLOCK_GATING=0` → identical to base

16.3 Common Parameters

In addition to all axi4_slave_wr parameters:

Parameter	Default	Description
`ENABLE_CLOCK_GATING`	1	Master enable (0=disable, identical to base)
`CG_IDLE_CYCLES`	8	Cycles before clock gating activates
`CG_GATE_*`	1	Domain-specific gating enables

16.4 Quick Usage

```
axi4_slave_wr_cg #(
    // Base module parameters (see axi4_slave_wr.md)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating (see CG guide for details)
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(8)
) u_cg (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as axi4_slave_wr
);
```

16.5 Documentation

- **Base Module Functionality:** axi4_slave_wr.md
- **Clock Gating Guide:** clock_gated_variants.md
- **Detailed CG Examples:**

- axi4_master_rd_mon_cg.md (AXI4 monitor)
- axil4_master_rd_mon_cg.md (AXIL4 monitor)
- apb_slave_cg.md (APB interface)

16.6 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

17 AXI4 Slave Write

Module: `axi4_slave_wr.sv` **Location:** `rtl/amba/axi4/` **Status:** √ Production Ready

17.1 Overview

The AXI4 Slave Write module provides a buffered AXI4 slave write interface with configurable depth skid buffers on all three write channels (AW, W, B). This module serves as an elastic buffer between an AXI4 interconnect and a memory or backend processing element, decoupling timing and providing backpressure handling.

17.1.1 Key Features

- √ **Full AXI4 Write Support:** Complete AW, W, and B channel implementation
- √ **Independent Channel Buffering:** Separate configurable depth buffers for each channel
- √ **Elastic Buffering:** Decouples interconnect and backend timing domains
- √ **Burst Support:** Full burst transaction handling with WLAST tracking
- √ **User Signal Support:** Carries AWUSER, WUSER, and BUSER signals
- √ **Clock Gating Support:** Busy signal for dynamic power management

---

17.2 Module Interface

```
module axi4_slave_wr #(
    parameter int SKID_DEPTH_AW     = 2,
    parameter int SKID_DEPTH_W      = 4,
```

```
    parameter int SKID_DEPTH_B      = 2,
    parameter int AXI_ID_WIDTH      = 8,
    parameter int AXI_ADDR_WIDTH    = 32,
    parameter int AXI_DATA_WIDTH    = 32,
    parameter int AXI_USER_WIDTH    = 1
) (
    // Clock and Reset
    input  logic                        aclk,
    input  logic                        aresetn,

    // Slave AXI Interface (s_axi_*)
    // Write address channel (AW)
    input  logic [AXI_ID_WIDTH-1:0]   s_axi_awid,
    input  logic [AXI_ADDR_WIDTH-1:0] s_axi_awaddr,
    input  logic [7:0]                 s_axi_awlen,
    input  logic [2:0]                 s_axi_awsize,
    input  logic [1:0]                 s_axi_awburst,
    input  logic                       s_axi_awlock,
    input  logic [3:0]                 s_axi_awcache,
    input  logic [2:0]                 s_axi_awprot,
    input  logic [3:0]                 s_axi_awqos,
    input  logic [3:0]                 s_axi_awregion,
    input  logic [AXI_USER_WIDTH-1:0] s_axi_awuser,
    input  logic                       s_axi_awvalid,
    output logic                       s_axi_awready,

    // Write data channel (W)
    input  logic [AXI_DATA_WIDTH-1:0] s_axi_wdata,
    input  logic [AXI_DATA_WIDTH/8-1:0] s_axi_wstrb,
    input  logic                       s_axi_wlast,
    input  logic [AXI_USER_WIDTH-1:0] s_axi_wuser,
    input  logic                       s_axi_wvalid,
    output logic                       s_axi_wready,

    // Write response channel (B)
    output logic [AXI_ID_WIDTH-1:0]   s_axi_bid,
    output logic [1:0]                 s_axi_bresp,
    output logic [AXI_USER_WIDTH-1:0] s_axi_buser,
    output logic                       s_axi_bvalid,
    input  logic                       s_axi_bready,

    // Backend Interface (fub_axi_* - to memory/backend)
    // Write address channel (AW)
    output logic [AXI_ID_WIDTH-1:0]   fub_axi_awid,
    output logic [AXI_ADDR_WIDTH-1:0] fub_axi_awaddr,
    output logic [7:0]                 fub_axi_awlen,
    output logic [2:0]                 fub_axi_awsize,
```

```
    output logic [1:0]                  fub_axi_awburst,
    output logic                        fub_axi_awlock,
    output logic [3:0]                  fub_axi_awcache,
    output logic [2:0]                  fub_axi_awprot,
    output logic [3:0]                  fub_axi_awqos,
    output logic [3:0]                  fub_axi_awregion,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_awuser,
    output logic                        fub_axi_awvalid,
    input  logic                        fub_axi_awready,

    // Write data channel (W)
    output logic [AXI_DATA_WIDTH-1:0] fub_axi_wdata,
    output logic [AXI_DATA_WIDTH/8-1:0] fub_axi_wstrb,
    output logic                        fub_axi_wlast,
    output logic [AXI_USER_WIDTH-1:0] fub_axi_wuser,
    output logic                        fub_axi_wvalid,
    input  logic                        fub_axi_wready,

    // Write response channel (B)
    input  logic [AXI_ID_WIDTH-1:0]   fub_axi_bid,
    input  logic [1:0]                  fub_axi_bresp,
    input  logic [AXI_USER_WIDTH-1:0] fub_axi_buser,
    input  logic                        fub_axi_bvalid,
    output logic                        fub_axi_bready,

    // Status
    output logic                        busy
);
```

17.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	Depth of write address (AW) channel skid buffer
SKID_DEPTH_W	int	4	Depth of write data (W) channel skid buffer
SKID_DEPTH_B	int	2	Depth of write response (B) channel skid buffer
AXI_ID_WIDTH	int	8	Width of transaction ID signals (AWID, BID)
AXI_ADDR_WIDTH	int	32	Width of address bus

Parameter	Type	Default	Description
			(AWADDR)
AXI_DATA_WIDTH	int	32	Width of data bus (WDATA), must be 8, 16, 32, 64, 128, 256, 512, or 1024
AXI_USER_WIDTH	int	1	Width of user-defined signals (AWUSER, WUSER, BUSER)

17.4 Port Groups

17.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	Input	1	AXI clock - all signals sampled on rising edge
aresetn	Input	1	Active-low asynchronous reset

17.4.2 Slave AXI Interface (s_axi_*)

Write Address Channel (AW)

Port	Direction	Width	Description
s_axi_awid	Input	AXI_ID_WIDTH	Write transaction ID
s_axi_awaddr	Input	AXI_ADDR_WIDTH	Write address
s_axi_awlen	Input	8	Burst length (0-255 beats)
s_axi_awsize	Input	3	Burst size (bytes per beat)

Port	Direction	Width	Description
s_axi_awburst	Input	2	Burst type (FIXED, INCR, WRAP)
s_axi_awlock	Input	1	Lock type (atomic access support)
s_axi_awcache	Input	4	Cache attributes
s_axi_awprot	Input	3	Protection attributes
s_axi_awqos	Input	4	Quality of Service identifier
s_axi_awregion	Input	4	Region identifier
s_axi_awuser	Input	AXI_USER_WIDTH	User-defined signal
s_axi_awvalid	Input	1	Write address valid
s_axi_awready	Output	1	Write address ready

Write Data Channel (W)

Port	Direction	Width	Description
s_axi_wdata	Input	AXI_DATA_WIDTH	Write data
s_axi_wstrb	Input	AXI_DATA_WIDTH/8	Write strobes (byte enables)
s_axi_wlast	Input	1	Last beat of burst indicator
s_axi_wuser	Input	AXI_USER_WIDTH	User-defined signal
s_axi_wvalid	Input	1	Write data valid
s_axi_wre	Output	1	Write data ready

Port	Direction	Width	Description
ady			

Write Response Channel (B)

Port	Direction	Width	Description
s_axi_bid	Output	AXI_ID_WIDTH	Response transaction ID
s_axi_bresp	Output	2	Write response (OKAY, EXOKAY, SLVERR, DECERR)
s_axi_buser	Output	AXI_USER_WIDTH	User-defined signal
s_axi_bvalid	Output	1	Write response valid
s_axi_bready	Input	1	Write response ready

17.4.3 Backend Interface (fub_axi_*)

Mirrors the slave interface but in the opposite direction (output on AW/W, input on B).

17.4.4 Status Outputs

Port	Direction	Width	Description
busy	Output	1	Indicates active transactions in buffers (for clock gating)

17.5 Functional Description

17.5.1 Architecture

The AXI4 Slave Write module uses three independent `gaxi_skid_buffer` instances to provide elastic buffering on each write channel:

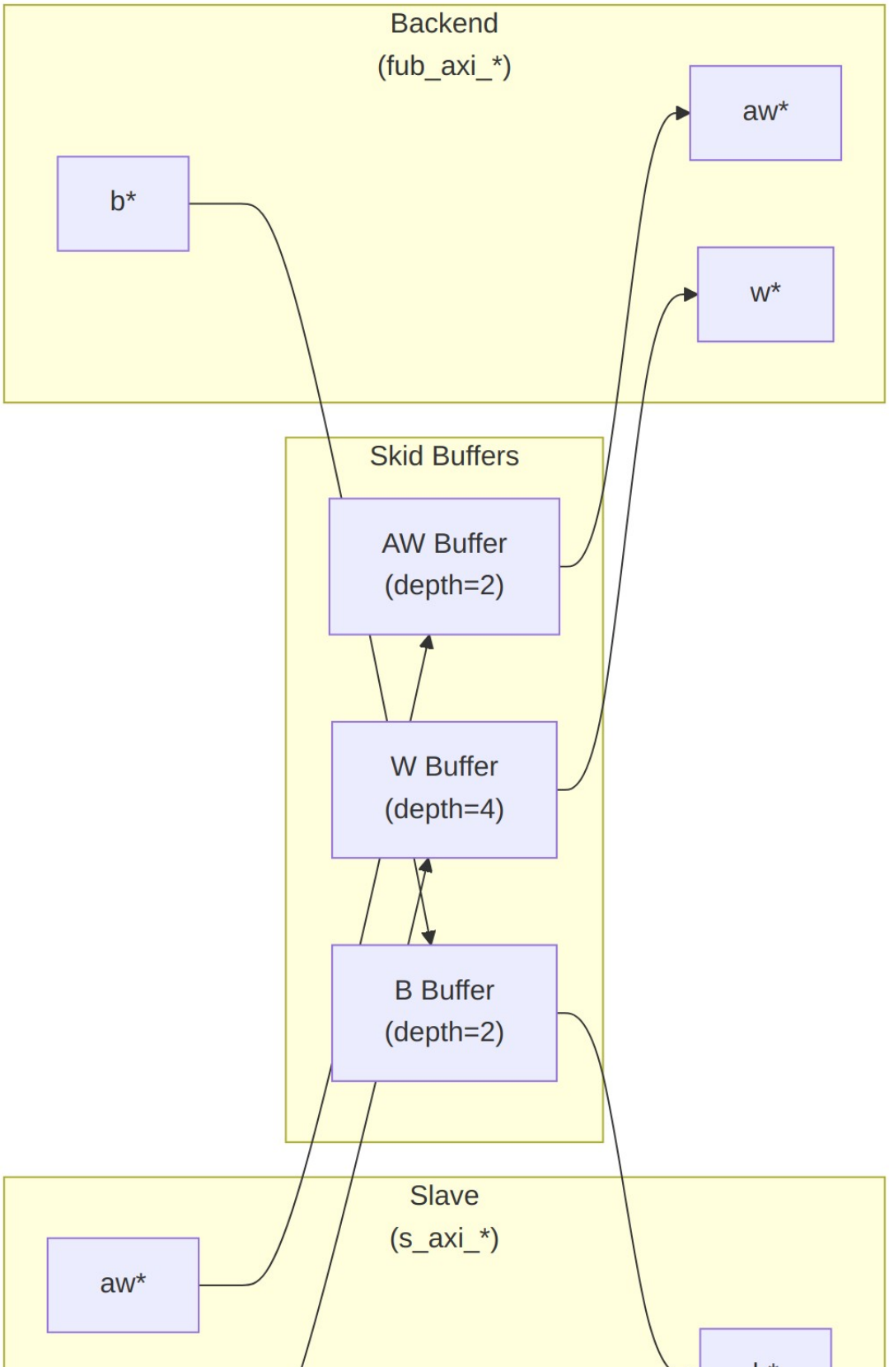

Backend
(fub_axi_*)
aw*
b*
w*
Skid Buffers
AW Buffer
(depth=2)
W Buffer
(depth=4)
B Buffer
(depth=2)
Slave
(s_axi_*)
aw*

Diagram 28

17.5.2 Channel Operations

17.5.2.1 Write Address (AW) Channel

1. Master presents write address and attributes via interconnect
2. AW skid buffer accepts when space available (`s_axi_awready` high)
3. Buffered address presented to backend when ready
4. Configurable depth (`SKID_DEPTH_AW`) smooths timing variations

17.5.2.2 Write Data (W) Channel

1. Master presents write data with strobes via interconnect
2. W skid buffer provides deeper buffering (default depth=4)
3. WLAST signal preserved to indicate burst boundaries
4. Independent from AW channel (AXI4 allows W before AW)

17.5.2.3 Write Response (B) Channel

1. Backend returns write response with transaction ID
2. B skid buffer captures response when master busy
3. Master receives response via interconnect when ready to accept
4. ID matching handled by AXI interconnect (not this module)

17.5.3 Busy Signal

The `busy` output indicates active transactions:

```
busy = (aw_count > 0) || (w_count > 0) || (b_count > 0) ||
       s_axi_awvalid || s_axi_wvalid || fub_axi_bvalid
```

Use cases: - **Clock gating:** Disable clock when `busy` is low - **Power management:** Enter low-power mode when idle - **Synchronization:** Wait for idle before configuration changes

---

17.6 Configuration Guidelines

17.6.1 Buffer Depth Selection

Write Address (SKID_DEPTH_AW): - Default: 2 (sufficient for most cases) - Increase if: - High-latency backend address decode - Frequent address channel backpressure - Multiple outstanding bursts needed

Write Data (SKID_DEPTH_W): - Default: 4 (deeper than AW for burst data) - Increase if: - Large burst sizes (AWLEN > 4) - High-bandwidth streaming writes - Variable backend write latency

Write Response (SKID_DEPTH_B): - Default: 2 (responses are typically single-beat) - Increase if: - Master slow to accept responses - Many concurrent outstanding writes - Response channel backpressure observed

17.6.2 Recommended Configurations

Low-Latency Memory (single outstanding write):

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(2),
    .SKID_DEPTH_B(2)
) u_slave_wr ( ... );
```

High-Throughput Streaming (burst writes):

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(4),
    .SKID_DEPTH_W(16),   // Deep for burst data
    .SKID_DEPTH_B(4)
) u_slave_wr ( ... );
```

Multiple Outstanding Writes:

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(8),
    .SKID_DEPTH_W(16),
    .SKID_DEPTH_B(8)
) u_slave_wr ( ... );
```

17.7 Usage Example

17.7.1 Basic Integration with Memory

```
// Instantiate AXI4 slave write buffer
axi4_slave_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .AXI_USER_WIDTH(1)
) u_axi_slave_wr (
    .aclk               (axi_aclk),
```

```
    .aresetn            (axi_aresetn),

    // Slave interface (from AXI interconnect)
    .s_axi_awid         (s_axi_awid),
    .s_axi_awaddr       (s_axi_awaddr),
    .s_axi_awlen        (s_axi_awlen),
    .s_axi_awsize       (s_axi_awsize),
    .s_axi_awburst      (s_axi_awburst),
    .s_axi_awlock       (s_axi_awlock),
    .s_axi_awcache      (s_axi_awcache),
    .s_axi_awprot       (s_axi_awprot),
    .s_axi_awqos        (s_axi_awqos),
    .s_axi_awregion     (s_axi_awregion),
    .s_axi_awuser       (s_axi_awuser),
    .s_axi_awvalid      (s_axi_awvalid),
    .s_axi_awready      (s_axi_awready),

    .s_axi_wdata        (s_axi_wdata),
    .s_axi_wstrb        (s_axi_wstrb),
    .s_axi_wlast        (s_axi_wlast),
    .s_axi_wuser        (s_axi_wuser),
    .s_axi_wvalid       (s_axi_wvalid),
    .s_axi_wready       (s_axi_wready),

    .s_axi_bid          (s_axi_bid),
    .s_axi_bresp        (s_axi_bresp),
    .s_axi_buser        (s_axi_buser),
    .s_axi_bvalid       (s_axi_bvalid),
    .s_axi_bready       (s_axi_bready),

    // Backend interface (to memory controller)
    .fub_axi_awid       (mem_awid),
    .fub_axi_awaddr     (mem_awaddr),
    .fub_axi_awlen      (mem_awlen),
    .fub_axi_awsize     (mem_awsize),
    .fub_axi_awburst    (mem_awburst),
    .fub_axi_awlock     (mem_awlock),
    .fub_axi_awcache    (mem_awcache),
    .fub_axi_awprot     (mem_awprot),
    .fub_axi_awqos      (mem_awqos),
    .fub_axi_awregion   (mem_awregion),
    .fub_axi_awuser     (mem_awuser),
    .fub_axi_awvalid    (mem_awvalid),
    .fub_axi_awready    (mem_awready),

    .fub_axi_wdata      (mem_wdata),
    .fub_axi_wstrb      (mem_wstrb),
```

```
    .fub_axi_wlast      (mem_wlast),
    .fub_axi_wuser      (mem_wuser),
    .fub_axi_wvalid     (mem_wvalid),
    .fub_axi_wready     (mem_wready),

    .fub_axi_bid        (mem_bid),
    .fub_axi_bresp      (mem_bresp),
    .fub_axi_buser      (mem_buser),
    .fub_axi_bvalid     (mem_bvalid),
    .fub_axi_bready     (mem_bready),

    // Status
    .busy               (wr_slave_busy)
);

// Memory controller backend
memory_controller u_mem_ctrl (
    .axi_aclk       (axi_aclk),
    .axi_aresetn    (axi_aresetn),
    // Connect to fub_axi_* signals
    .axi_awid       (mem_awid),
    .axi_awaddr     (mem_awaddr),
    // ... rest of signals
);
```

17.7.2 Clock Gating Example

```
// Use busy signal for clock gating
logic axi_clk_gated;

clock_gate_ctrl u_cg (
    .i_clk          (axi_clk),
    .i_enable       (wr_slave_busy),
    .o_clk_gated    (axi_clk_gated)
);

// Connect module to gated clock
axi4_slave_wr #( ... ) u_slave_wr (
    .aclk (axi_clk_gated),
    ...
);
```

17.8 Design Notes

17.8.1 Buffer Independence

The three skid buffers operate independently: - AW and W channels can progress at different rates - AXI4 allows W data before AW address - B responses return asynchronously based on backend timing

17.8.2 Packet Preservation

All signal groups packed and unpacked atomically: - AW channel: `{awid, awaddr, awlen, awsize, awburst, awlock, awcache, awprot, awqos, awregion, awuser}` - W channel: `{wdata, wstrb, wlast, wuser}` - B channel: `{bid, bresp, buser}`

17.8.3 Backpressure Handling

Ready signals propagate backpressure: - Interconnect sees `awready/wready` low when buffers full - Backend backpressure captured in buffers - Prevents data loss while decoupling timing

17.8.4 ID and Burst Handling

This module is a pass-through buffer: - Does NOT track transaction IDs - Does NOT enforce burst ordering - Relies on backend to: - Match BID to AWID - Generate appropriate BRESP

17.8.5 Reset Behavior

On `aresetn` assertion (active-low): - All skid buffers flush - Valid signals deasserted - Busy signal goes low - No data retained across reset

---

17.9 Related Modules

17.9.1 Companion Modules

- **axi4_slave_rd** - AXI4 slave read with buffering
- **axi4_slave_wr_cg** - Clock-gated variant with additional CG logic
- **axi4_slave_wr_mon** - Write transaction monitor for verification

17.9.2 Used Components

- **gaxi_skid_buffer** - Elastic buffer with valid/ready handshake
- **clock_gate_ctrl** - Clock gating control (example)

17.9.3 Related Infrastructure

- **axi4_master_wr** - Corresponding AXI4 write master module

- **axi4_interconnect** - Multi-master/multi-slave crossbar

17.10 References

17.10.1 Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)

17.10.2 Source Code

- RTL: `rtl/amba/axi4/axi4_slave_wr.sv`
- Tests: `val/amba/test_axi4_slave_wr.py`
- Framework: `bin/TBClasses/components/axi4/`

17.10.3 Documentation

- Architecture: RTLAmba Overview
- AXI4 Index: axi4/README.md
- GAXI Buffers: gaxi/README.md

Last Updated: 2025-10-20

17.11 Navigation

- **← Back to AXI4 Index**
- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**

18 AXI4 Slave Write Stub

Module: `axi4_slave_wr_stub.sv` **Location:** `rtl/amba/axi4/stubs/` **Status:** Production Ready

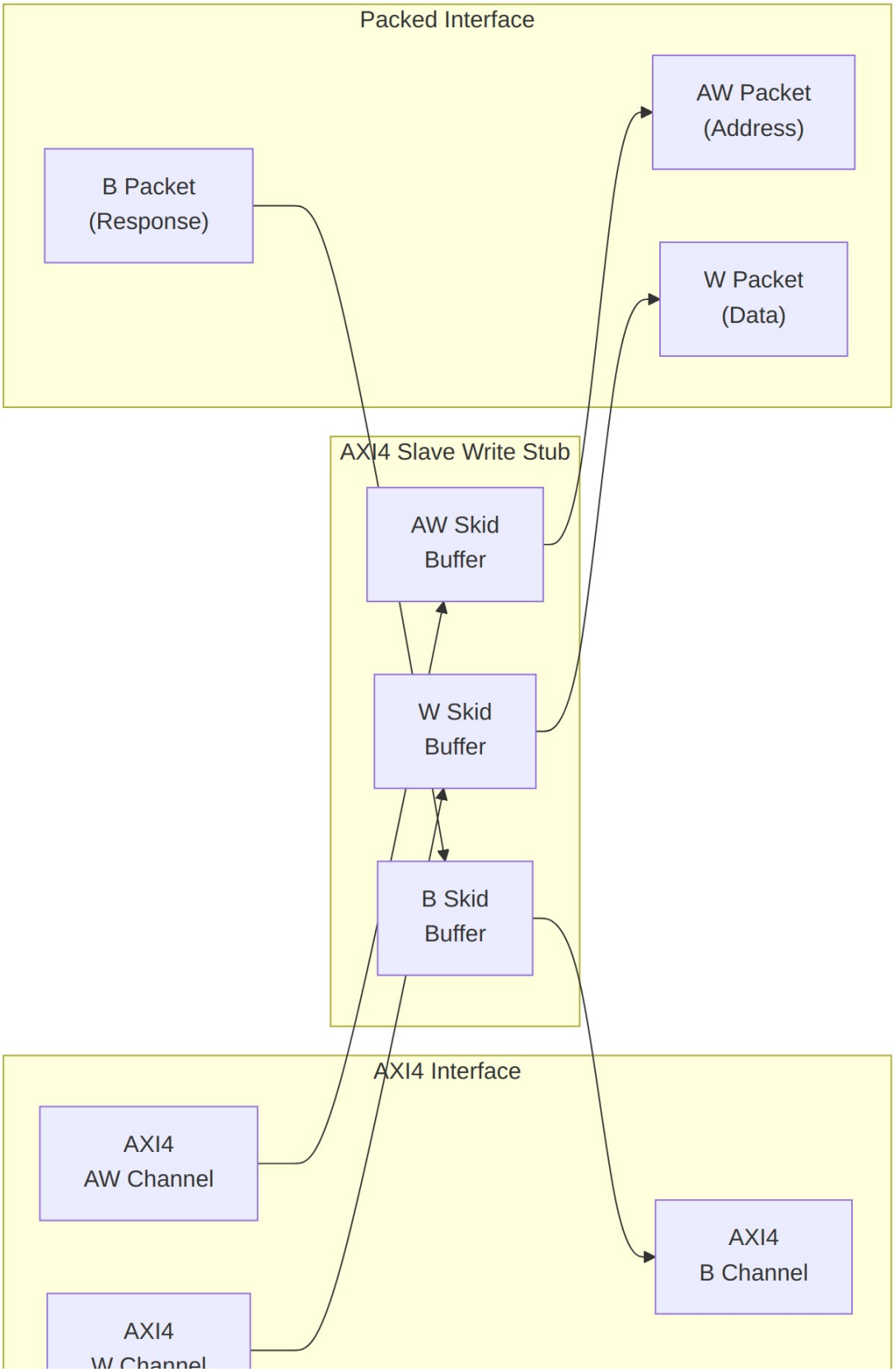
18.1 Overview

The AXI4 Slave Write Stub provides a simplified packed-data interface for receiving AXI4 write transactions from a master. It uses skid buffers to pack/unpack AXI4 AW (write address), W (write

data), and B (write response) channels into simple packet interfaces, making it ideal for testbenches and integration scenarios where a simplified slave interface is needed.

18.1.1 Key Features

- Packed packet interface for AW, W, and B channels
- Configurable skid buffer depths for each channel
- Full AXI4 write transaction support
- Burst, ID, strobe, user signal support
- Parameterized data widths


Packed Interface
AW Packet
(Address)
B Packet
(Response)
W Packet
(Data)
AXI4 Slave Write Stub
AW Skid
Buffer
W Skid
Buffer
B Skid
Buffer
AXI4 Interface
AXI4
AW Channel
AXI4
B Channel
AXI4

18.2 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW channel skid buffer depth (log2)
SKID_DEPTH_W	int	4	W channel skid buffer depth (log2)
SKID_DEPTH_B	int	2	B channel skid buffer depth (log2)
AXI_ID_WIDTH	int	8	AXI transaction ID width
AXI_ADDR_WIDTH	int	32	AXI address bus width
AXI_DATA_WIDTH	int	32	AXI data bus width
AXI_USER_WIDTH	int	1	AXI user signal width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width
AW	int	AXI_ADDR_WIDTH	Short alias for address width
DW	int	AXI_DATA_WIDTH	Short alias for data width
IW	int	AXI_ID_WIDTH	Short alias for ID width
SW	int	AXI_WSTRB_WIDTH	Short alias for strobe width
UW	int	AXI_USER_WIDTH	Short alias for user width
AWSize	int	IW+AW+8+3+2+1+4+3+4+4+UW	AW packet size (calculated)
WSize	int	DW+SW+1+UW	W packet size (calculated)
BSize	int	IW+2+UW	B packet size (calculated)

18.3 Ports

18.3.1 Clock and Reset

Port	Width	Direction	Description
aclk	1	Input	AXI clock
aresetn	1	Input	AXI reset (active low)

18.3.2 AXI4 Write Address Channel (AW)

Port	Width	Direction	Description
s_axi_awid	IW	Input	Write address ID
s_axi_awaddr	AW	Input	Write address
s_axi_awlen	8	Input	Burst length
s_axi_awsize	3	Input	Burst size
s_axi_awburst	2	Input	Burst type
s_axi_awlock	1	Input	Lock type
s_axi_awcache	4	Input	Cache type
s_axi_awprot	3	Input	Protection type
s_axi_awqos	4	Input	Quality of service
s_axi_awregion	4	Input	Region identifier
s_axi_awuser	UW	Input	User signal
s_axi_awvalid	1	Input	Write address valid
s_axi_awready	1	Output	Write address ready

18.3.3 AXI4 Write Data Channel (W)

Port	Width	Direction	Description
s_axi_wdata	DW	Input	Write data
s_axi_wstrb	SW	Input	Write strobes
s_axi_wlast	1	Input	Write last

Port	Width	Direction	Description
s_axi_wuser	UW	Input	User signal
s_axi_wvalid	1	Input	Write data valid
s_axi_wready	1	Output	Write data ready

18.3.4 AXI4 Write Response Channel (B)

Port	Width	Direction	Description
s_axi_bid	IW	Output	Response ID
s_axi_bresp	2	Output	Write response
s_axi_buser	UW	Output	User signal
s_axi_bvalid	1	Output	Write response valid
s_axi_bready	1	Input	Write response ready

18.3.5 AW Packet Interface

Port	Width	Direction	Description
fub_axi_awvalid	1	Output	AW packet valid
fub_axi_awready	1	Input	Ready to accept AW packet
fub_axi_aw_count	4	Output	AW buffer occupancy
fub_axi_aw_pkt	AWSize	Output	Packed AW packet data

18.3.6 W Packet Interface

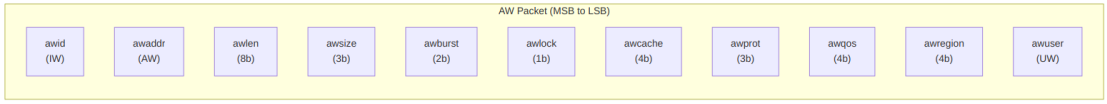
Port	Width	Direction	Description
fub_axi_wvalid	1	Output	W packet valid
fub_axi_wready	1	Input	Ready to accept W packet

Port	Width	Direction	Description
fub_axi_w_pkt	WSize	Output	Packed W packet data

18.3.7 B Packet Interface

Port	Width	Direction	Description
fub_axi_bvalid	1	Input	B packet valid
fub_axi_bready	1	Output	Ready to accept B packet
fub_axi_b_pkt	BSize	Input	Packed B packet data

18.4 Packet Formats

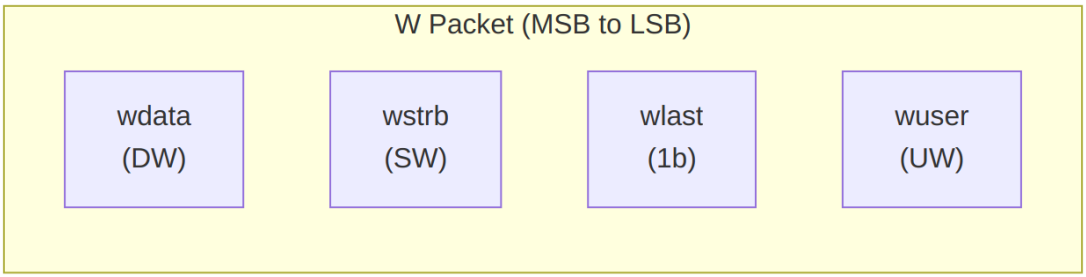


AW Packet Structure (Write Address)

Bit Positions:

```
fub_axi_aw_pkt = {awid, awaddr, awlen, awsize, awburst, awlock,
awcache, awprot, awqos, awregion, awuser}

Width = IW + AW + 8 + 3 + 2 + 1 + 4 + 3 + 4 + 4 + UW
```

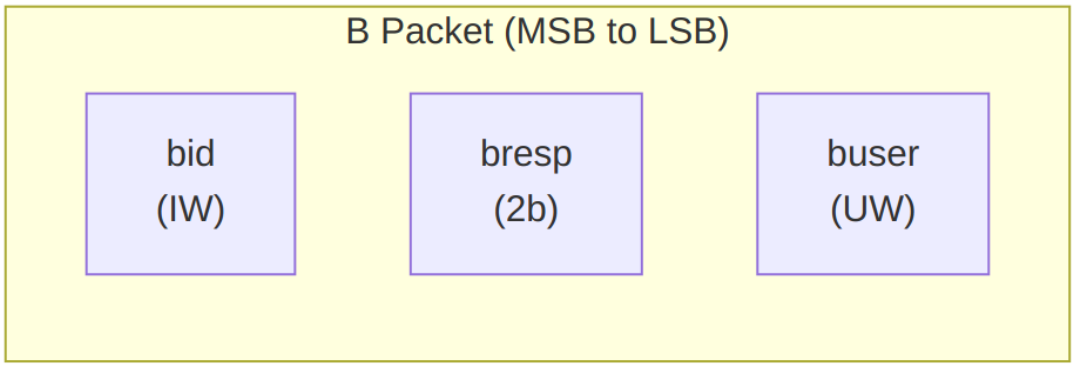


W Packet Structure (Write Data)

Bit Positions:

```
fub_axi_w_pkt = {wdata, wstrb, wlast, wuser}

Width = DW + SW + 1 + UW
```



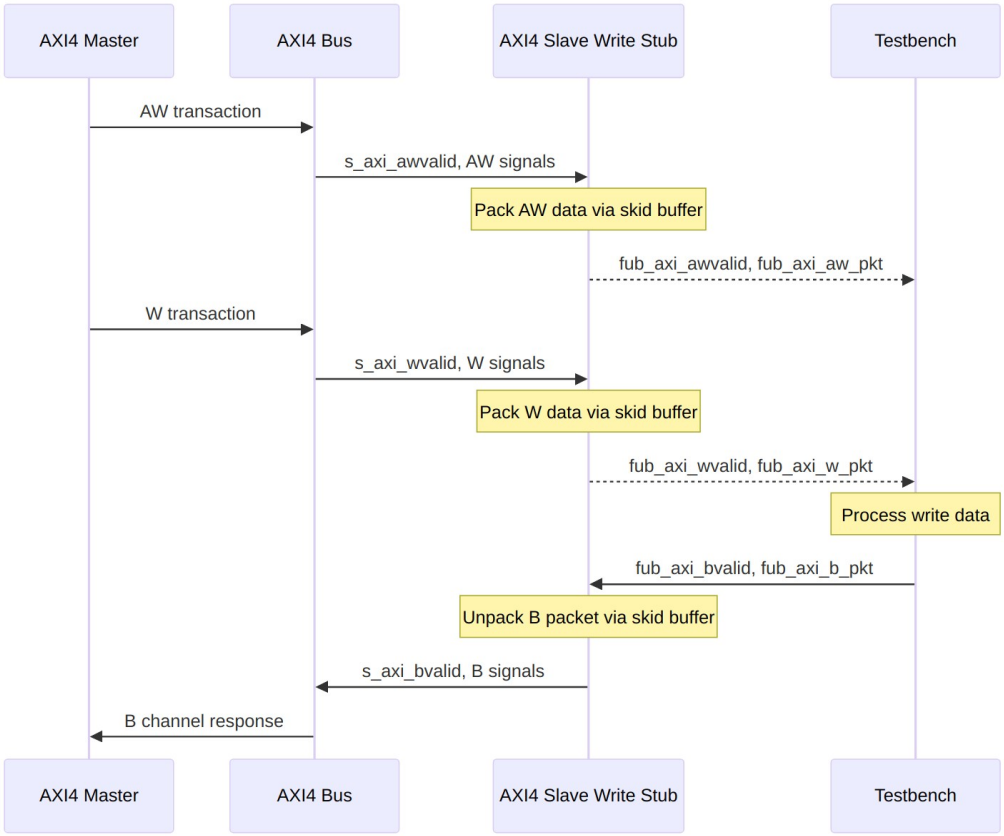
B Packet Structure (Write Response)

Bit Positions:

```
fub_axi_b_pkt = {bid, bresp, buser}

Width = IW + 2 + UW
```

18.5 Transaction Flow



Write Transaction

18.5.1 Timing

```
TODO: Wavedrom timing diagram showing:
- aclk
- AXI AW signals (s_axi_awvalid, s_axi_awaddr, s_axi_awlen, etc.)
- fub_axi_awvalid, fub_axi_awready, fub_axi_aw_pkt
- AXI W signals (s_axi_wvalid, s_axi_wdata, s_axi_wstrb, s_axi_wlast,
etc.)
- fub_axi_wvalid, fub_axi_wready, fub_axi_w_pkt
- fub_axi_bvalid, fub_axi_bready, fub_axi_b_pkt
- AXI B signals (s_axi_bvalid, s_axi_bid, s_axi_bresp, etc.)
- Packet-to-AXI timing relationship with skid buffer operation
```

18.6 Usage Example

```
axi4_slave_wr_stub #(
    .SKID_DEPTH_AW   (2),
```

```
    .SKID_DEPTH_W    (4),
    .SKID_DEPTH_B    (2),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .AXI_USER_WIDTH  (4)
) u_axi4_slave_wr_stub (
    .aclk            (axi_clk),
    .aresetn         (axi_rst_n),

    // AXI4 slave write interface
    .s_axi_awid      (s_axi_awid),
    .s_axi_awaddr    (s_axi_awaddr),
    .s_axi_awlen     (s_axi_awlen),
    .s_axi_awsize    (s_axi_awsize),
    .s_axi_awburst   (s_axi_awburst),
    .s_axi_awlock    (s_axi_awlock),
    .s_axi_awcache   (s_axi_awcache),
    .s_axi_awprot    (s_axi_awprot),
    .s_axi_awqos     (s_axi_awqos),
    .s_axi_awregion  (s_axi_awregion),
    .s_axi_awuser    (s_axi_awuser),
    .s_axi_awvalid   (s_axi_awvalid),
    .s_axi_awready   (s_axi_awready),

    .s_axi_wdata     (s_axi_wdata),
    .s_axi_wstrb     (s_axi_wstrb),
    .s_axi_wlast     (s_axi_wlast),
    .s_axi_wuser     (s_axi_wuser),
    .s_axi_wvalid    (s_axi_wvalid),
    .s_axi_wready    (s_axi_wready),

    .s_axi_bid       (s_axi_bid),
    .s_axi_bresp     (s_axi_bresp),
    .s_axi_buser     (s_axi_buser),
    .s_axi_bvalid    (s_axi_bvalid),
    .s_axi_bready    (s_axi_bready),

    // Packed AW interface
    .fub_axi_awvalid (tb_aw_valid),
    .fub_axi_awready (tb_aw_ready),
    .fub_axi_aw_count(tb_aw_count),
    .fub_axi_aw_pkt  (tb_aw_pkt),

    // Packed W interface
    .fub_axi_wvalid  (tb_w_valid),
    .fub_axi_wready  (tb_w_ready),
```

```
    .fub_axi_w_pkt   (tb_w_pkt),

    // Packed B interface
    .fub_axi_bvalid  (tb_b_valid),
    .fub_axi_bready  (tb_b_ready),
    .fub_axi_b_pkt   (tb_b_pkt)
);

// Parse AW packet
wire [7:0]  aw_id     = tb_aw_pkt[AWSize-1:AWSize-8];
wire [31:0] aw_addr   = tb_aw_pkt[AWSize-9:AWSize-40];
wire [7:0]  aw_len    = tb_aw_pkt[AWSize-41:AWSize-48];
wire [2:0]  aw_size   = tb_aw_pkt[AWSize-49:AWSize-51];
wire [1:0]  aw_burst  = tb_aw_pkt[AWSize-52:AWSize-53];
// ... additional fields as needed

// Parse W packet
wire [63:0] w_data = tb_w_pkt[WSize-1:WSize-64];
wire [7:0]  w_strb = tb_w_pkt[WSize-65:WSize-72];
wire        w_last = tb_w_pkt[WSize-73];
wire [3:0]  w_user = tb_w_pkt[3:0];

// Build B packet (OKAY response)
localparam BSize = 8 + 2 + 4;  // Calculate size
assign tb_b_pkt = {
    aw_id,    // bid (match request ID)
    2'b00,    // bresp (OKAY)
    4'h0      // buser
};
```

18.7 Design Notes

18.7.1 Skid Buffer Operation

The stub uses `gaxi_skid_buffer` modules to: - Decouple timing between AXI bus and testbench - Provide configurable buffering depth per channel - Handle backpressure gracefully - Support burst transactions without stalling

Recommended Depths: - **AW Channel:** 2-4 (address transactions) - **W Channel:** 4-8 (data beats for bursts) - **B Channel:** 2-4 (responses)

18.7.2 Channel Independence

The AW, W, and B channels are independent: - AW and W arrive in any order from master - Stub handles proper AXI timing - B responses generated asynchronously by testbench

18.7.3 Packet Packing Order

AW, W, and B packets are packed MSB-to-LSB following AXI signal order: - Simplifies testbench packet parsing - Matches common concatenation order - Efficient for burst transaction handling

18.7.4 Internal Architecture

The stub instantiates three `gaxi_skid_buffer` modules: - **AW Skid Buffer:** Packs AXI AW channel to AW packets - **W Skid Buffer:** Packs AXI W channel to W packets - **B Skid Buffer:** Unpacks B packets to AXI B channel

All AXI protocol handling is done by the skid buffers and upstream modules.

18.8 Related Documentation

- **AXI4 Slave Write** - Full AXI4 slave write module (if wrapping one)
- **AXI4 Slave Read Stub** - Corresponding read stub
- **AXI4 Slave Stub** - Combined read/write stub
- **AXI4 Master Write Stub** - Master-side write stub

18.9 Navigation

- **<- Back to AXI4 Index**
- **<- Back to RTLAmba Index**
- **<- Back to Main Documentation Index**

19 AXI4 (Advanced eXtensible Interface) Modules

Location: `rtl/amba/axi4/` **Test Location:** `val/amba/` **Status:** √ Production Ready

---

19.1 Overview

The AXI4 subsystem provides a complete implementation of the ARM AMBA AXI4 protocol, including masters, slaves, monitors, data width converters, and interconnect components. AXI4 is a high-performance, high-frequency protocol designed for multi-master, multi-slave systems with out-of-order transaction support.

19.1.1 Key Features

- ✓ **Full AXI4 Protocol Support:** Complete implementation of read and write channels
- ✓ **Burst Transactions:** Support for fixed, incrementing, and wrapping bursts
- ✓ **Out-of-Order Support:** ID-based transaction tracking and completion
- ✓ **Quality of Service (QoS):** Priority-based arbitration support
- ✓ **Monitoring Infrastructure:** Comprehensive verification and debug capabilities
- ✓ **Data Width Conversion:** Automatic upsizing and downsizing
- ✓ **Clock Gating Variants:** Power-optimized versions with dynamic gating

---

19.2 Module Categories

19.2.1 Core Master/Slave Modules

Module	Description	Documentation	Status
axi4_master_rd	AXI4 read master with buffered channels	axi4_master_rd.md	✓ Documented
axi4_master_wr	AXI4 write master with buffered channels	axi4_master_wr.md	✓ Documented
axi4_slave_rd	AXI4 read slave with buffered channels	axi4_slave_rd.md	✓ Documented
axi4_slave_wr	AXI4 write slave with buffered channels	axi4_slave_wr.md	✓ Documented

19.2.2 Monitor Modules (Verification)

Module	Description	Documentation	Status
axi4_master_rd_mon	Read master with integrated monitoring	axi4_master_rd_mon.md	✓ Documented
axi4_master_wr_mon	Write master with integrated monitoring	axi4_master_wr_mon.md	✓ Documented
axi4_slave_	Read slave with	axi4_slave_rd_mon.md	✓

Module	Description	Documentation	Status
rd_mon	integrated monitoring		Documented
axi4_slave_wr_mon	Write slave with integrated monitoring	axi4_slave_wr_mon.md	✓ Documented

19.2.3 Data Width Converters

Module	Description	Documentation	Status
axi4_dwidth_converter	Bidirectional data width conversion	axi4_dwidth_converter.md	✓ Documented
axi4_dwidth_converter_rd	Read-only data width conversion	axi4_dwidth_converter_rd.md	✓ Documented
axi4_dwidth_converter_wr	Write-only data width conversion	axi4_dwidth_converter_wr.md	✓ Documented

19.2.4 Clock-Gated Variants

All clock-gated variants documented in: axi4_clock_gating_guide.md

Module	Base Module	Status
axi4_master_rd_cg	axi4_master_rd	✓ Documented
axi4_master_wr_cg	axi4_master_wr	✓ Documented
axi4_slave_rd_cg	axi4_slave_rd	✓ Documented
axi4_slave_wr_cg	axi4_slave_wr	✓ Documented
axi4_master_rd_mon_cg	axi4_master_rd_mon	✓ Documented
axi4_master_wr_mon_cg	axi4_master_wr_mon	✓ Documented
axi4_slave_rd_mon_cg	axi4_slave_rd_mon	✓ Documented
axi4_slave_wr_mon_cg	axi4_slave_wr_mon	✓ Documented

19.3 AXI4 Protocol Overview

19.3.1 Channel Architecture

AXI4 uses five independent channels for read and write transactions:

Write Transaction: 1. **AW (Write Address)** - Master → Slave: Address and control 2. **W (Write Data)** - Master → Slave: Data and strobes 3. **B (Write Response)** - Slave → Master: Completion status

Read Transaction: 1. **AR (Read Address)** - Master → Slave: Address and control 2. **R (Read Data)** - Slave → Master: Data and response

19.3.2 Key Signals

Address Channels (AR/AW): - `ARID/AWID` - Transaction ID for reordering - `ARADDR/AWADDR` - Byte address - `ARLEN/AWLEN` - Burst length (1-256 beats) - `ARSIZE/AWSIZE` - Transfer size (1, 2, 4, 8, 16, ... 128 bytes) - `ARBURST/AWBURST` - Burst type (FIXED, INCR, WRAP) - `ARCACHE/AWCACHE` - Cache attributes - `ARPROT/AWPROT` - Protection attributes - `ARQOS/AWQOS` - Quality of Service

Data Channels (R/W): - `RID/WID` - Transaction ID (RID only in AXI4) - `RDATA/WDATA` - Transfer data - `RRESP/BRESP` - Response (OKAY, EXOKAY, SLVERR, DECERR) - `RLAST/WLAST` - Last transfer in burst - `WSTRB` - Write strobes (byte lane enables)

19.4 Quick Start

19.4.1 Basic Read Master

```
axi4_master_rd #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64)
) u_rd_master (
    .aclk           (axi_clk),
    .aresetn        (axi_resetn),

    // Frontend interface
    .fub_axi_arid     (cpu_arid),
    .fub_axi_araddr   (cpu_araddr),
    .fub_axi_arlen    (cpu_arlen),
    // ... rest of AR/R signals

    // Master interface
    .m_axi_arid       (m_axi_arid),
```

```
    .m_axi_araddr     (m_axi_araddr),
    // ... rest of AR/R signals

    .busy             (rd_busy)
);
```

19.4.2 Basic Write Slave

```
axi4_slave_wr #(
    .SKID_DEPTH_AW(2),
    .SKID_DEPTH_W(4),
    .SKID_DEPTH_B(2),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64)
) u_wr_slave (
    .aclk             (axi_clk),
    .aresetn          (axi_resetn),

    // Slave interface (from interconnect)
    .s_axi_awid       (s_axi_awid),
    .s_axi_awaddr     (s_axi_awaddr),
    // ... rest of AW/W/B signals

    // Backend interface (to memory/logic)
    .fub_axi_awid     (mem_awid),
    .fub_axi_awaddr   (mem_awaddr),
    // ... rest of AW/W/B signals

    .busy             (wr_busy)
);
```

19.4.3 Monitor Integration

```
axi4_master_rd_mon #(
    .SKID_DEPTH_AR(2),
    .SKID_DEPTH_R(4),
    .AXI_ID_WIDTH(4),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),
    .UNIT_ID(1),
    .AGENT_ID(10),
    .MAX_TRANSACTIONS(16),
    .ENABLE_FILTERING(1)
) u_rd_mon (
    .aclk               (axi_clk),
    .aresetn            (axi_resetn),
```

```
    // AXI interfaces
    .fub_axi_*(frontend_axi_*),
    .m_axi_*(master_axi_*),

    // Monitor configuration
    .cfg_monitor_enable     (1'b1),
    .cfg_error_enable       (1'b1),
    .cfg_timeout_enable     (1'b1),
    .cfg_perf_enable        (1'b0),
    .cfg_timeout_cycles     (16'd1000),

    // Filtering configuration
    .cfg_axi_pkt_mask       (16'b1111_1111_0000_0011),
    .cfg_axi_error_mask     (16'h0000),
    // ... rest of mask signals

    // Monitor bus output
    .monbus_valid           (mon_valid),
    .monbus_ready           (mon_ready),
    .monbus_packet          (mon_packet),

    // Status
    .busy                   (rd_busy),
    .active_transactions    (rd_active),
    .error_count            (rd_errors)
);
```

19.5 Testing

All AXI4 modules are verified using CocoTB-based testbenches:

```
# Run all AXI4 tests
pytest val/amba/test_axi4*.py -v

# Run specific module tests
pytest val/amba/test_axi4_master_rd.py -v
pytest val/amba/test_axi4_slave_wr.py -v
pytest val/amba/test_axi4_master_rd_mon.py -v

# Run data width converter tests
pytest val/amba/test_axi4_dwidth_converter*.py -v

# Run with waveforms
pytest val/amba/test_axi4_master_rd.py --vcd=waves.vcd -v
```

19.6 Design Patterns

19.6.1 Pattern 1: Buffered Master/Slave

All core modules use `gaxi_skid_buffer` for elastic buffering: - Decouples frontend and backend timing - Configurable depth per channel - Minimal latency overhead (1 cycle) - Full backpressure handling

19.6.2 Pattern 2: Monitor Integration

Monitor modules combine functional core with verification: - Non-invasive monitoring (tap signals) - 3-level filtering hierarchy - 128-bit standardized monitor bus + 64-bit side-band timestamp - Real-time error detection

19.6.3 Pattern 3: Clock Gating

Clock-gated variants (`*_cg`) add power management: - Dynamic clock gating based on busy signal - Test mode support for scan insertion - Minimal area overhead

19.7 Performance Characteristics

19.7.1 Throughput

Configuration	Address CH	Data CH	Notes
Single transaction	1 txn/cycle	1 beat/cycle	When buffers not stalled
Burst (len=16)	1 txn/16 cycles	1 beat/cycle	Sustained
Multiple outstanding	Up to MAX_TRANS	1 beat/cycle	Out-of-order

19.7.2 Latency

Path	Cycles	Notes
AR → R (single)	2-3	Minimum read latency
AW → B (single)	2-3	Minimum write latency
Buffer overhead	+1 per channel	Skid buffer latency

19.7.3 Resource Usage

Module Type	LUTs	FFs	BRAM	Notes
Master/ Slave (32-bit)	~500	~600	0	Per direction (rd/wr)
Monitor (+mon)	+1000	+800	0	Additional overhead
Clock-gated (+cg)	+50	+30	0	Clock gating logic

19.8 Related Documentation

19.8.1 Protocol Specifications

- ARM IHI 0022E: AMBA AXI Protocol Specification (AXI4)
- ARM IHI 0022D: AMBA AXI and ACE Protocol Specification (AXI3/AXI4/AXI5)

19.8.2 RTL Design Sherpa Documentation

- **RTLAmba Overview** - Complete AMBA subsystem architecture
- **APB Modules** - Advanced Peripheral Bus
- **AXIL4 Modules** - AXI4-Lite (simplified protocol)
- **AXIS4 Modules** - AXI4-Stream (streaming data)
- **GAXI Modules** - Generic AXI utilities (buffers, FIFOs)

19.8.3 Configuration and Integration

- **AXI Monitor Configuration Guide** - Monitor setup strategies
- **Monitor Packet Specification** - 64-bit packet format

19.8.4 Source Code

- RTL: `rtl/amba/axi4/`
- Tests: `val/amba/test_axi4*.py`
- Framework: `bin/TBClasses/components/axi4/`
- Shared Infrastructure: `rtl/amba/shared/` (monitor base, transaction manager)

19.9 Design Notes

19.9.1 ID Width Selection

Choose ID width based on system requirements: - **4 bits (16 IDs):** Single master systems, simple slaves - **8 bits (256 IDs):** Multi-master systems, reordering support - **12+ bits:** Large systems, extensive out-of-order capability

19.9.2 Buffer Depth Guidelines

Address Channels (AR/AW): - Default: 2 (4 entries) - sufficient for most cases - Increase for high-latency address decode or frequent backpressure

Data Channels (R/W): - Default: 4 (16 entries) - accommodates moderate bursts - Increase for large bursts (AWLEN/ARLEN > 8) - Deep buffers for variable latency backends

Response Channel (B): - Default: 2 (4 entries) - responses are single-beat - Increase for many concurrent outstanding writes

19.9.3 Burst Optimization

Optimize burst parameters for efficiency:

```
// Cache line burst (64 bytes, 8x 64-bit)
.ARLEN    = 8'd7,          // 8 beats
.ARSIZE   = 3'b011,        // 8 bytes per beat
.ARBURST  = 2'b01          // INCR
```

Last Updated: 2025-10-20

19.10 Navigation

- **← Back to RTLAmba Index**
- **← Back to Main Documentation Index**